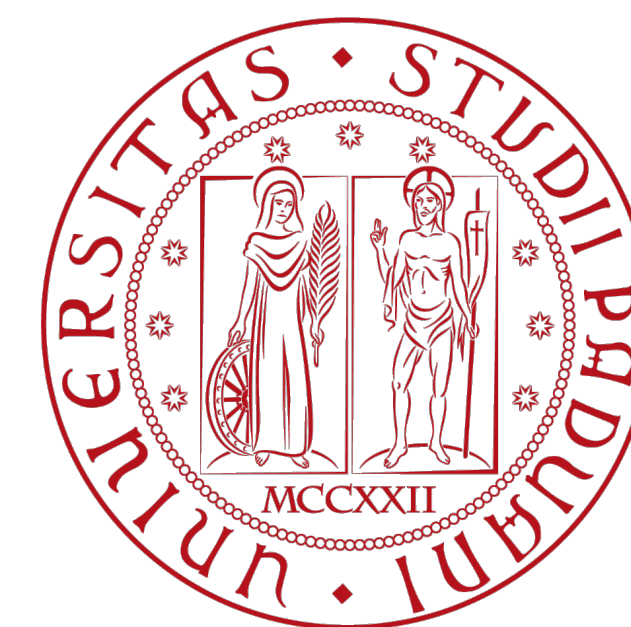
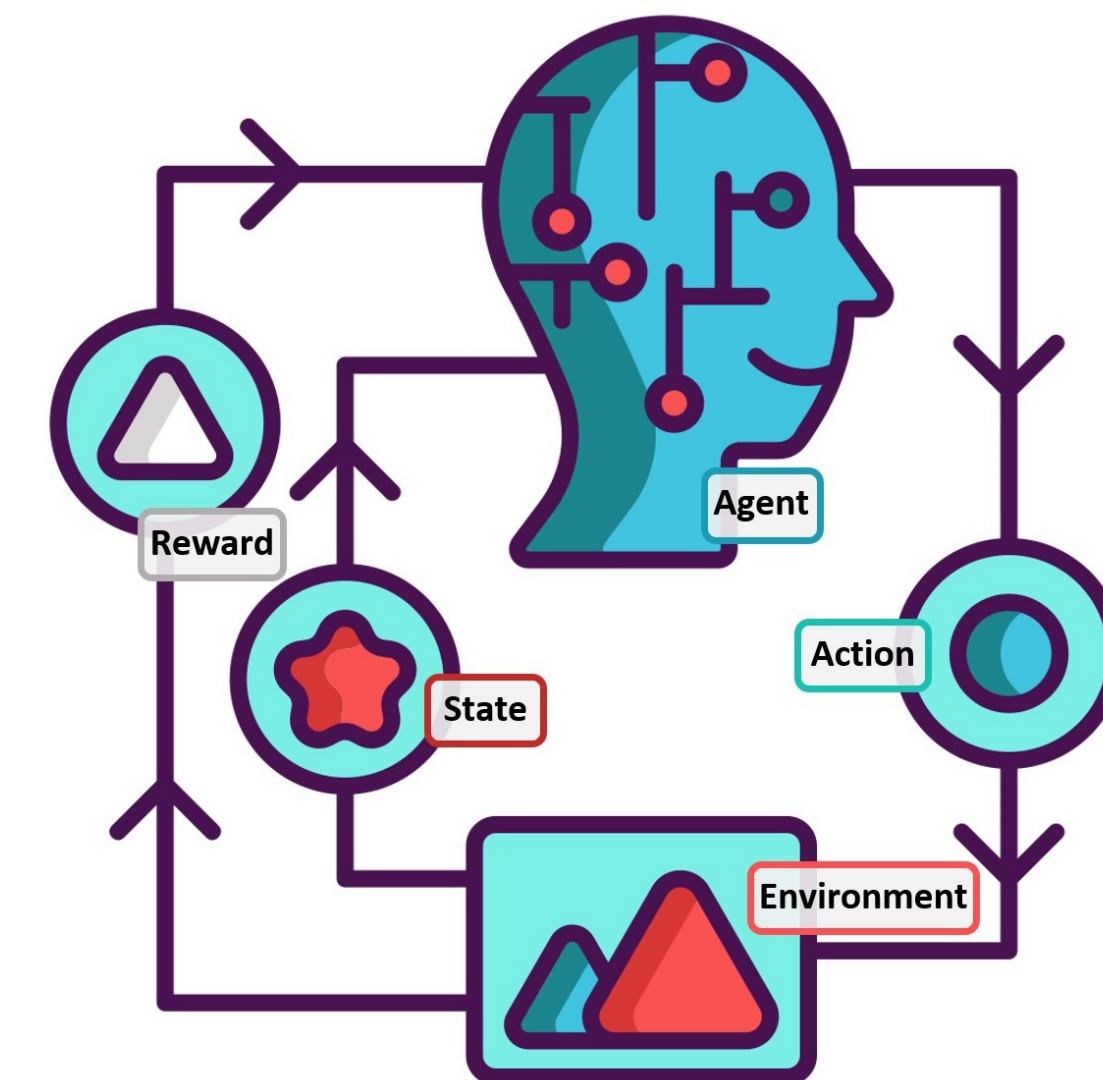


Reinforcement Learning

Intro to Deep Learning #03: Convolutional Neural Networks

Gian Antonio Susto
Ruggero Carli



Reference Material (used for this presentation):

- MIT *Introduction to Deep Learning* <http://introtodeeplearning.com>
- L. Fridman MIT Deep Learning <https://deeplearning.mit.edu/>
- I. Goodfellow, Y. Bengio, A. Courville. *Deep learning*. MIT press, 2016.
- ImageNET <http://www.image-net.org/>
- C. Szegedy et al. *Going Deeper with Convolutions* CVPR2015
- F. Li, J. Johnson, S. Yeung. *Stanford CS231n Convolutional Neural Networks for Visual Recognition* <http://cs231n.stanford.edu/> + <http://cs231n.github.io/convolutional-networks/>
- Y. LeCun <http://yann.lecun.com/exdb/lenet/>
- <https://www.kaggle.com/zalando-research/fashionmnist>
- MIT <https://groups.csail.mit.edu/vision/TinyImages/>
- ImageNET <http://www.image-net.org/>
- <https://github.com/GoogleCloudPlatform/tensorflow-without-a-phd>
- Krizhevsky, Alex, Ilya Sutskever, and Geoffrey E. Hinton. *Imagenet classification with deep convolutional neural networks*. Advances in neural information processing systems. 2012.
- C. Szegedy et al. *Going Deeper with Convolutions* CVPR2015
- Lee, Honglak, et al. "Convolutional deep belief networks for scalable unsupervised learning of hierarchical representations." Proceedings of the 26th annual international conference on machine learning. ACM, 2009
- Razavian et al, "CNN Features Off-the-Shelf: An Astounding Baseline for Recognition", CVPR Workshops 2014
- Lee, Honglak, et al. "Convolutional deep belief networks for scalable unsupervised learning of hierarchical representations." Proceedings of the 26th annual international conference on machine learning. ACM, 2009
- Razavian et al, "CNN Features Off-the-Shelf: An Astounding Baseline for Recognition", CVPR Workshops 2014
- The complete history of Lenna <http://www.ee.cityu.edu.hk/~lmpo/lenna/Lenna97.html>
- Style transfer <https://deepart.io/>
- <https://paperswithcode.com/sota/image-classification-on-imagenet>

Outline:

- CNN building blocks
- Tasks in Computer Vision
- Datasets for Computer Vision & Historical CNNs
- What a CNN sees?

Human Vision



A system trained on 540 million years
of data

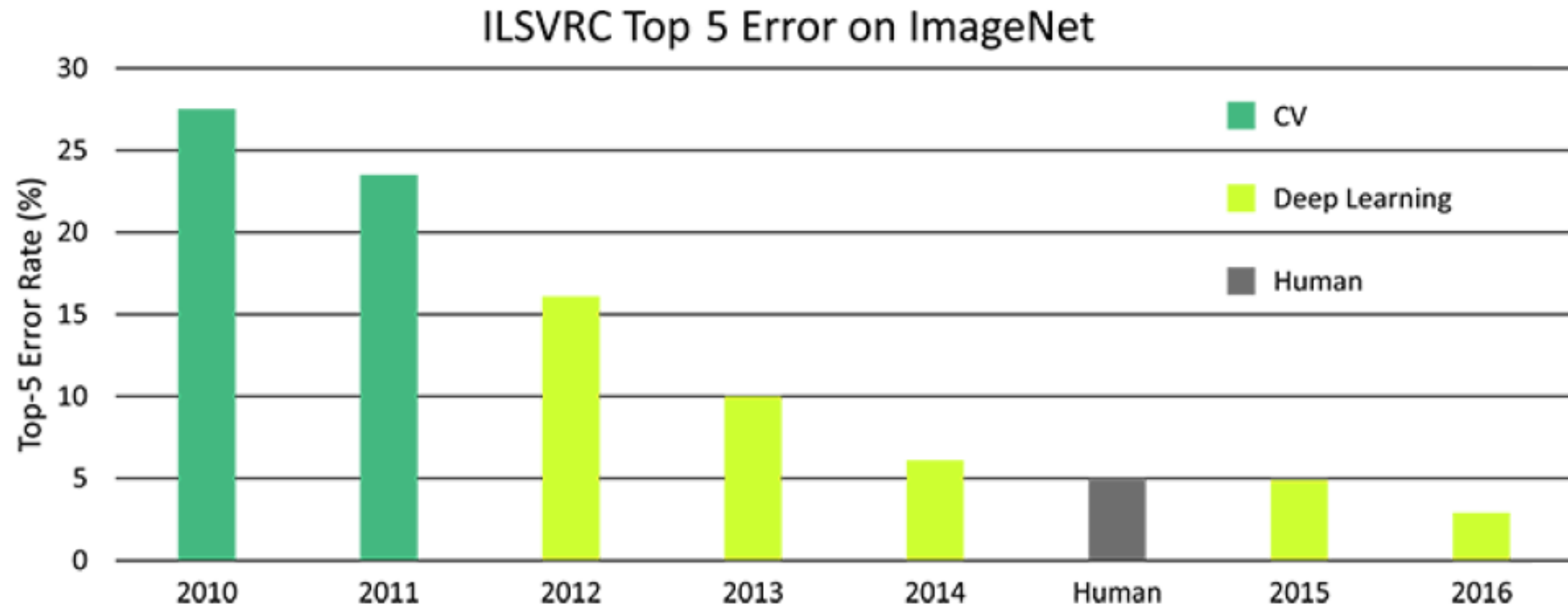
Computer Vision



Started in the 60s aiming at automatically doing what human vision can do

CV is undoubtedly THE field that mostly benefit from Deep Learning

DL has been disruptive in computer vision!



A dataset with more than 14 Million annotated images of 20K categories

Images are numbers

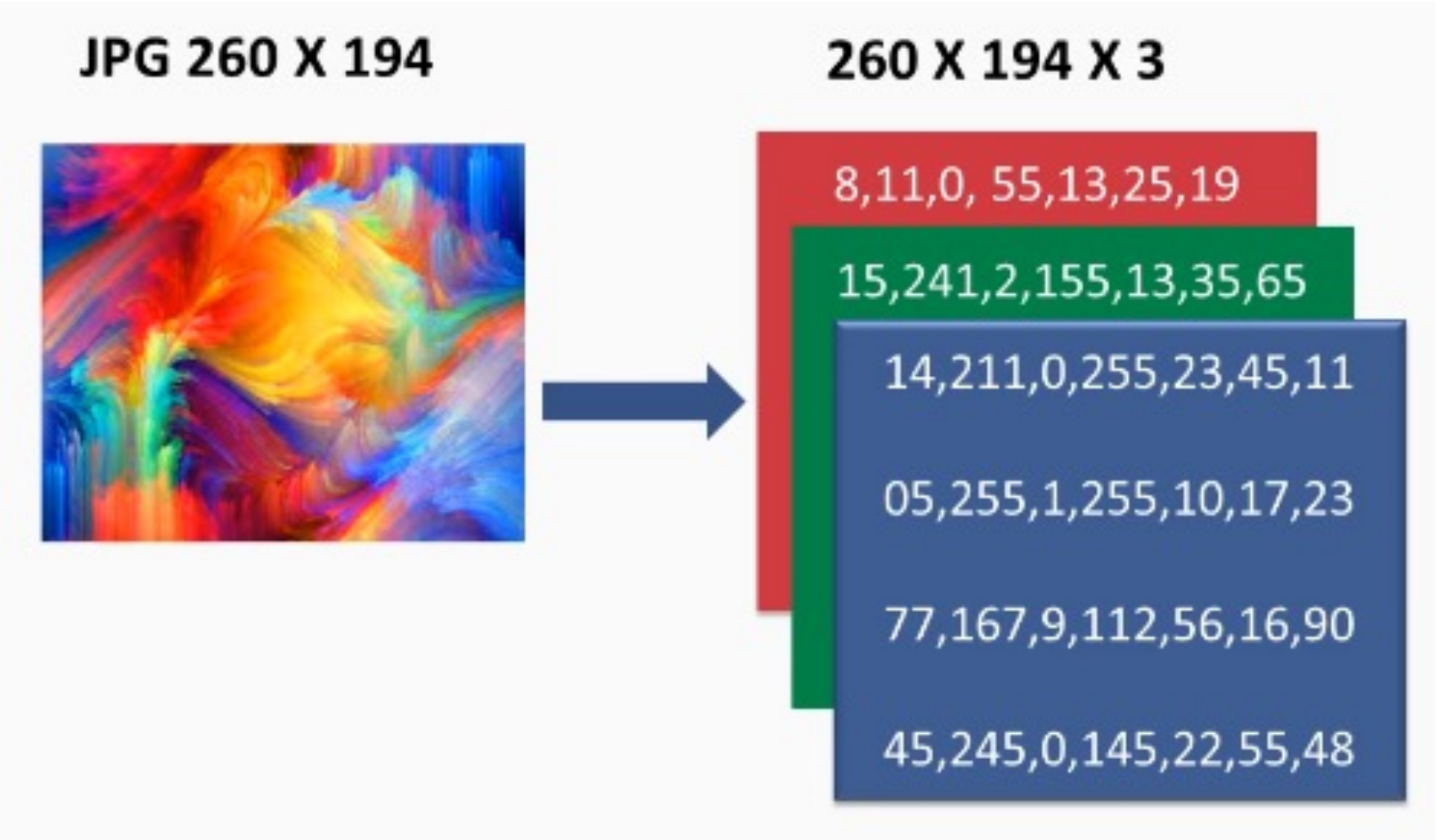


157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	105	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

What the computer sees

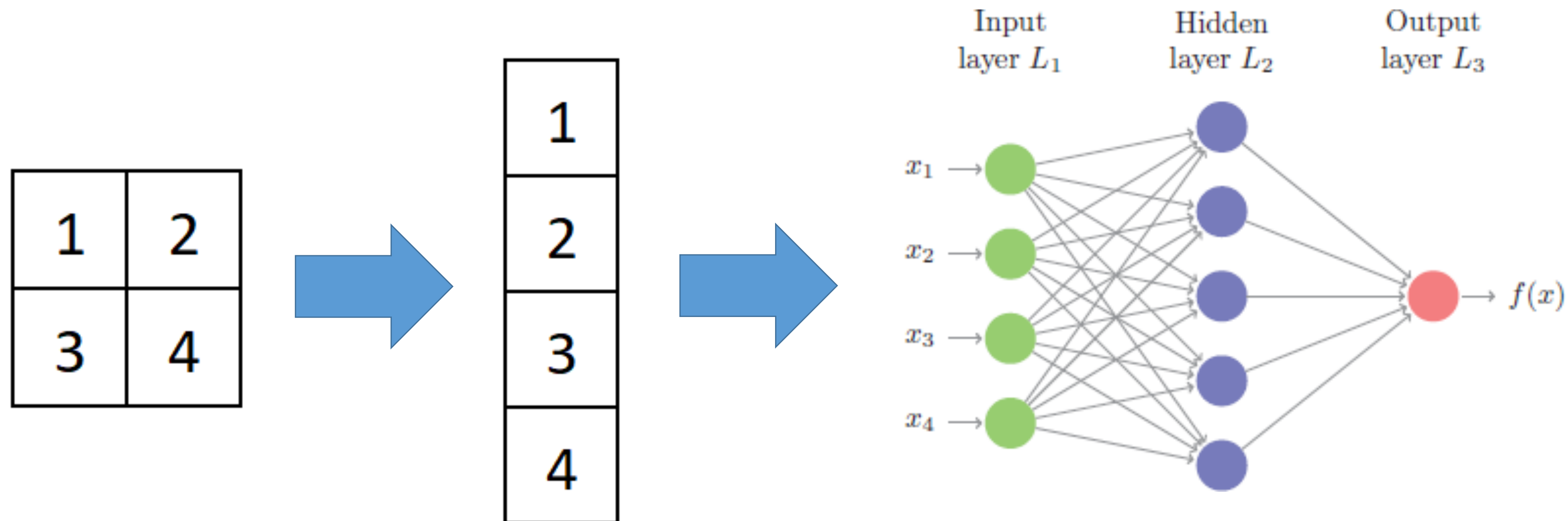
157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	105	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

L. Fridman MIT Deep Learning <https://deeplearning.mit.edu/>



Using FFNN is not a good idea...

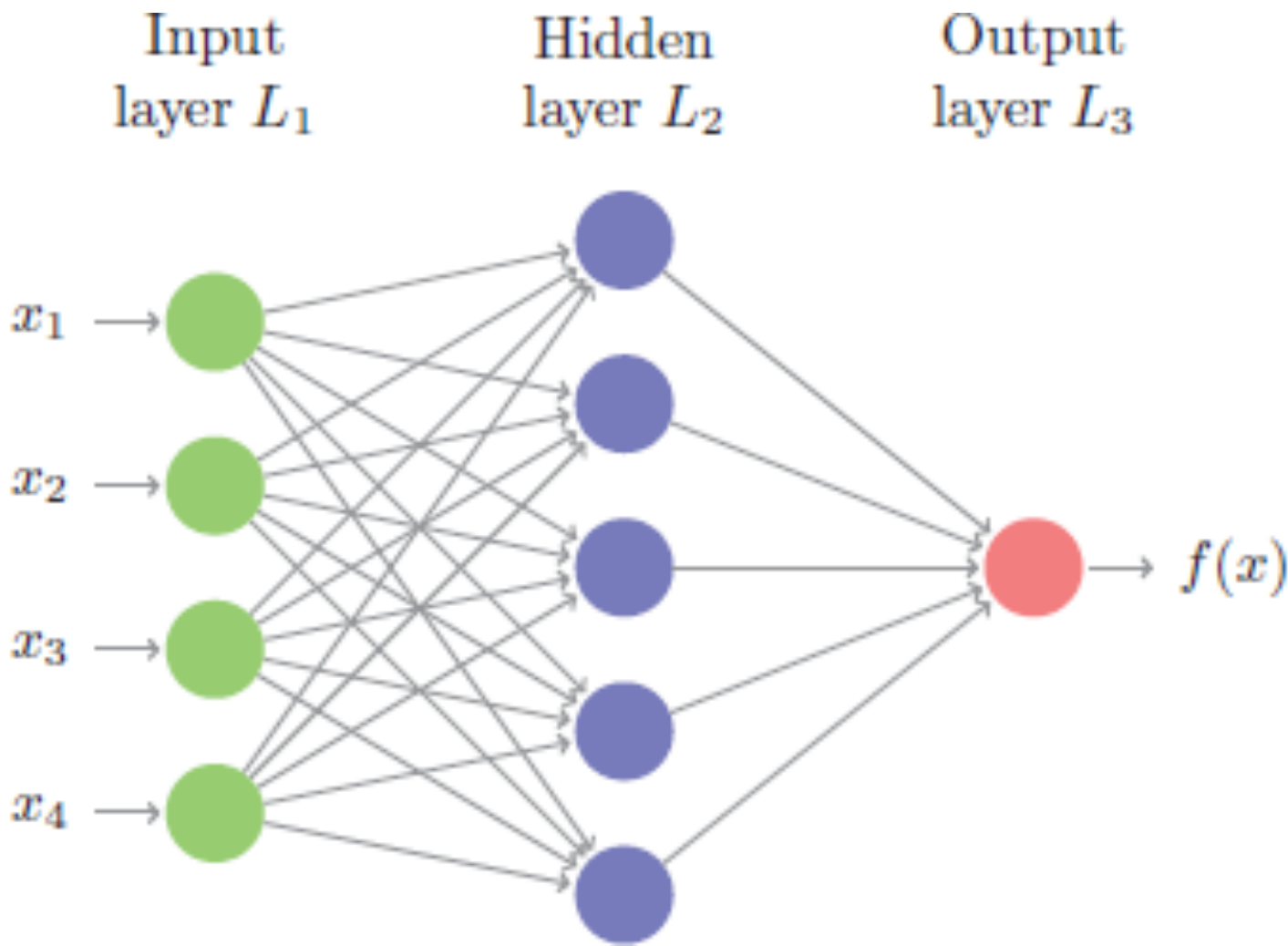
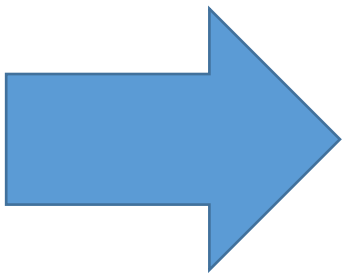
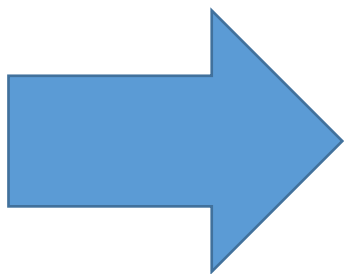
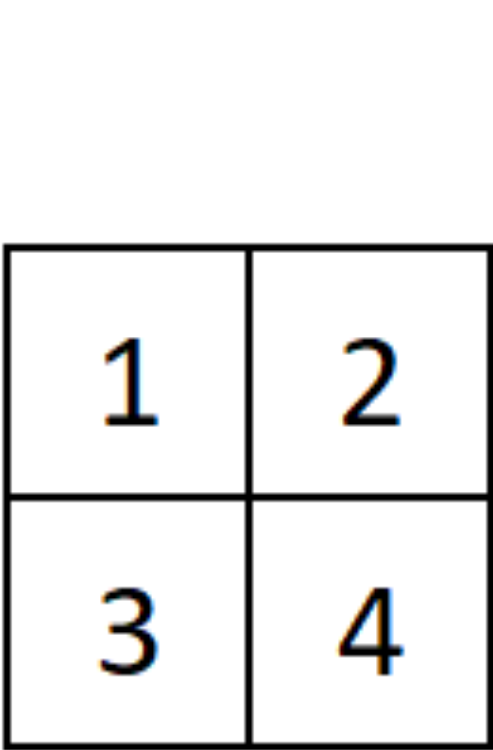
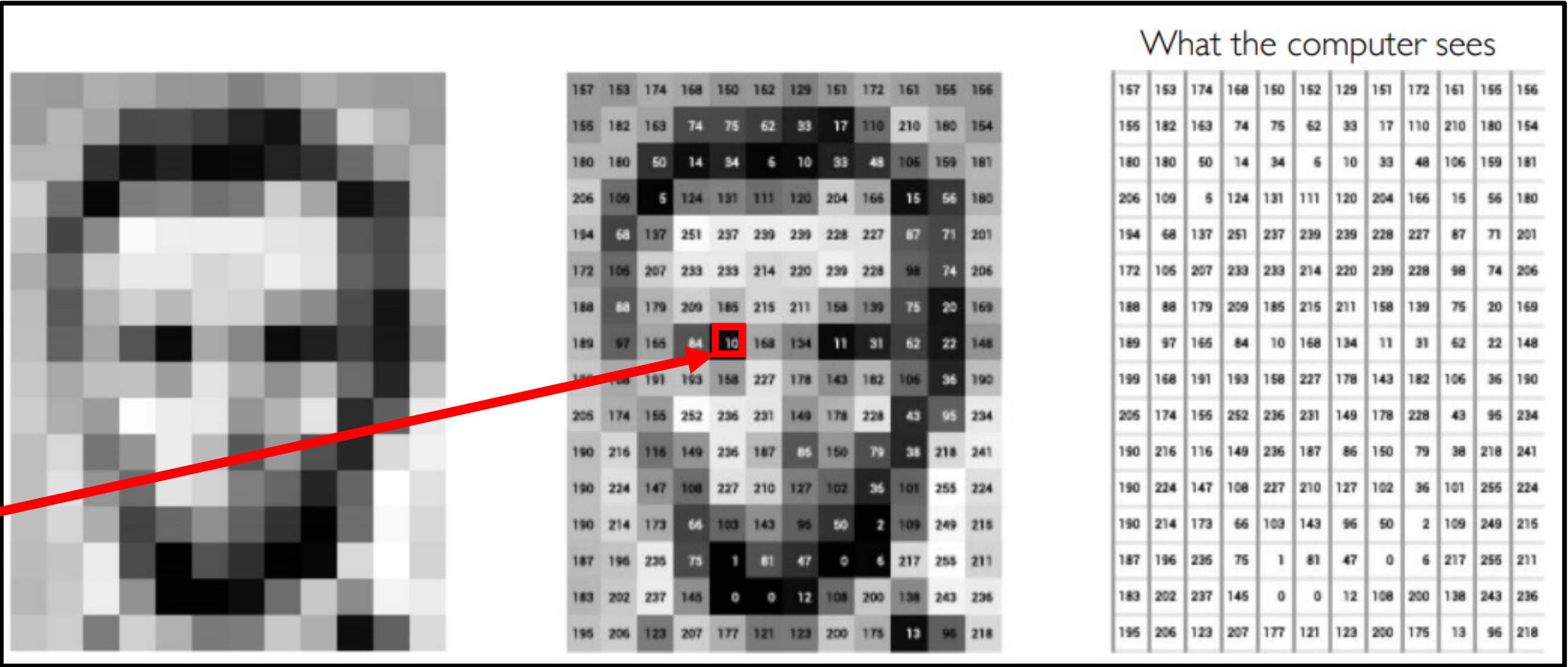
- 2D images converted to arrays: **spatial information is lost!**
- Each hidden unit is connected to all units in the previous layers: **lots of parameters!**



Using FFNN is not a good

- 2D images converted to array

Does the value in this specific pixel really matters by itself?



In Computer Vision, feature engineering is hard

Domain knowledge

Define features

Detect features
to classify

Viewpoint variation



Scale variation



Deformation



Occlusion



Illumination conditions



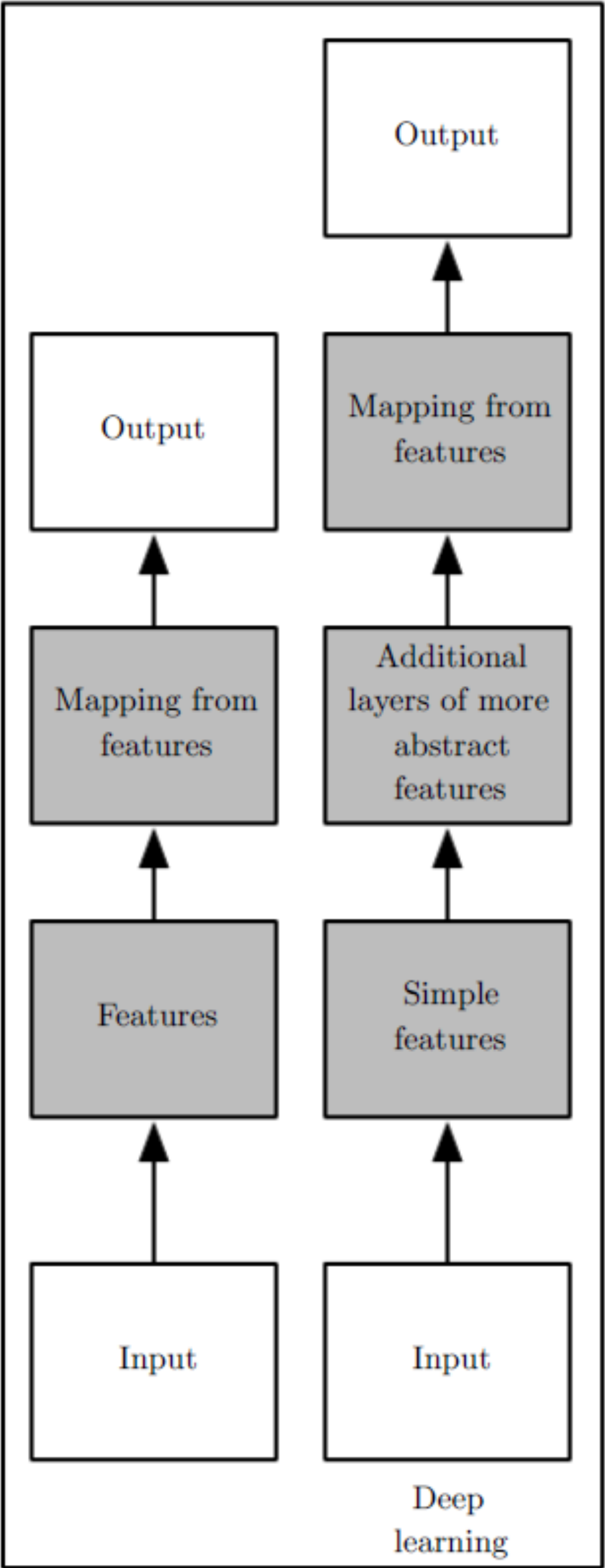
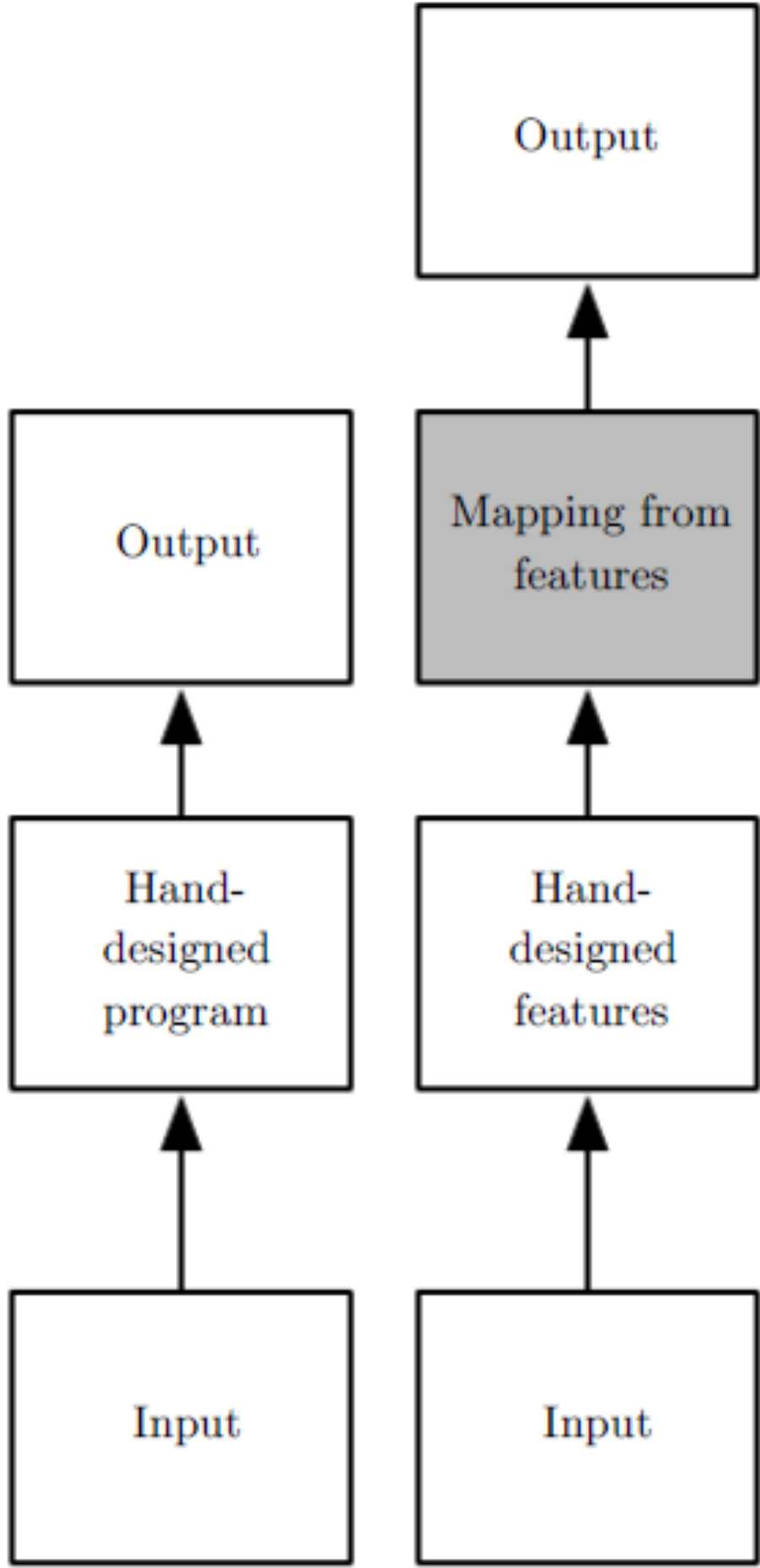
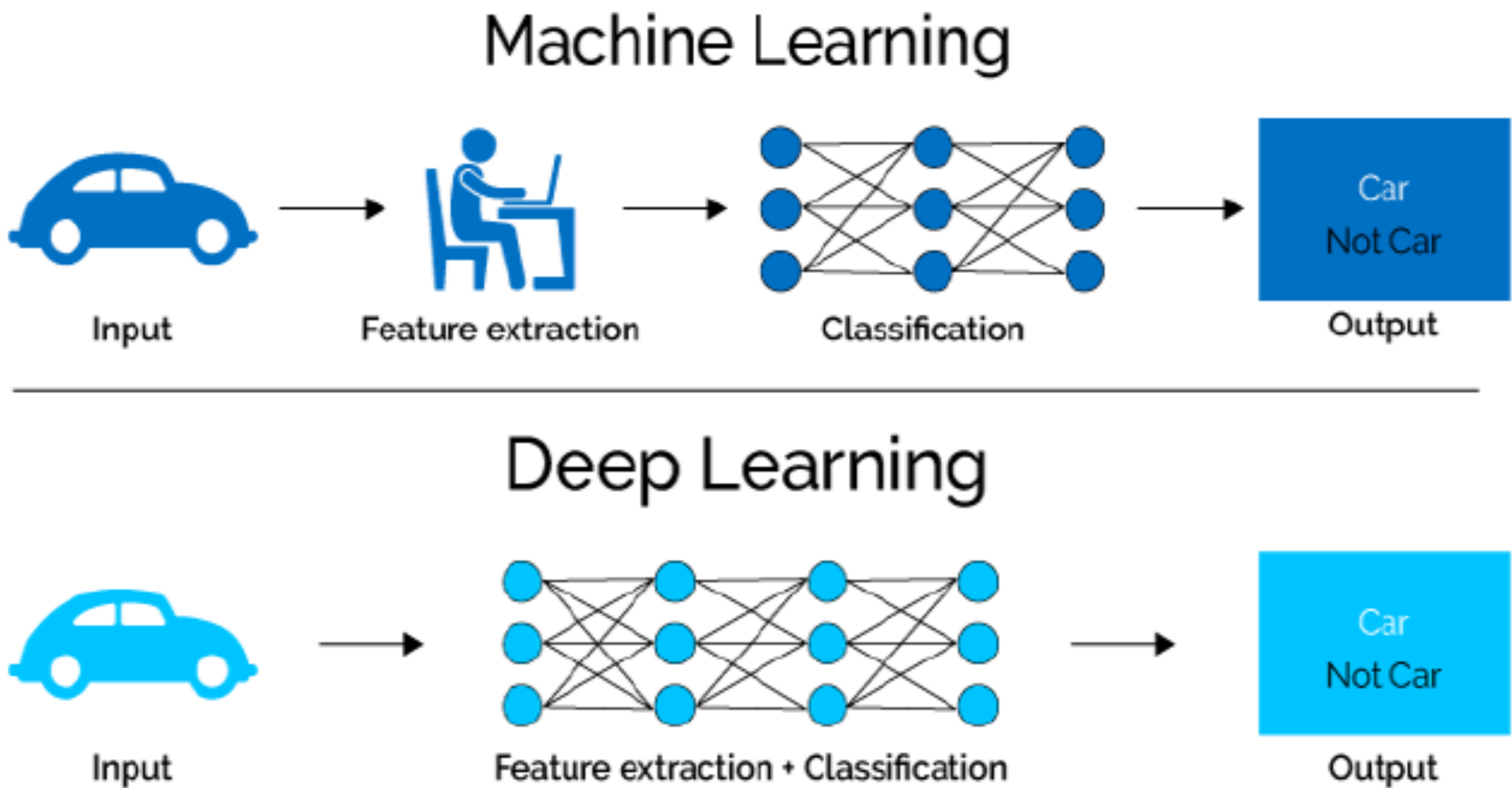
Background clutter



Intra-class variation



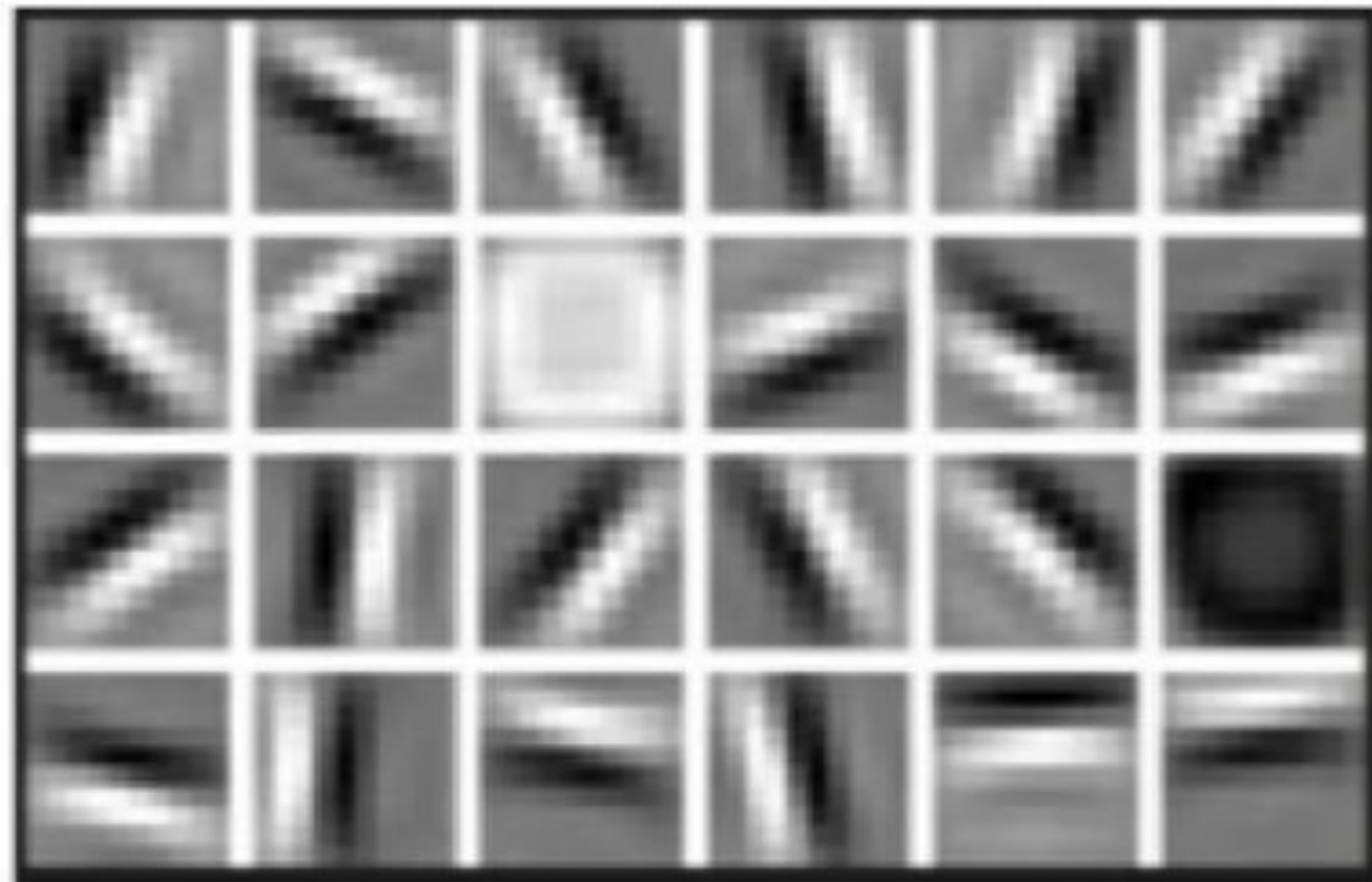
No feature engineering is needed with DL



L. Fridman MIT Deep Learning <https://deeplearning.mit.edu/>

Features extracted by Deep Neural Networks

Low Level Features



Lines & Edges

Mid Level Features



Eyes & Nose & Ears

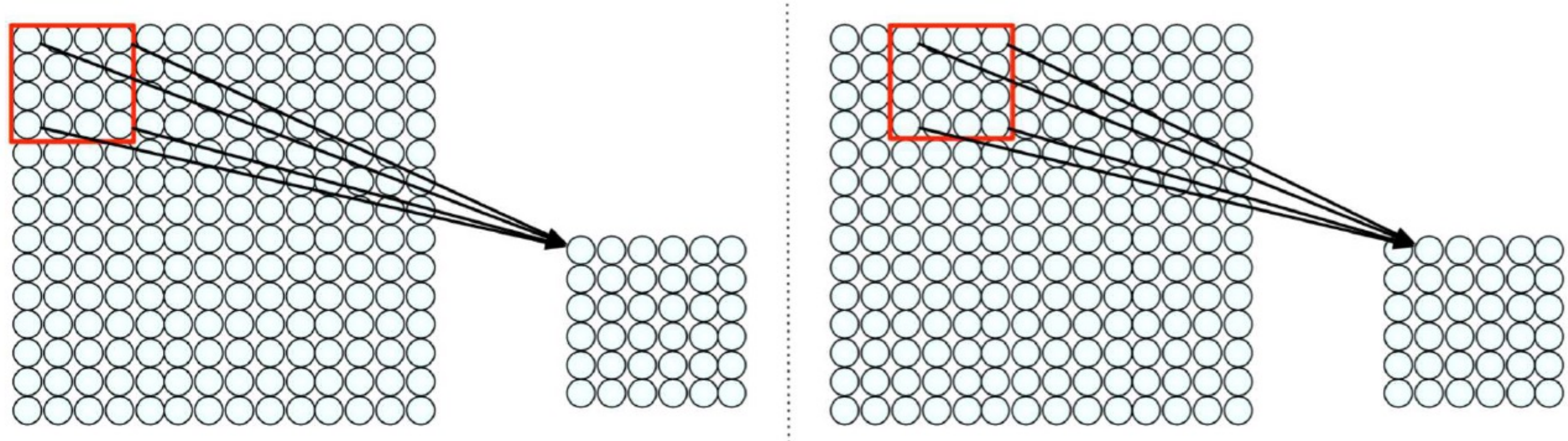
High Level Features



Facial Structure

These features were obtained in an automatic fashion!

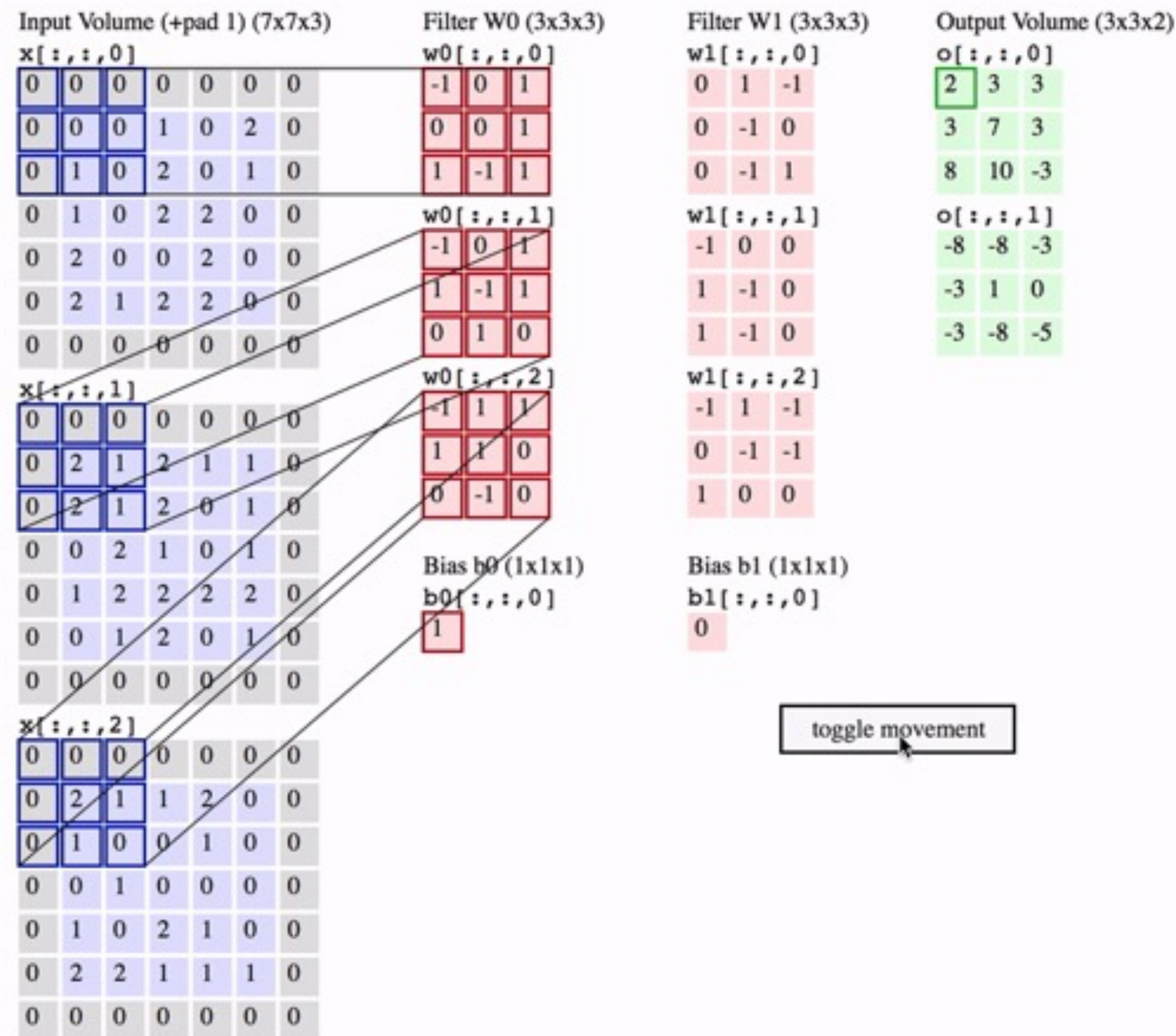
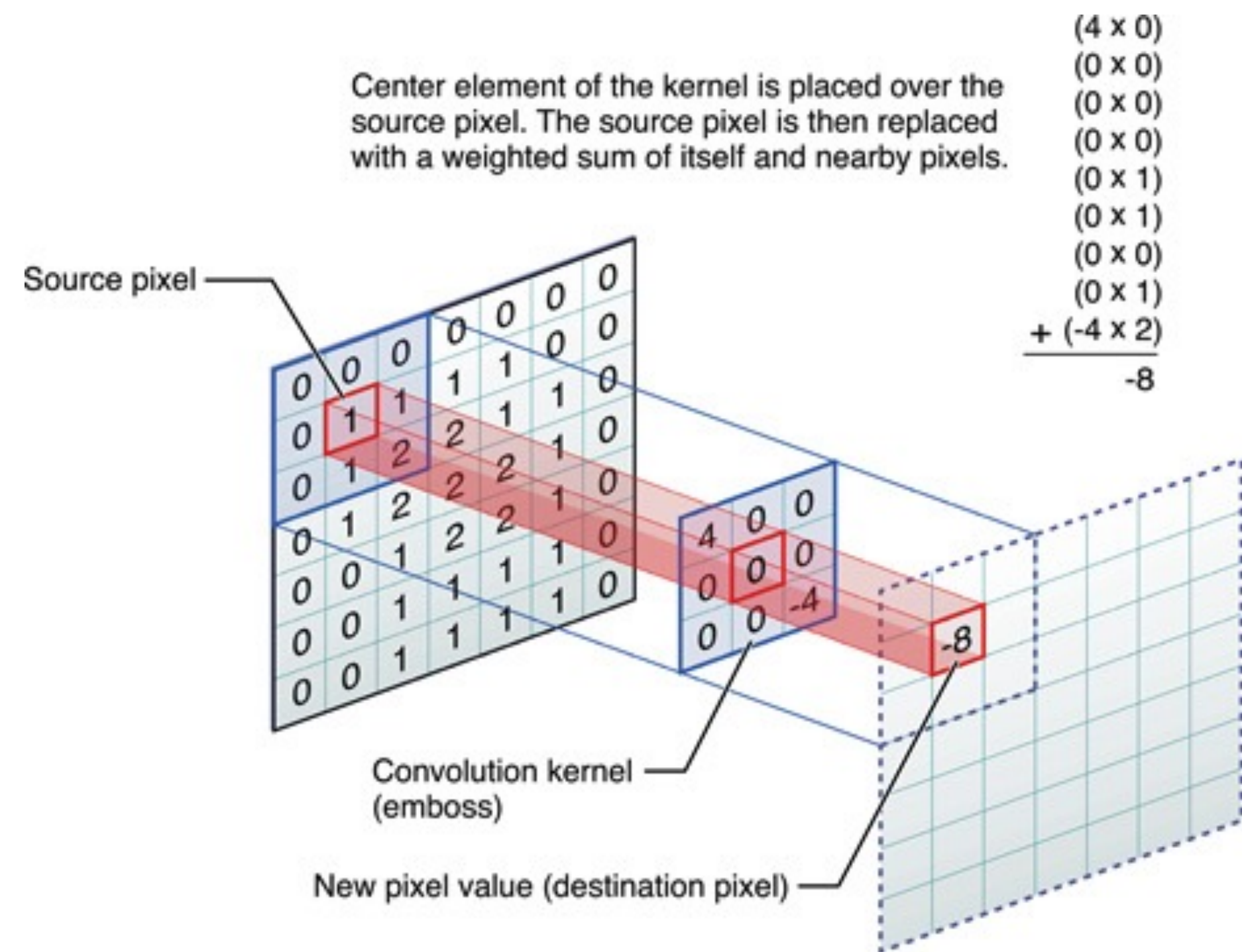
Idea: using the spatial structure (patches)



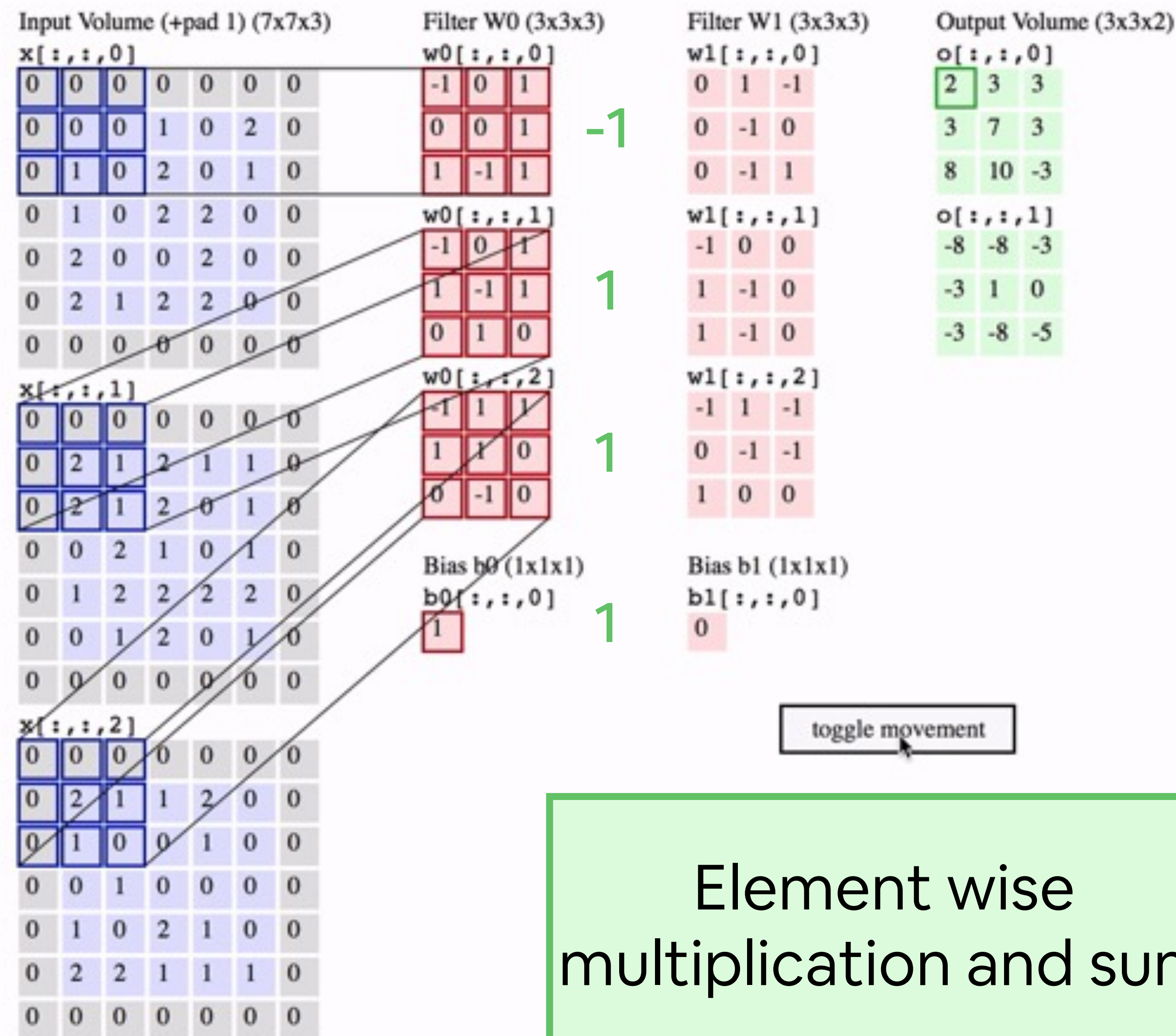
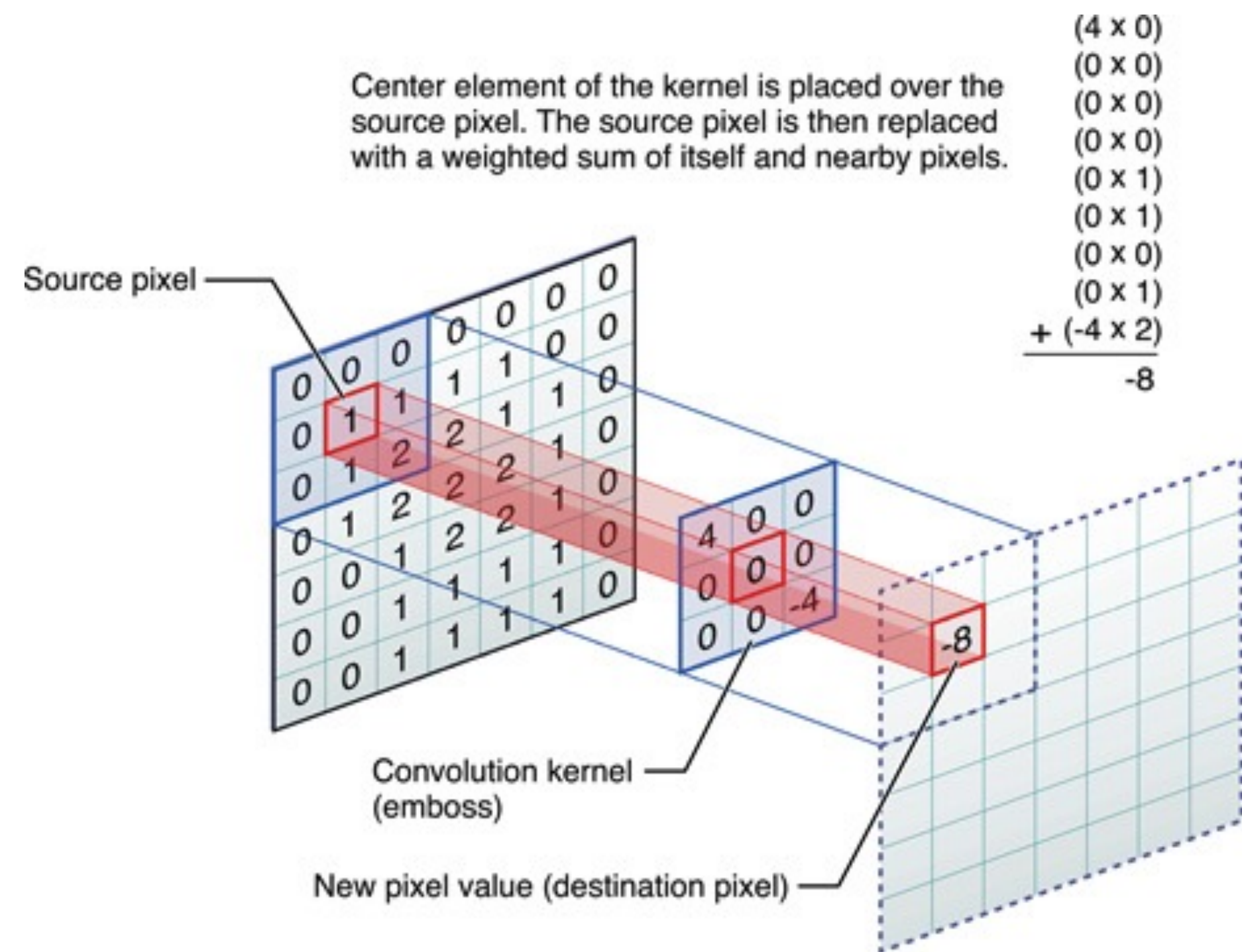
-MIT *Introduction to Deep Learning* <http://introtodeeplearning.com>

- Connect patch in input layer to a single neuron in subsequent layer.
- Use a sliding window to define connections.

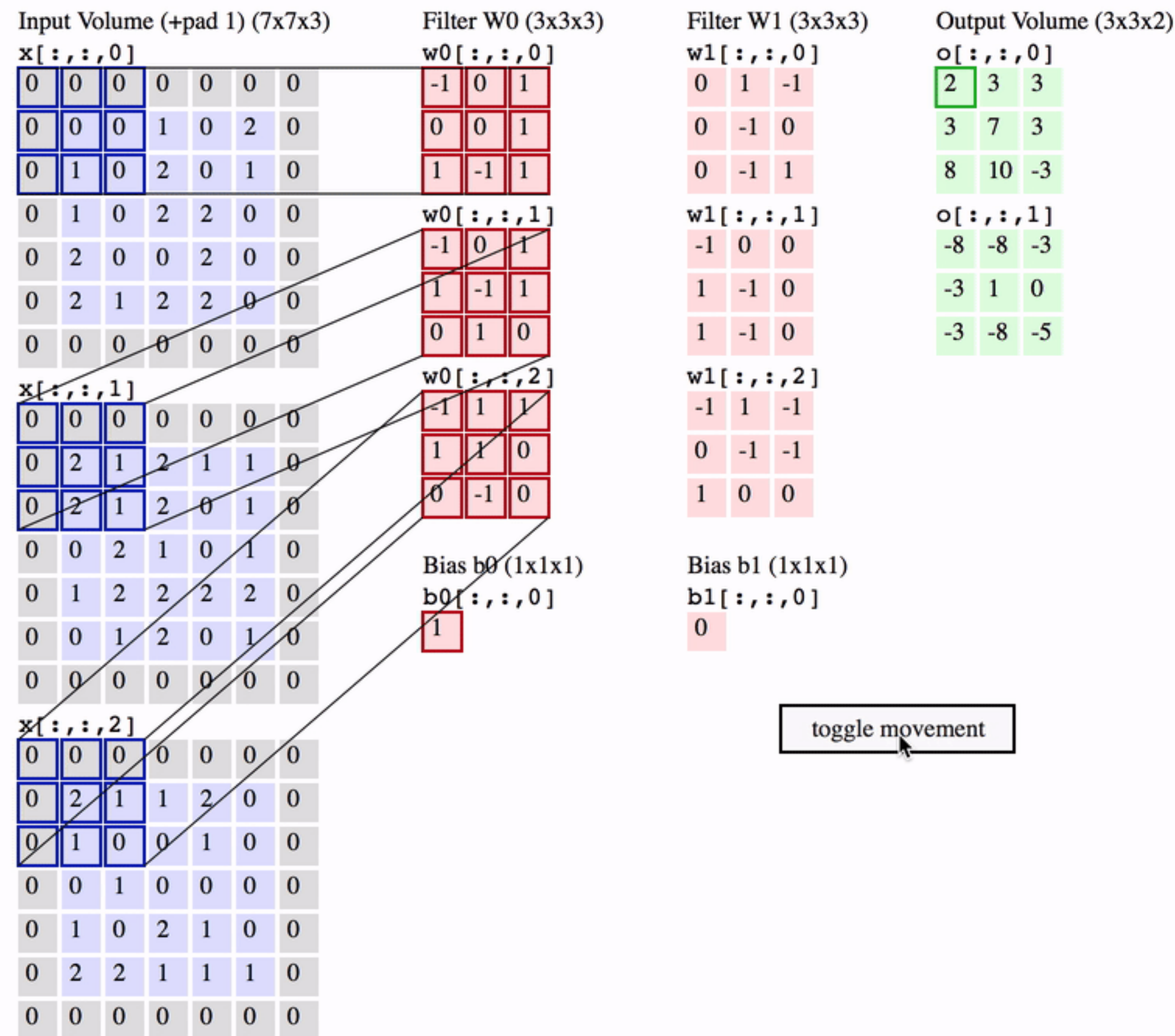
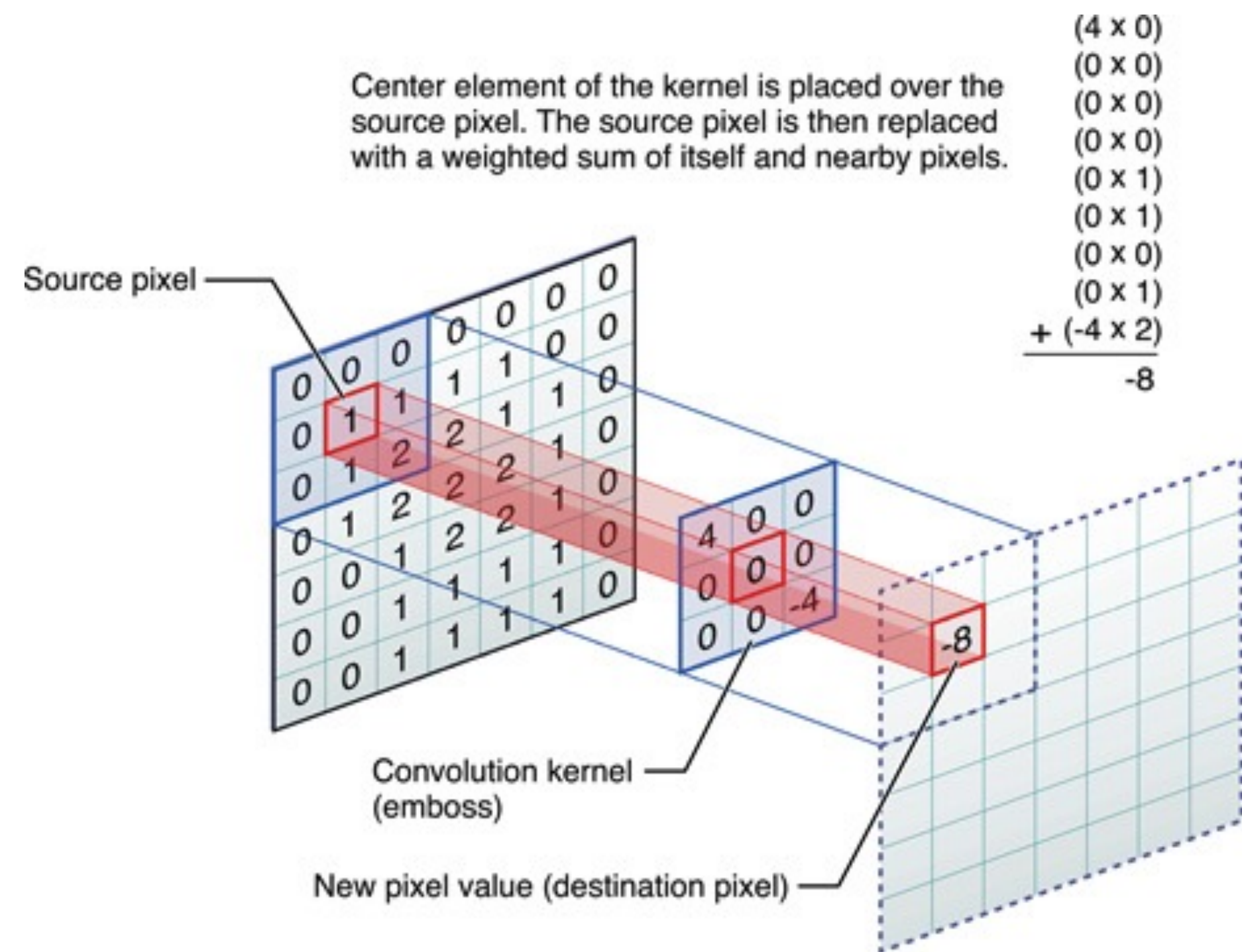
Idea: 2D convolutions



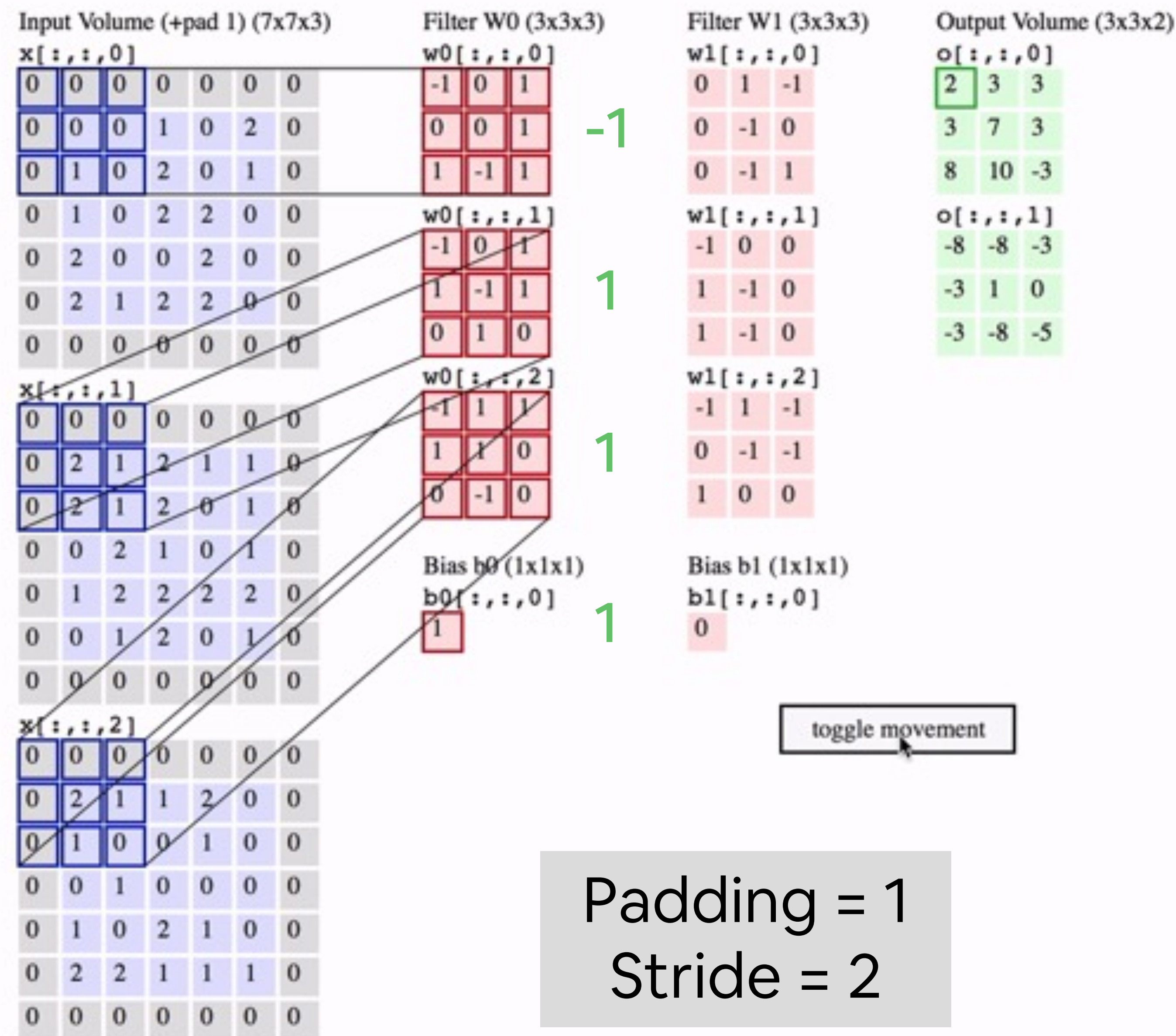
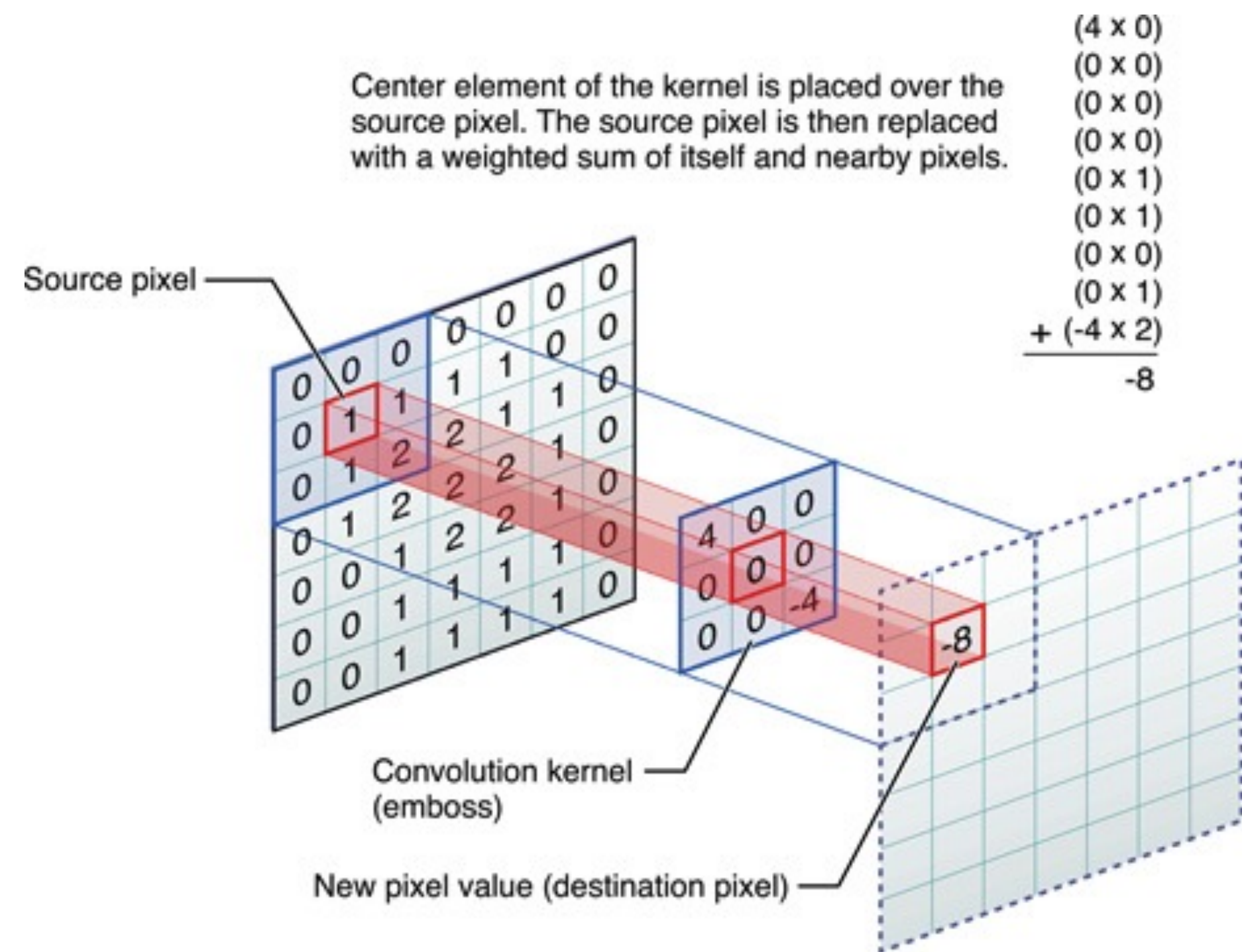
Idea: 2D convolutions



Idea: 2D convolutions



Idea: 2D convolutions



Feature maps induced by convolutions: some examples



Original



Sharpen



Edge Detect



"Strong" Edge
Detect

Feature maps induced by convolutions: some examples



Original



Sharpen



Edge Detect



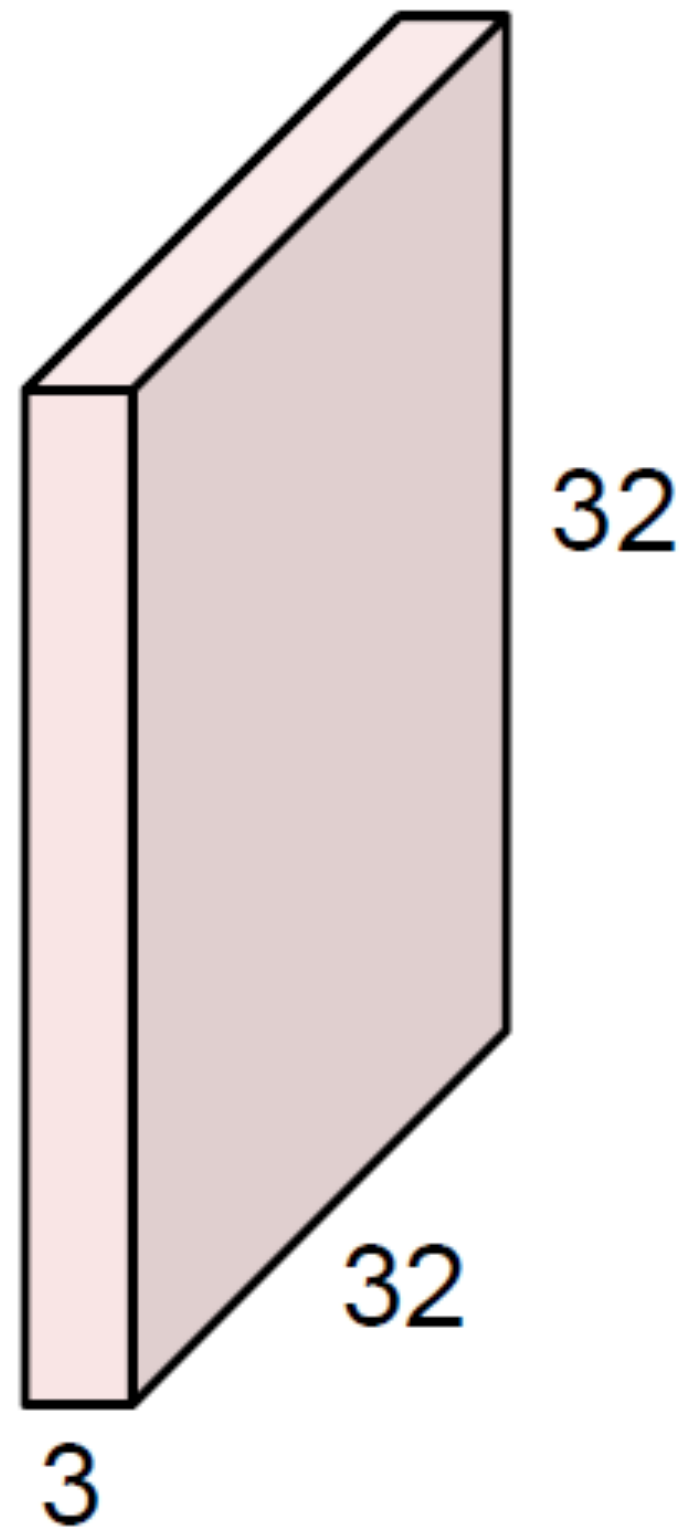
“Strong” Edge
Detect

-MIT *Introduction to Deep Learning* <http://introtodeeplearning.com>

?	?	?
?	?	?
?	?	?

Recap

32x32x3 image

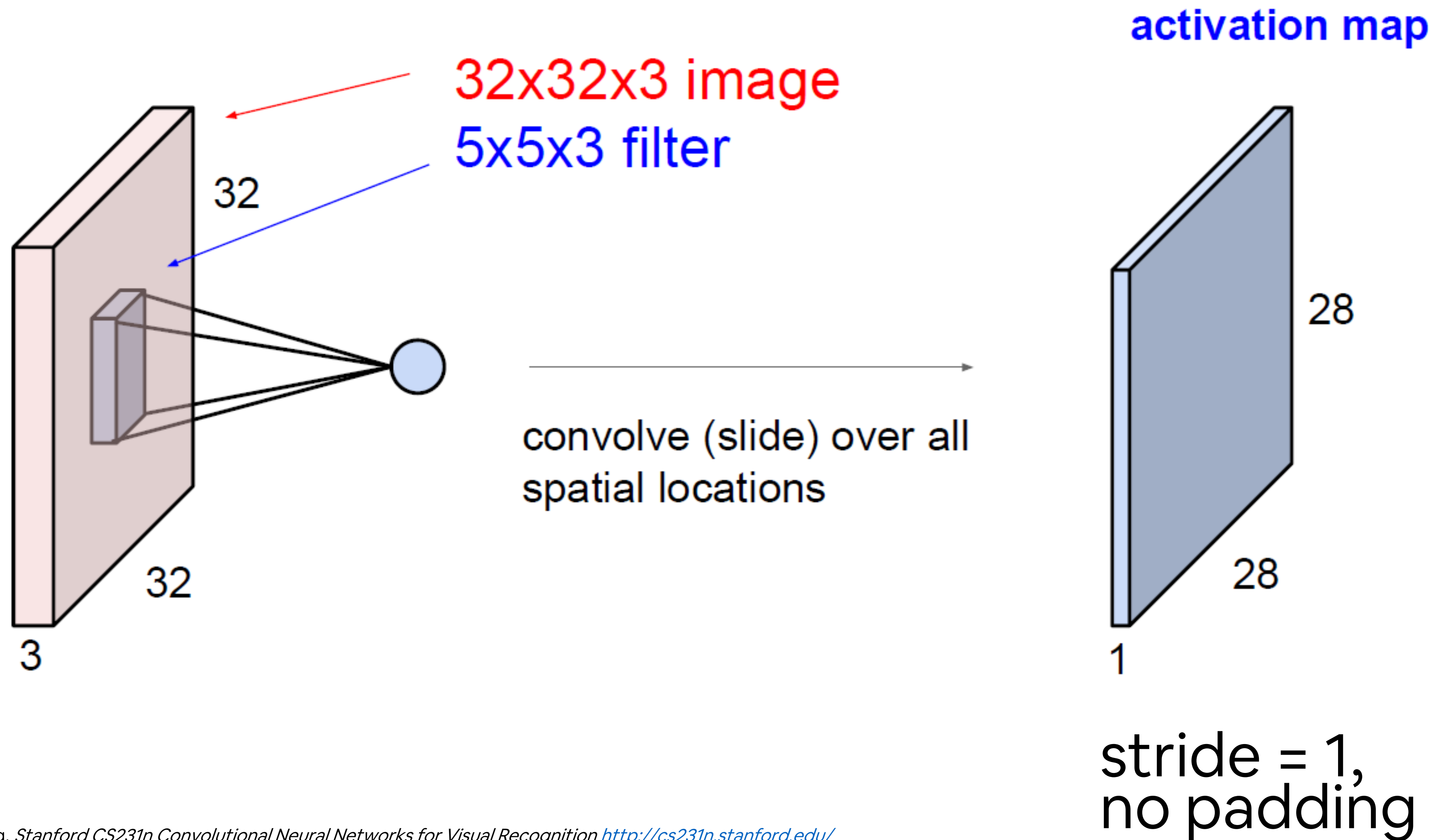


5x5x3 filter

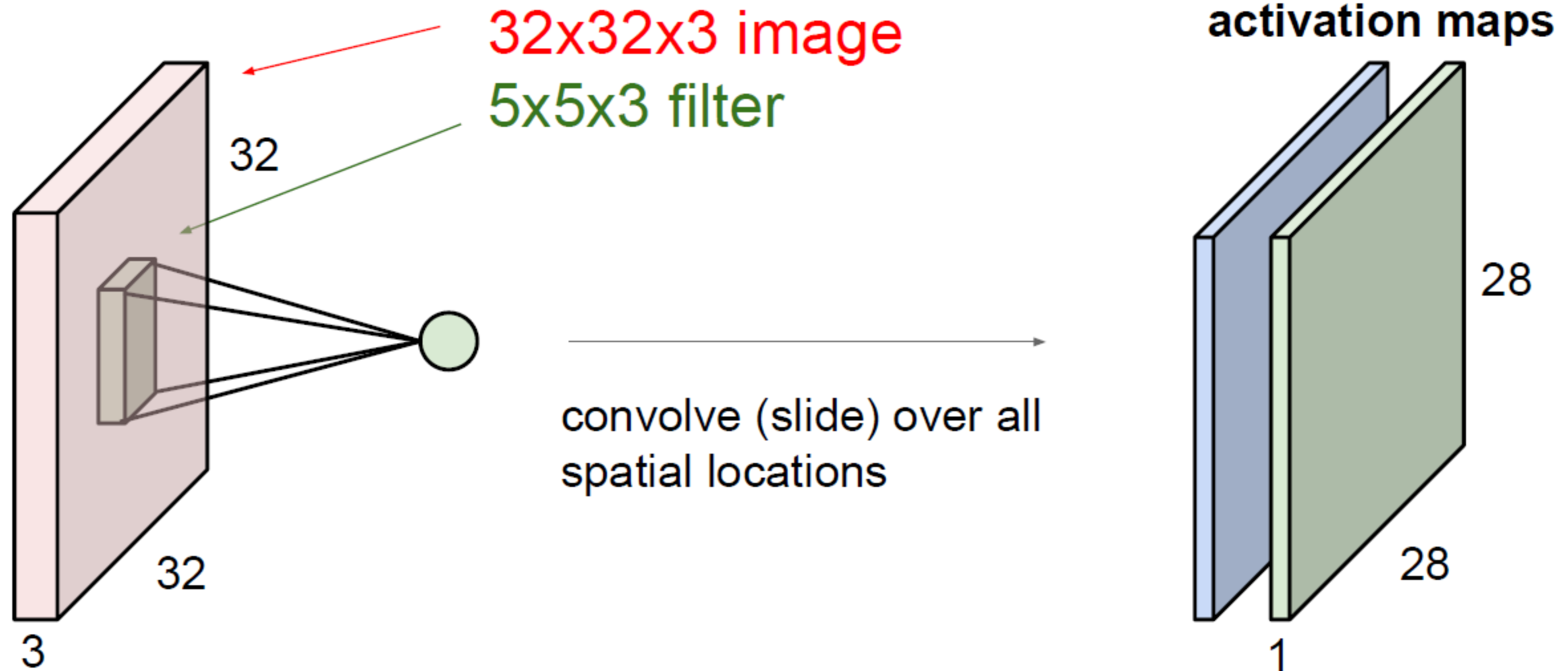


Convolve the filter with the image
i.e. “slide over the image spatially,
computing dot products”

Recap

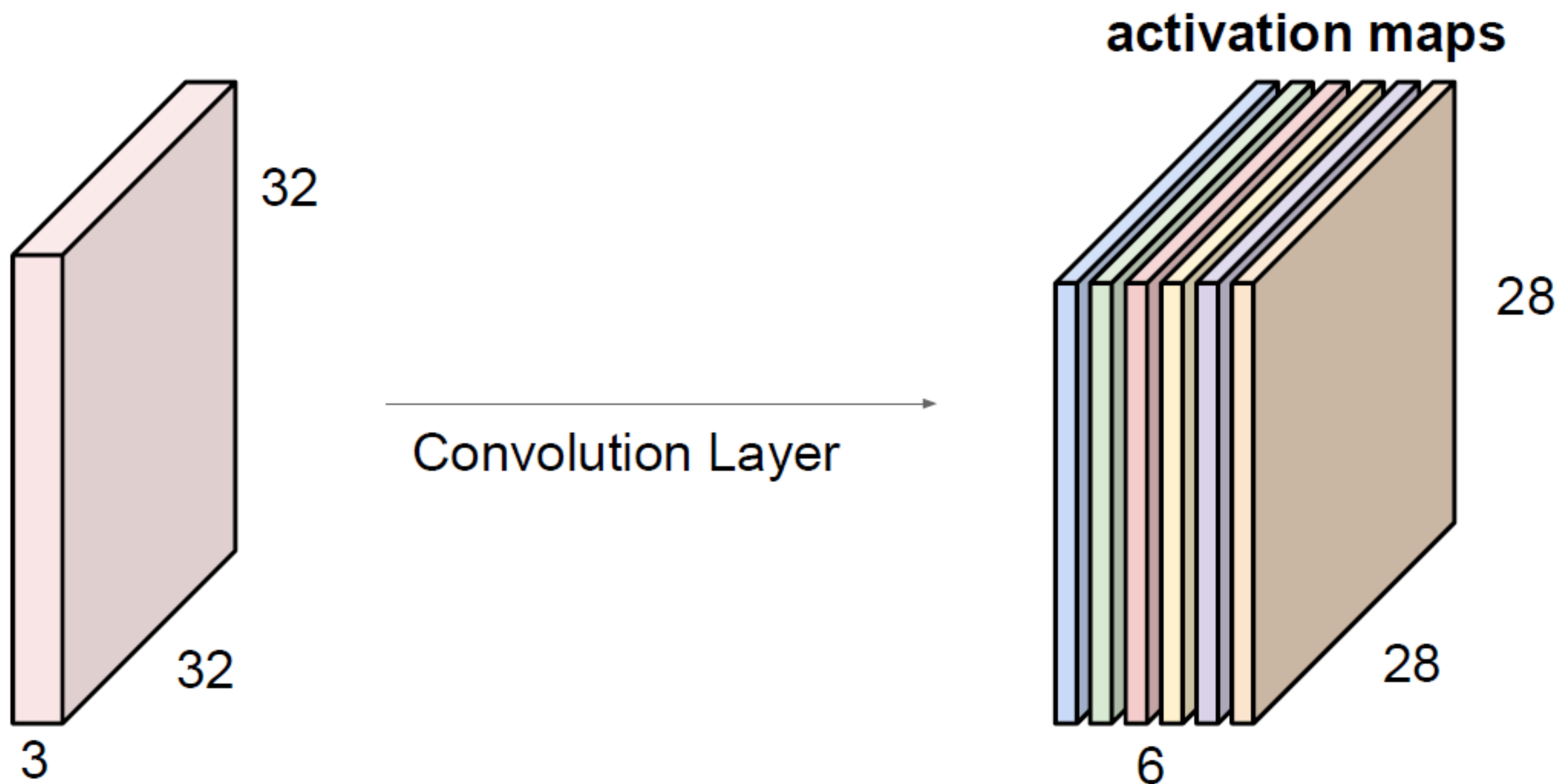


Recap

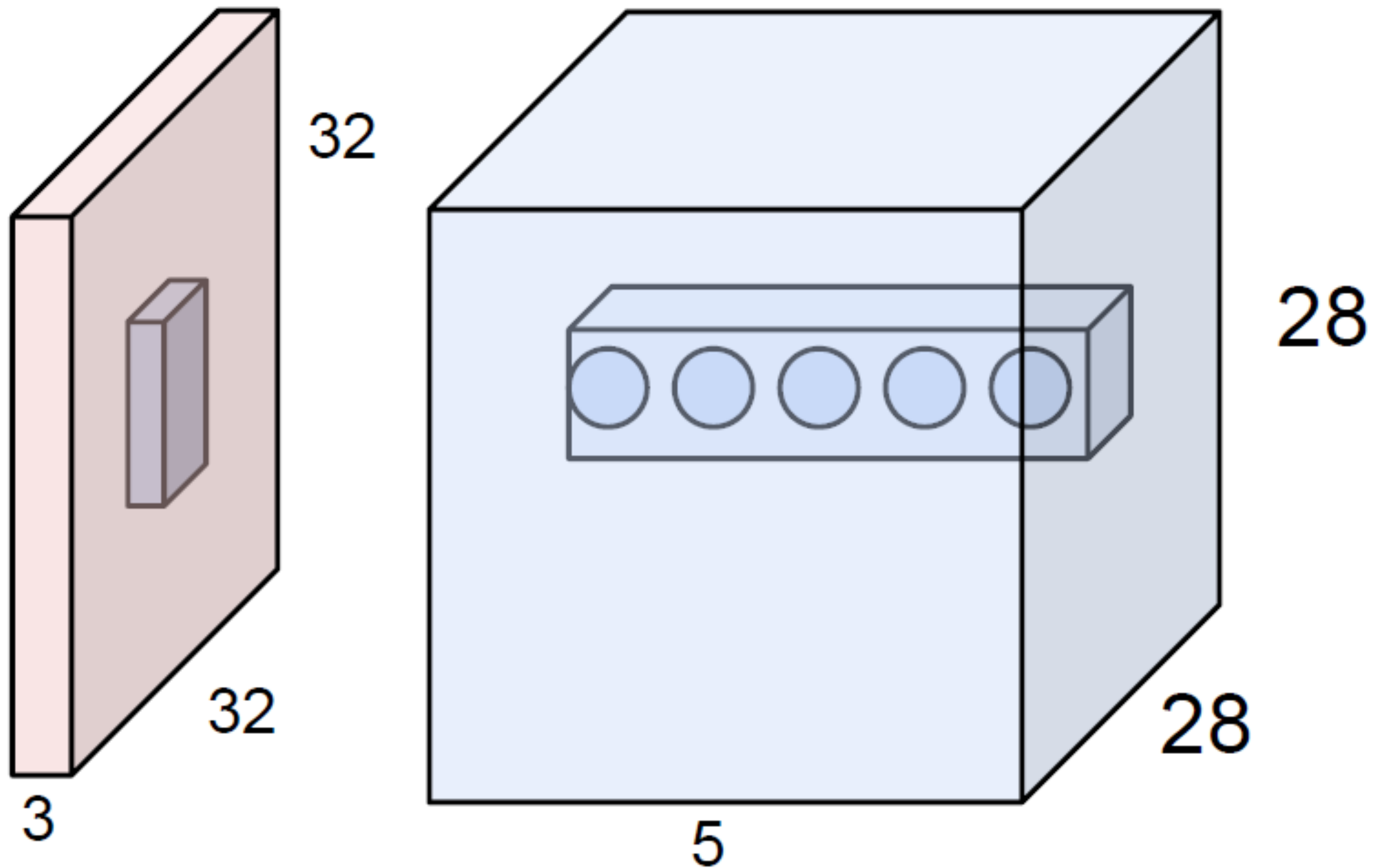


stride = 1,
no padding

Recap



Receptive field

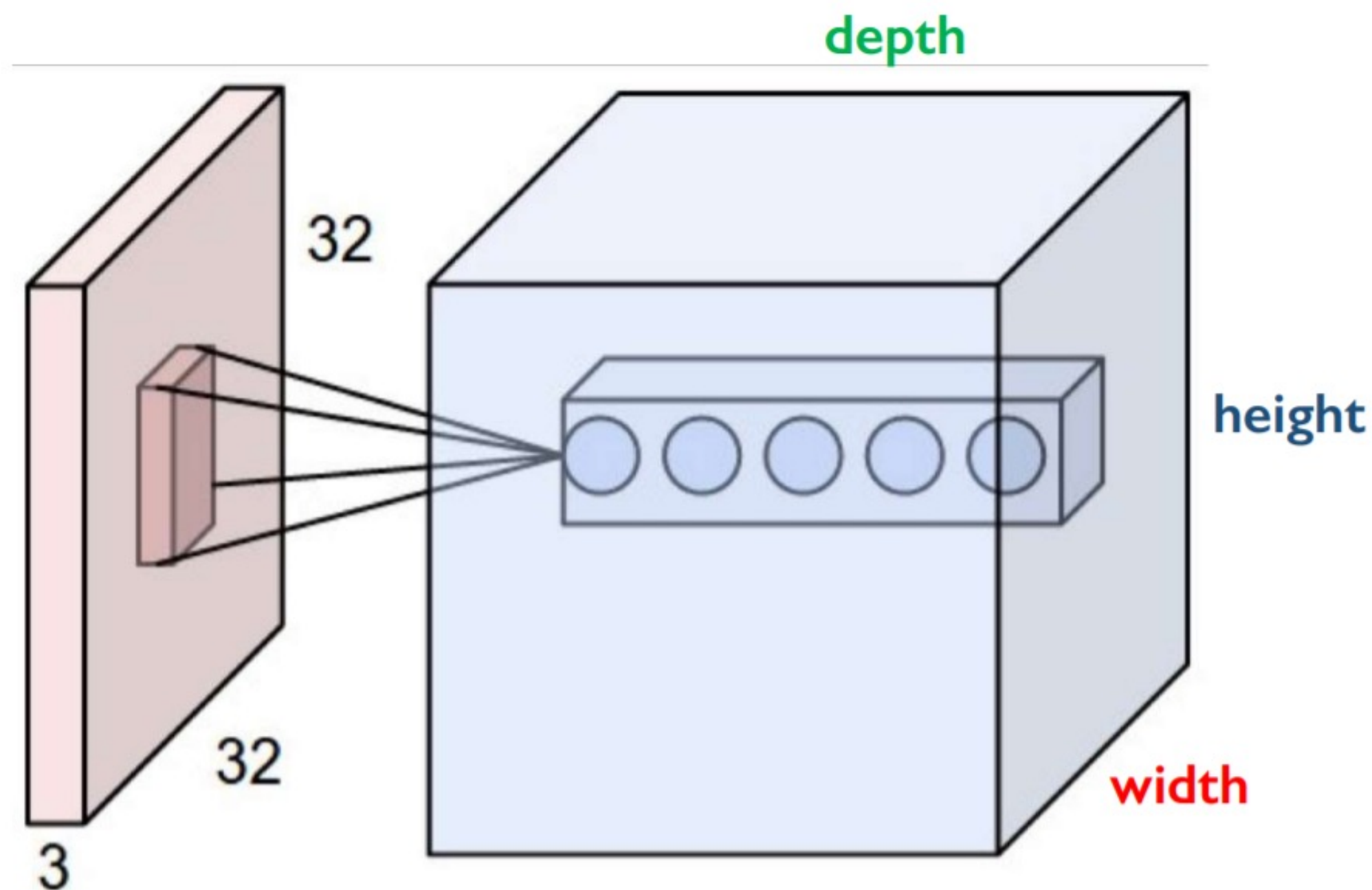


E.g. with 5 filters, CONV layer consists of neurons arranged in a 3D grid (28x28x5)

There will be 5 different neurons all looking at the same region in the input volume

Pros of 2D convolutions

- Use convolution filters (weights) to extract **local features**
- Use multiple filters to extract **different features**
- Filter parameters are **shared spatially** across the whole input



Layer Dimensions:

$$h \times w \times d$$

where h and w are spatial dimensions
 d (depth) = number of filters

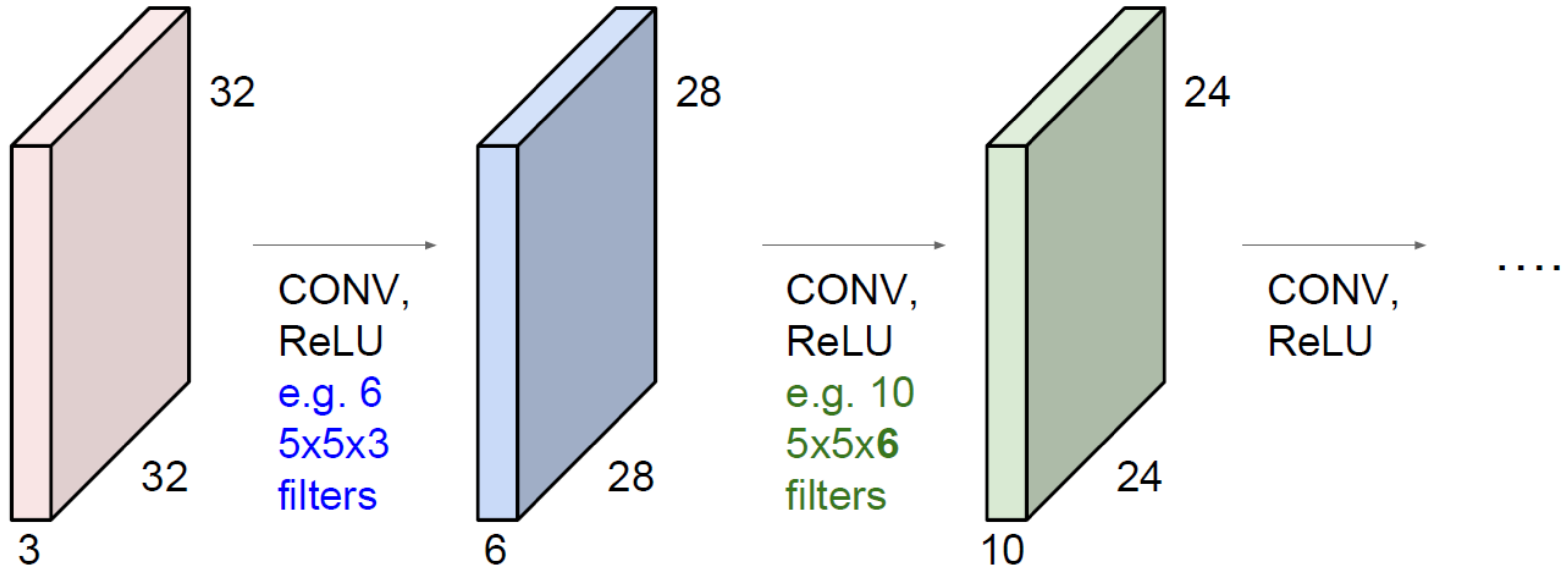
Stride:

Filter step size

Receptive Field:

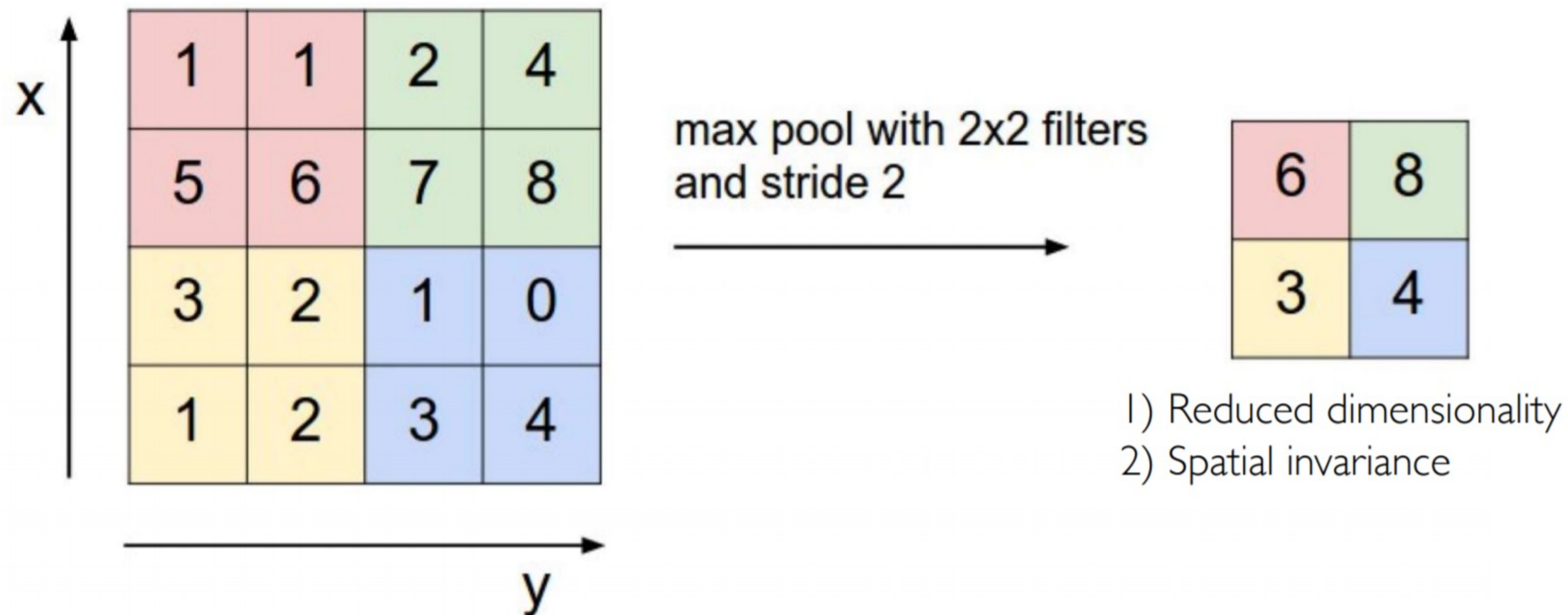
Locations in input image that
a node is path connected to

Deep Learning: we stack different layers!

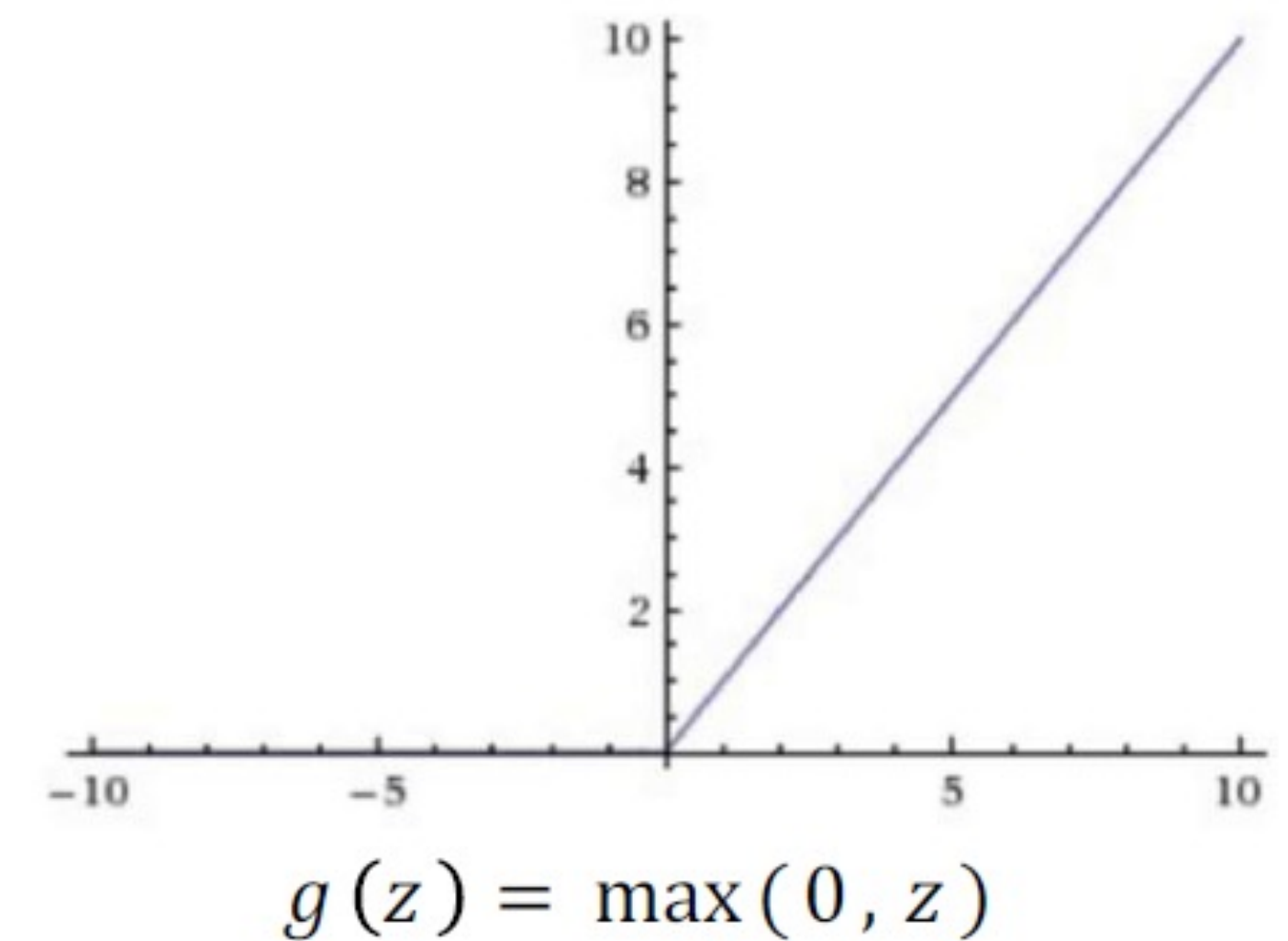


Make extracted features rich, robust and low-dimensional

- Introduce non-linearity: apply non-linear activation to convolution output
- Downsample and preserve spatial invariance: pooling



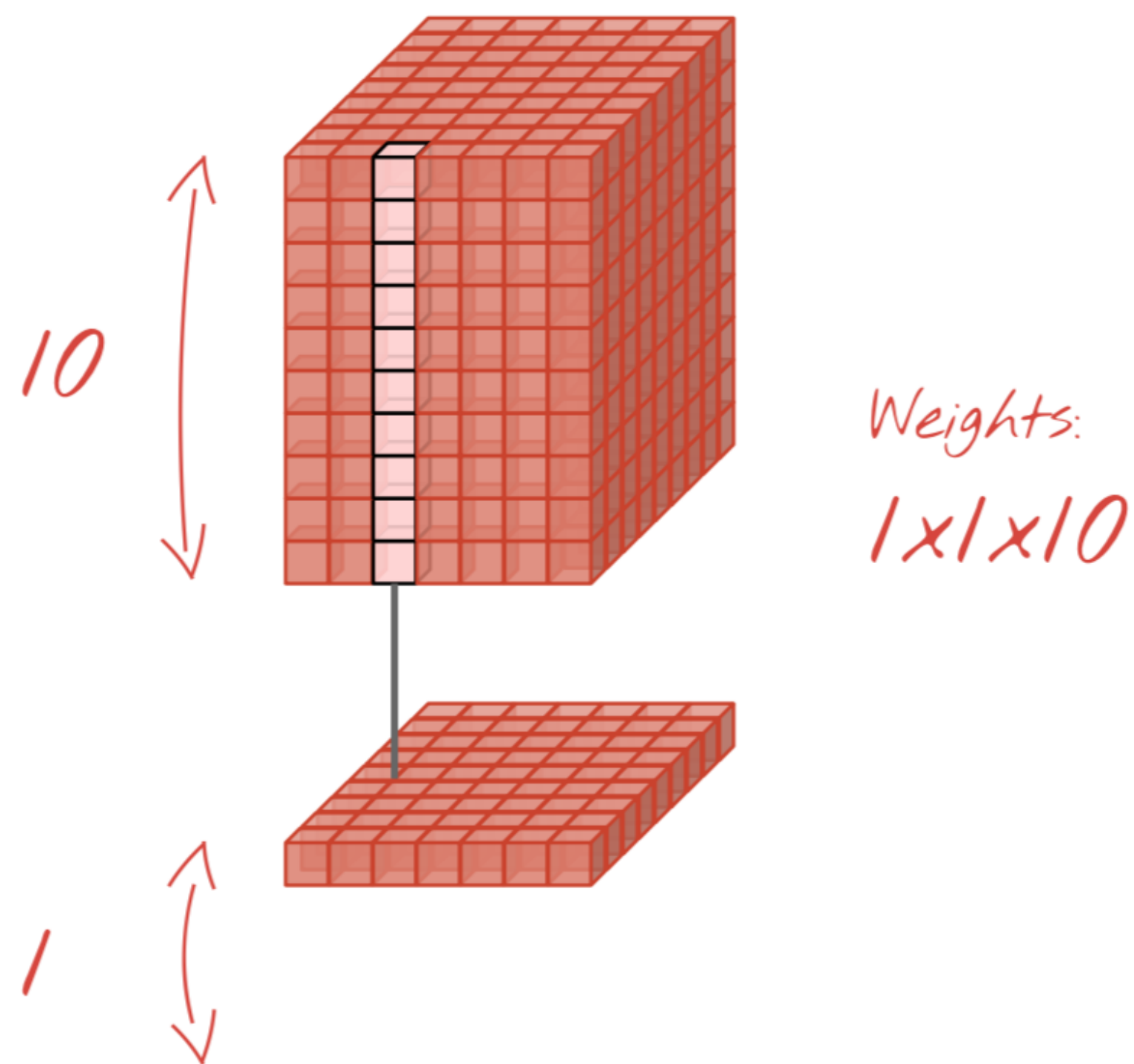
Rectified Linear Unit (ReLU)



Dimensionality reduction

- While to reduce width and height we can exploit pooling (or bigger strides), to reduce depth we can use 1d convolutions

1d convolutions



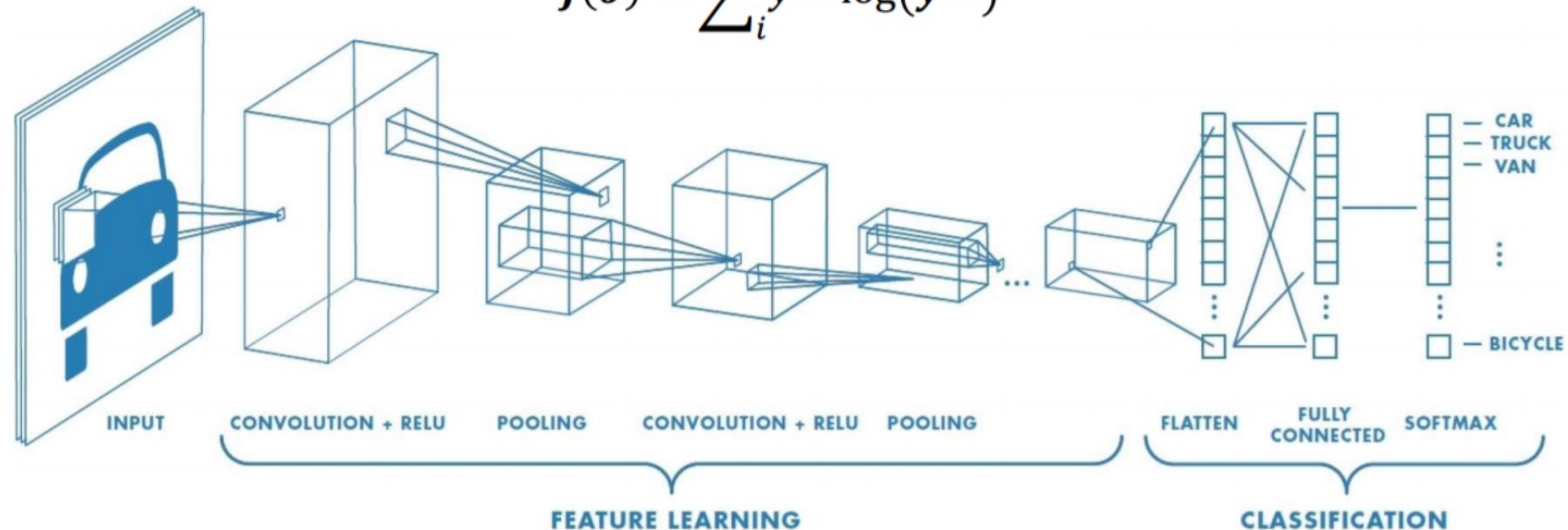
Why 1x1 convolutions?

- *Efficiency*: reduces the depth (number of channels). Width and height are unchanged. To reduce the horizontal dimensions, you would use pooling (or increase the stride of the conv).
- The 1x1 conv computes a weighted sum of input channels (or features). This allows it to "select" certain combinations of features that are useful downstream.

Now put it all together to perform **classification**

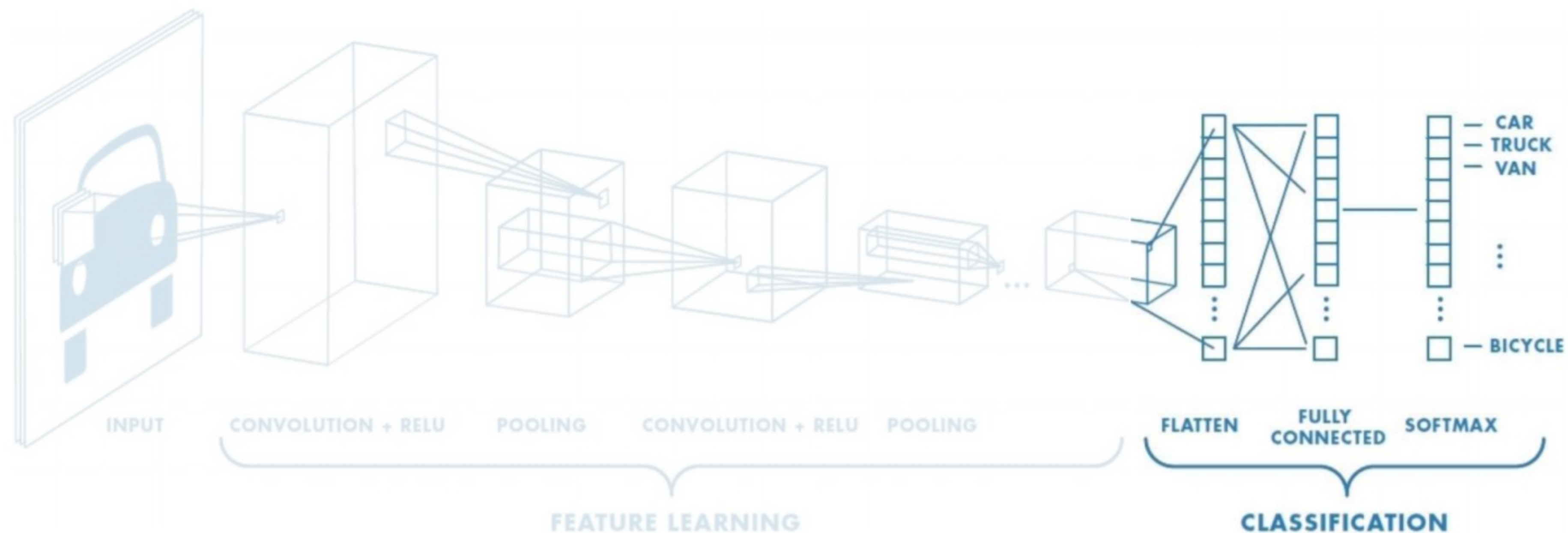
- We apply back-propagation to train model (weights for convolution and dense layers)
- Cross entropy loss

$$J(\theta) = \sum_i y^{(i)} \log(\hat{y}^{(i)})$$



Now put it all together to perform **classification**

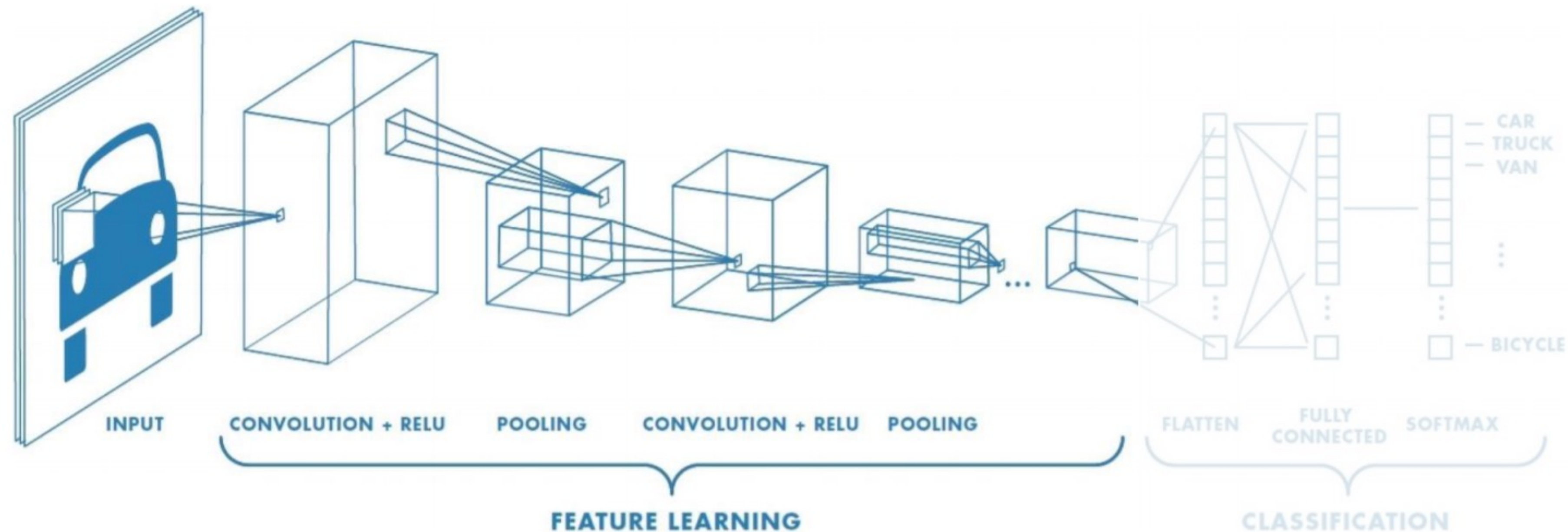
- Extract high-level features from last conv and pool layer
- Add fully connected layers to learn complex transformation of the features
- Output expressed as probability distribution over classes



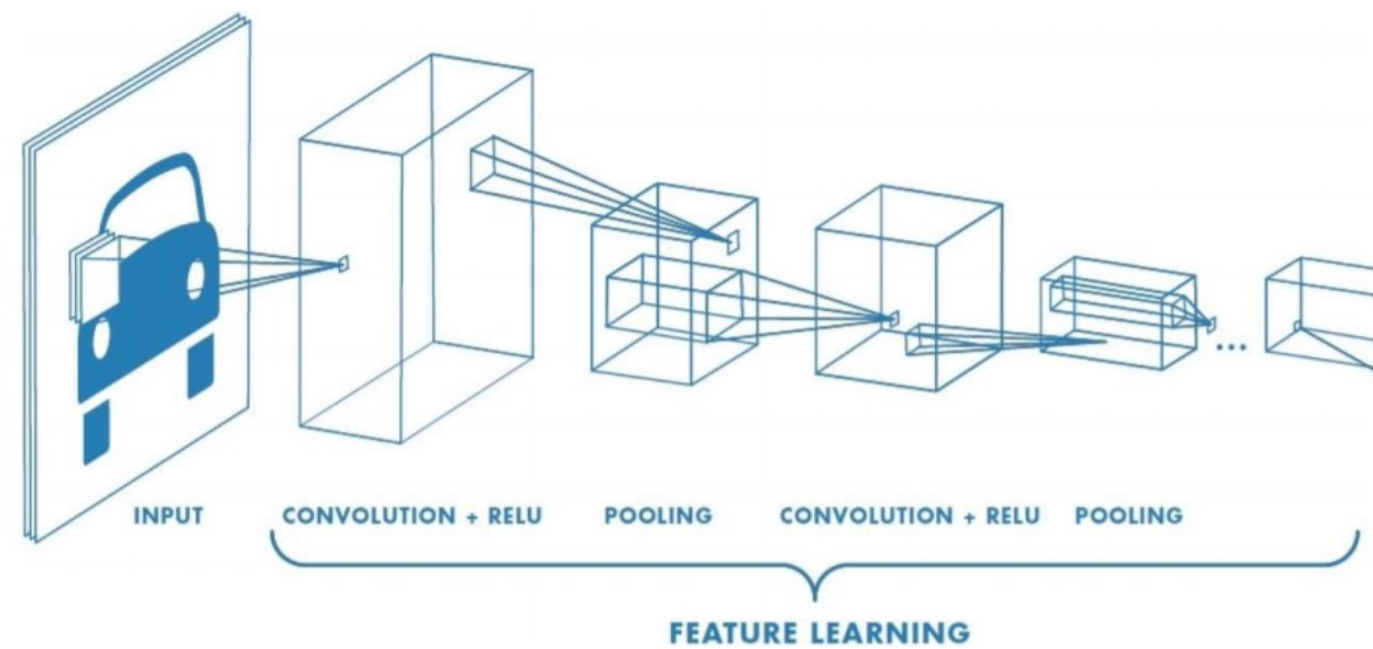
$$\text{softmax}(y_i) = \frac{e^{y_i}}{\sum_j e^{y_j}}$$

Now put it all together to perform **classification**

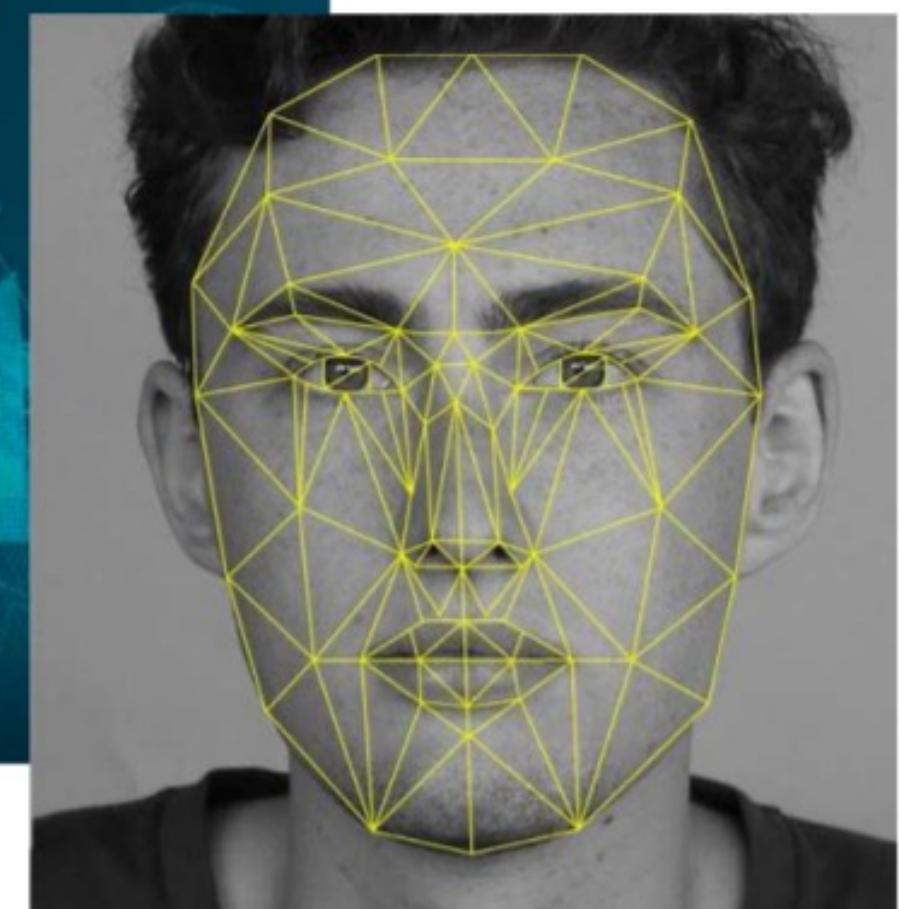
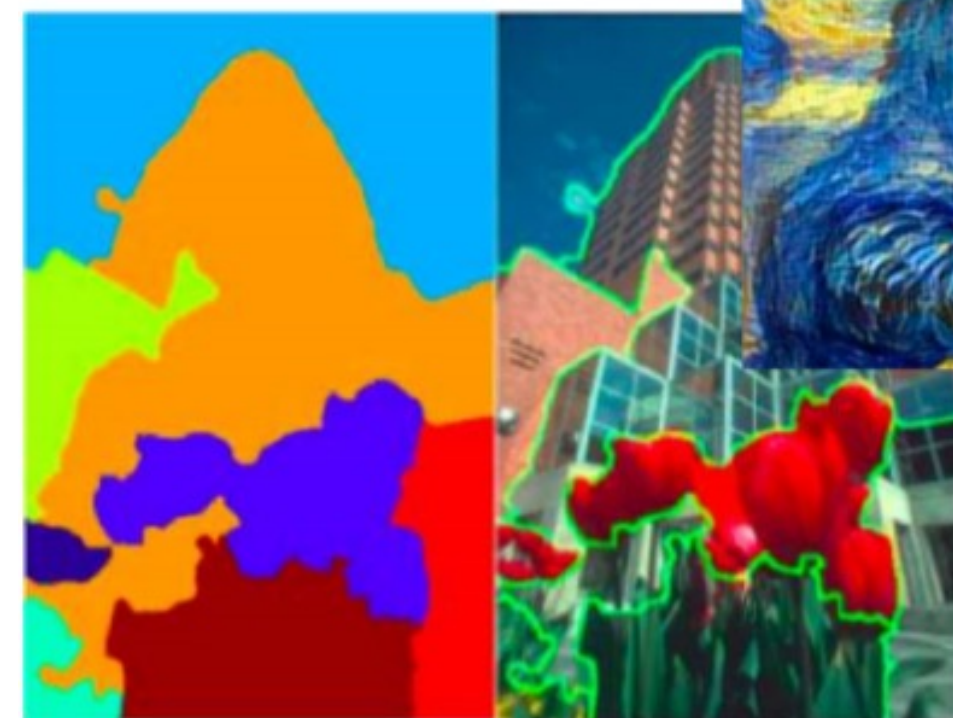
- Convolution to learn local features from input image
- Introduce non-linear activation
- Reduce dimensionality and preserve local translation invariance with pooling



One feature extraction tool, many tasks!



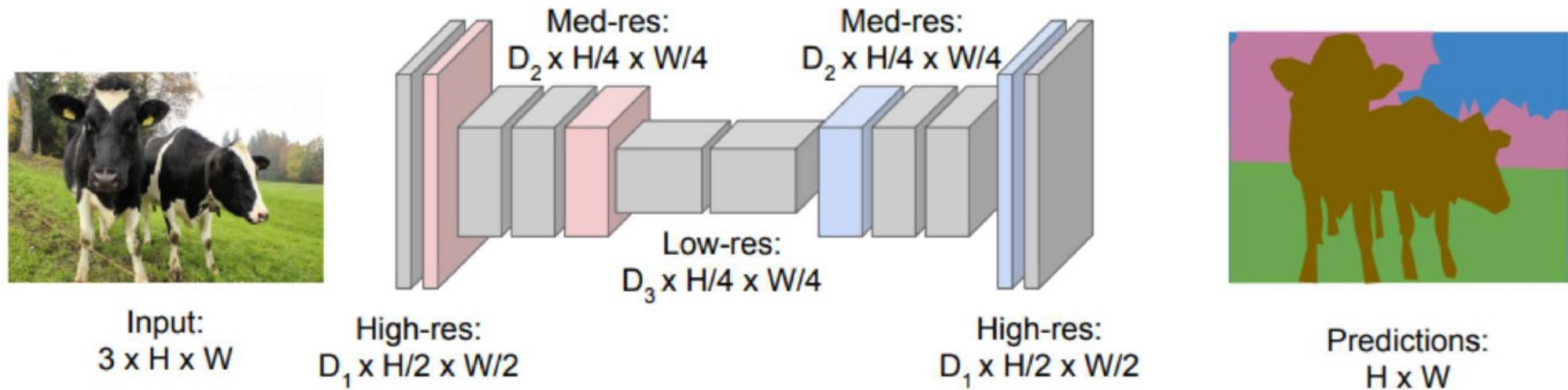
You can use feature learning part to perform a variety of tasks!



Example Task #1: Classification Chihuahua vs Muffin



Example Task #2: Semantic Segmentation

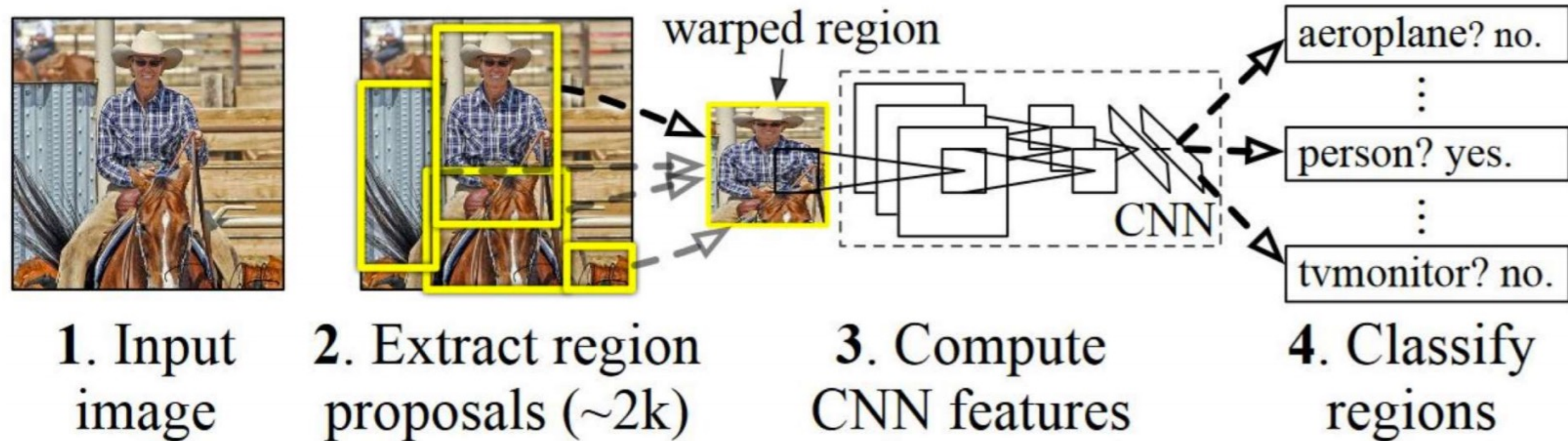


All convolutional layers with downsampling and upsampling operations

Example Task #2: Semantic Segmentation



Example Task #3: Object Detection

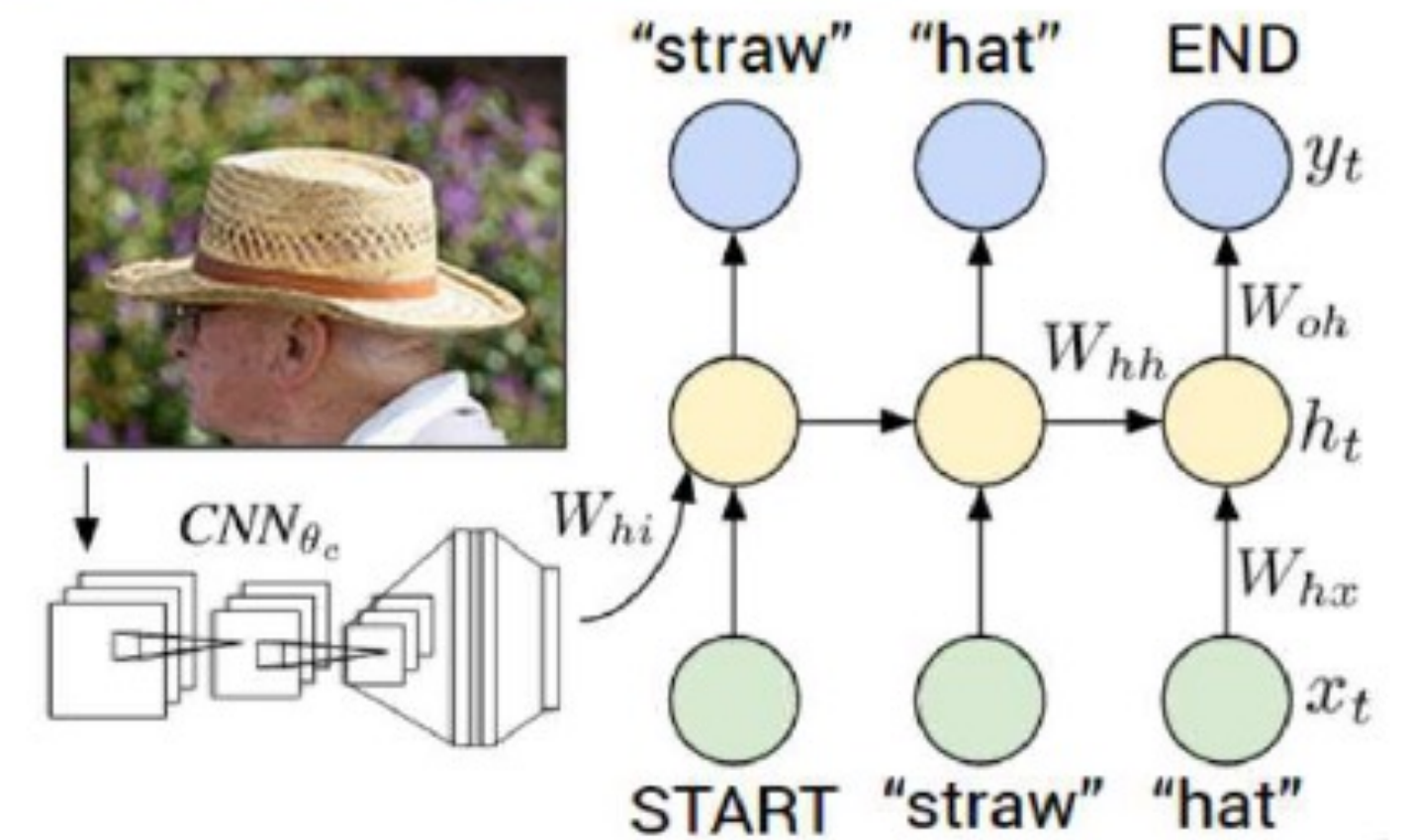
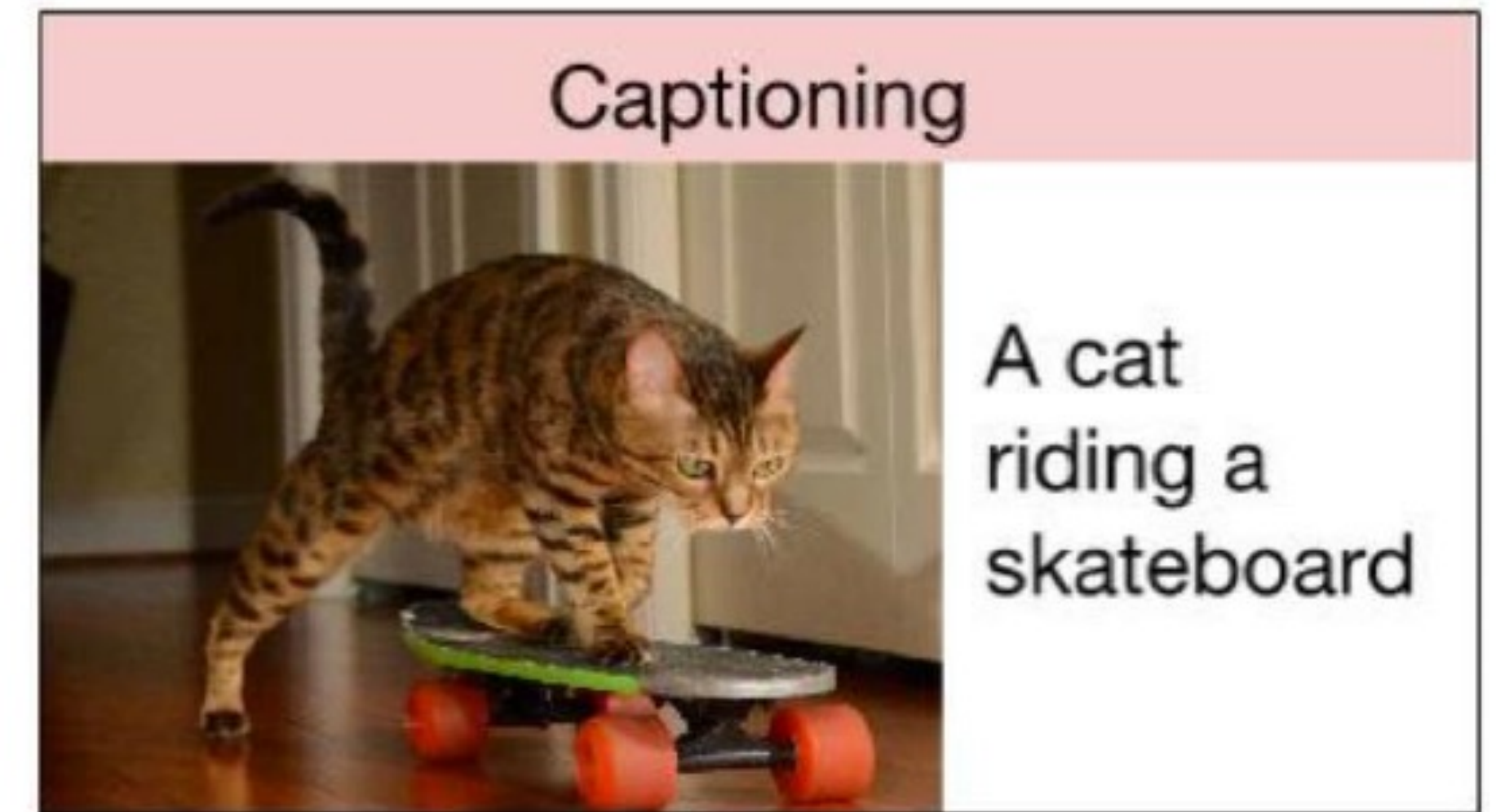


MIT Introduction to Deep Learning <http://introtodeeplearning.com>

1. Find regions that we think have objects
2. Classify such objects

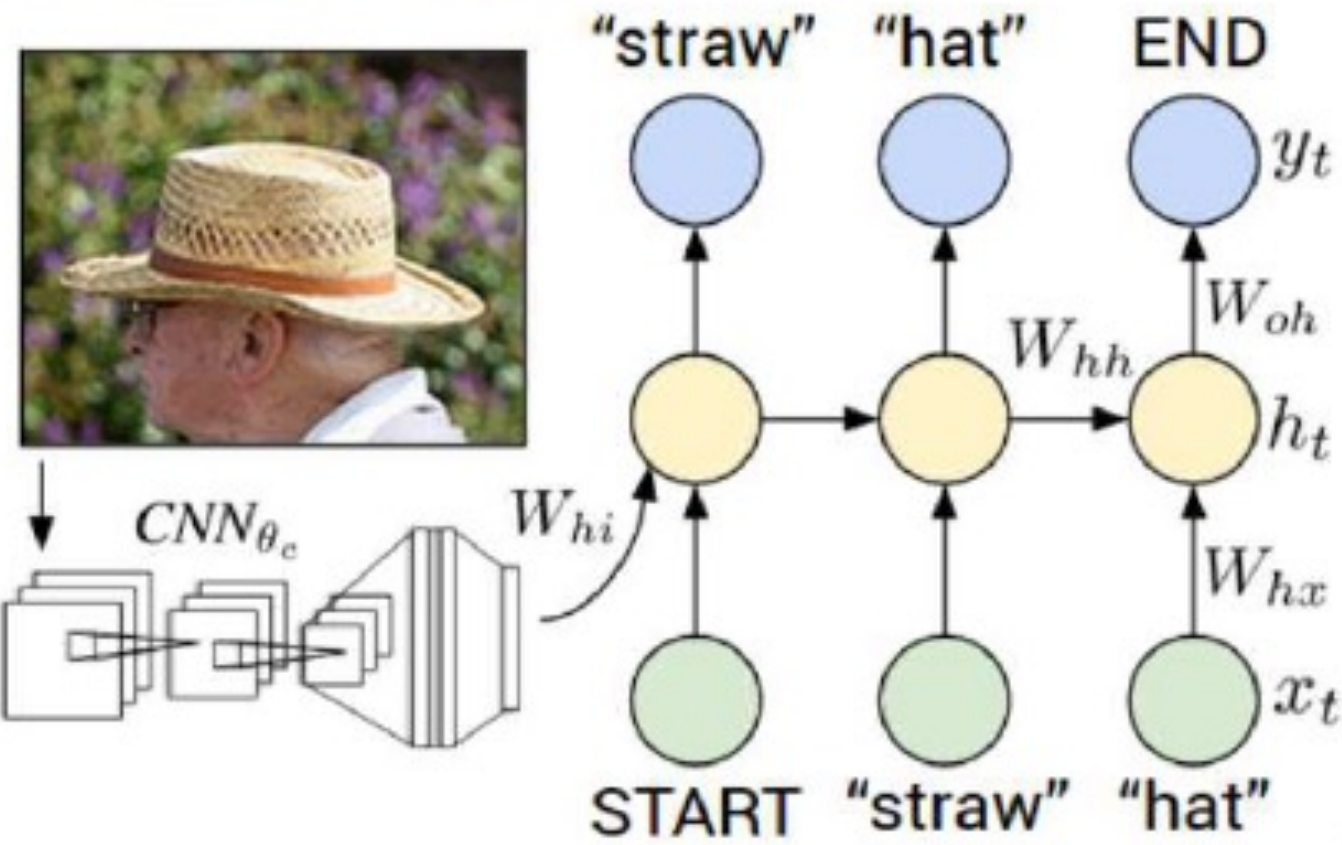
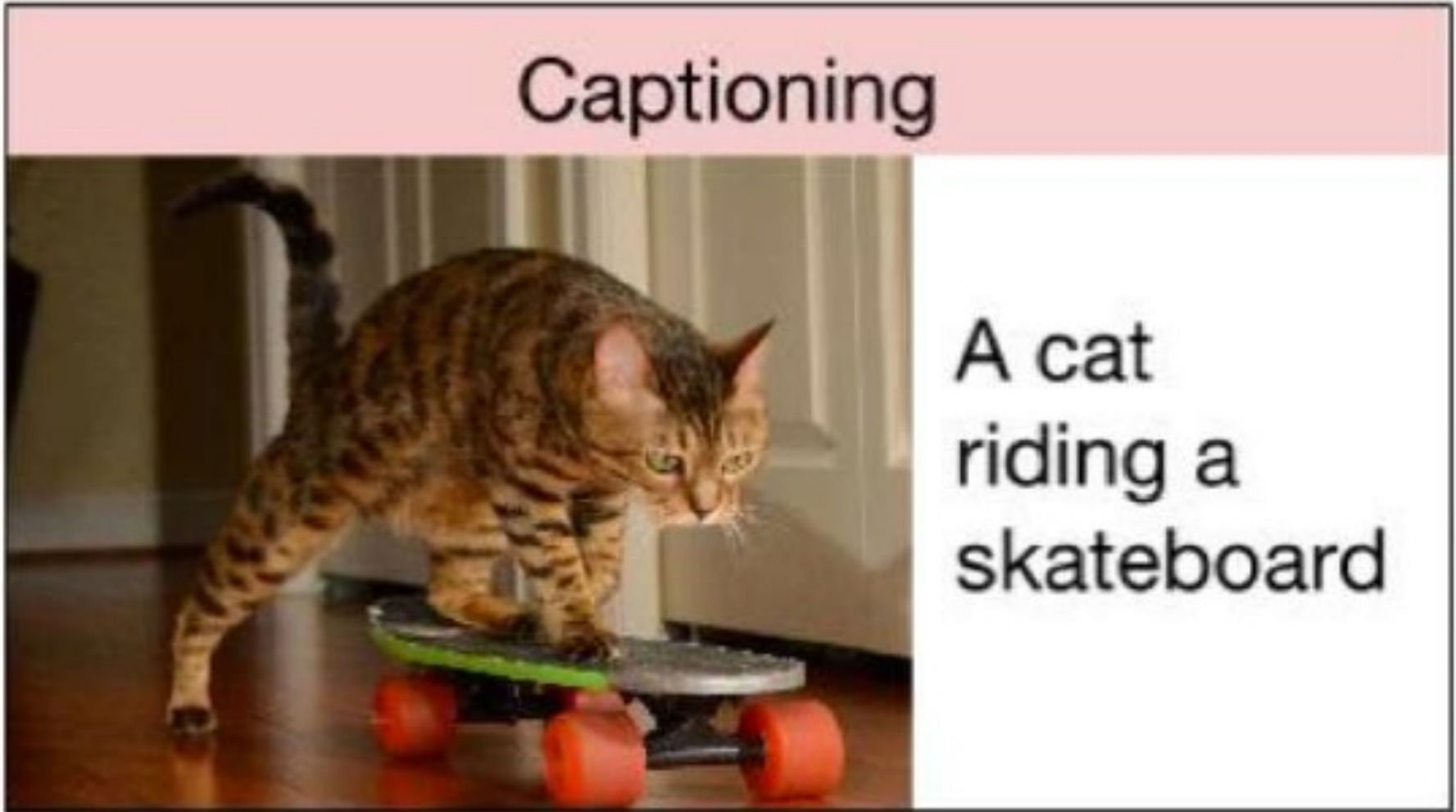
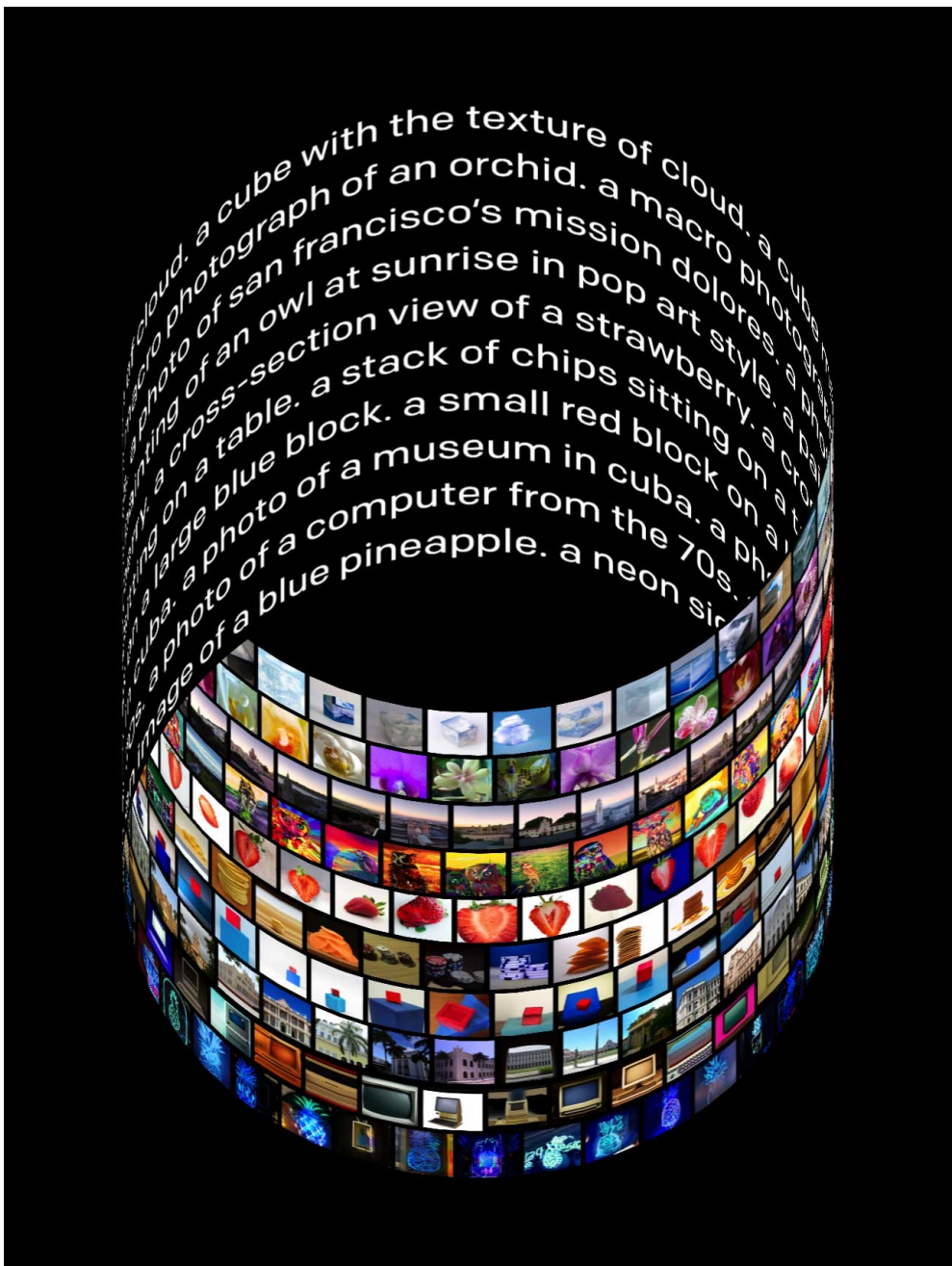
Example Task #4: Image Captioning

Combination of CNN and sequence learning

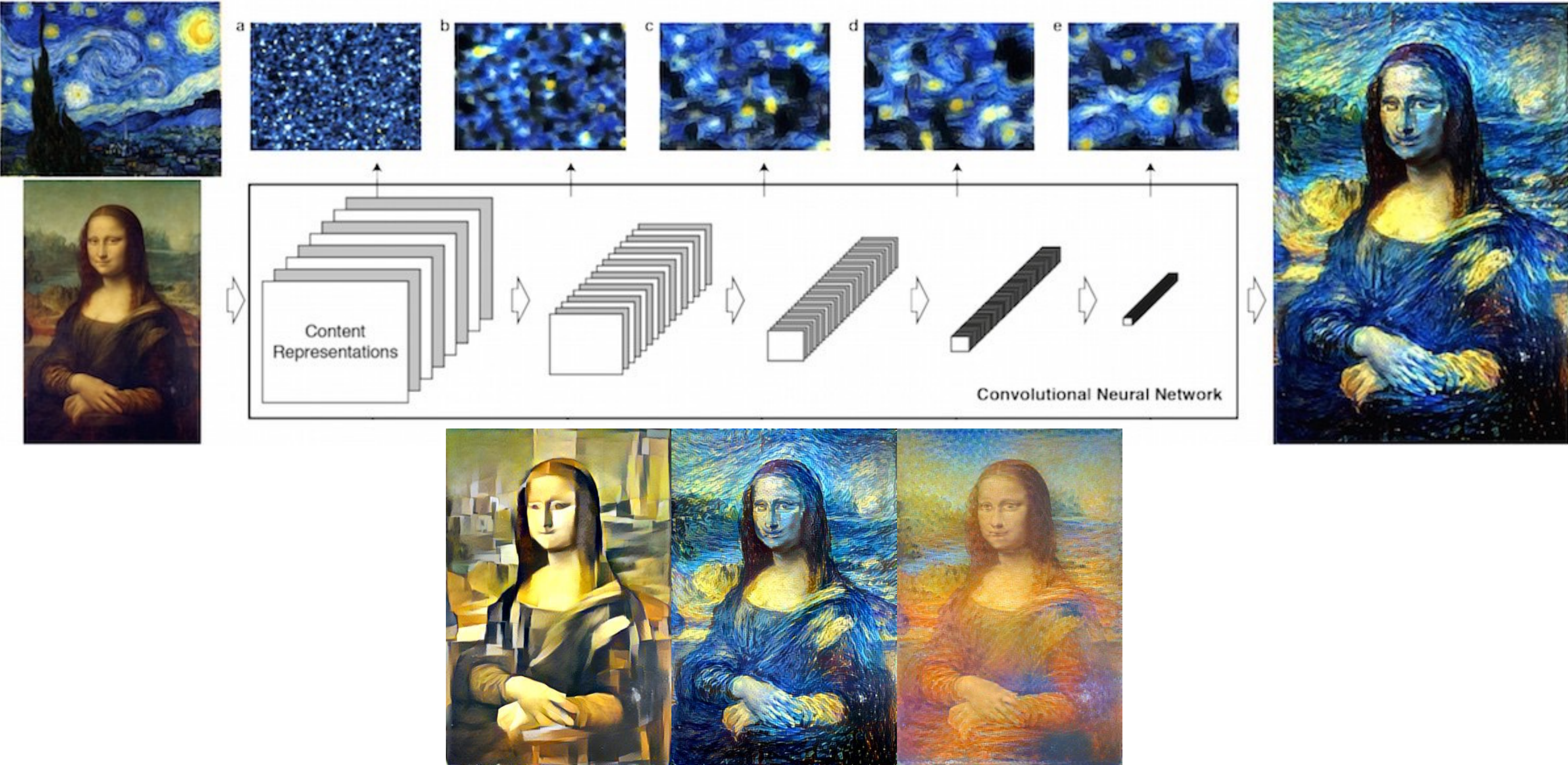


Example Task #4: Image Captioning

Combination of CNN and sequence learning



Example Task #5: Style Transfer



Example Task #5: Style Transfer



-<https://deepart.io/>

Example Task #5: Style Transfer



-<https://deepart.io/>

Example Task #5: Style Transfer



-<https://deepart.io/>

Example Task #5: Style Transfer



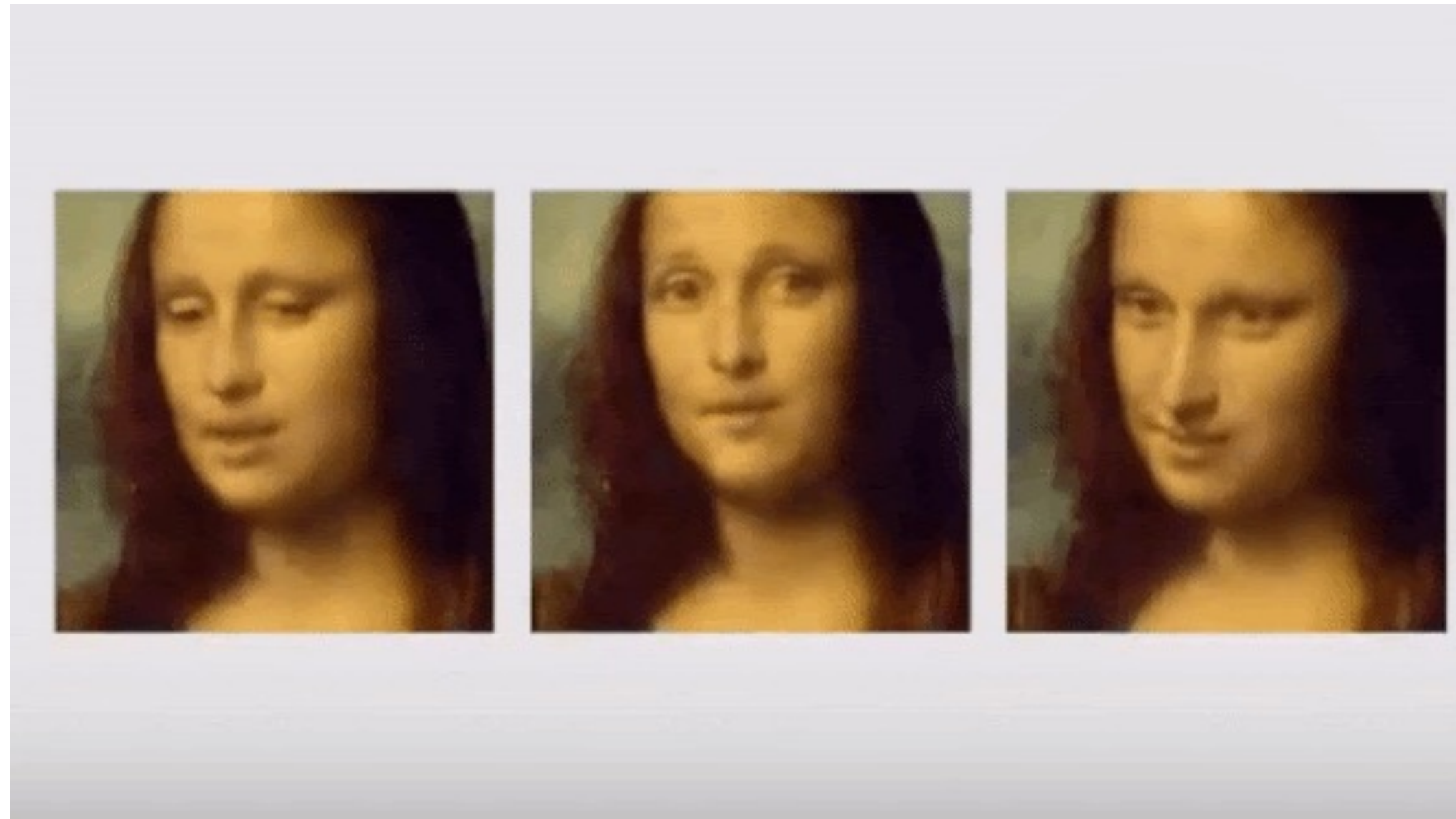
[-https://deepart.io/](https://deepart.io/)

Example Task #5: Style Transfer

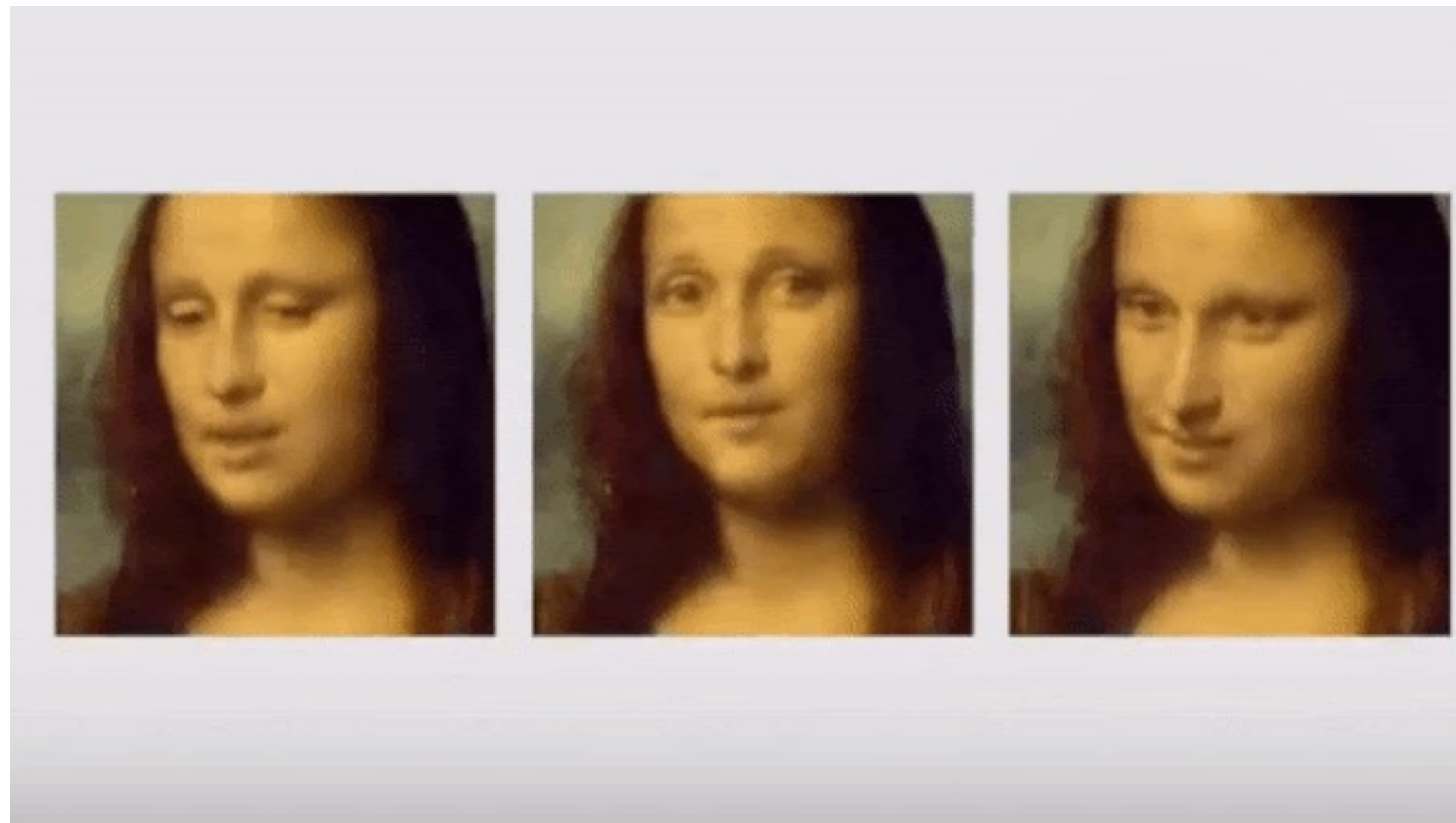


-<https://deepart.io/>

Example Task #5: Style Transfer



Example Task #5: Style Transfer



Some style transfer works

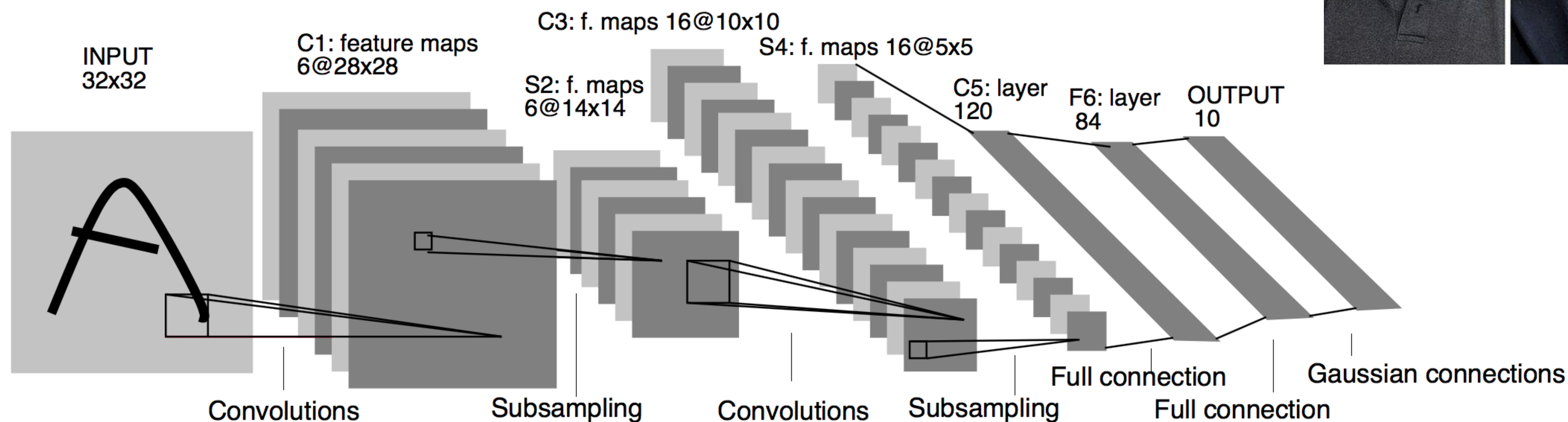
- Gatys, L. A., Ecker, A. S., & Bethge, M. (2016). Image style transfer using convolutional neural networks. In Proceedings of the IEEE conference on computer vision and pattern recognition (pp. 2414-2423).
- Johnson, J., Alahi, A., & Fei-Fei, L. (2016, October). Perceptual losses for real-time style transfer and super-resolution. In European conference on computer vision (pp. 694-711). Springer, Cham.
- Luan, F., Paris, S., Shechtman, E., & Bala, K. (2017). Deep photo style transfer. In Proceedings of the IEEE conference on computer vision and pattern recognition (pp. 4990-4998).
- i, X., Liu, S., Kautz, J., & Yang, M. H. (2019). Learning linear transformations for fast image and video style transfer. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (pp. 3809-3817)

An overview of famous architectures and datasets

LeNet (LeNet-5)

Once upon a time... LeNet [LeCun et al., 1998]

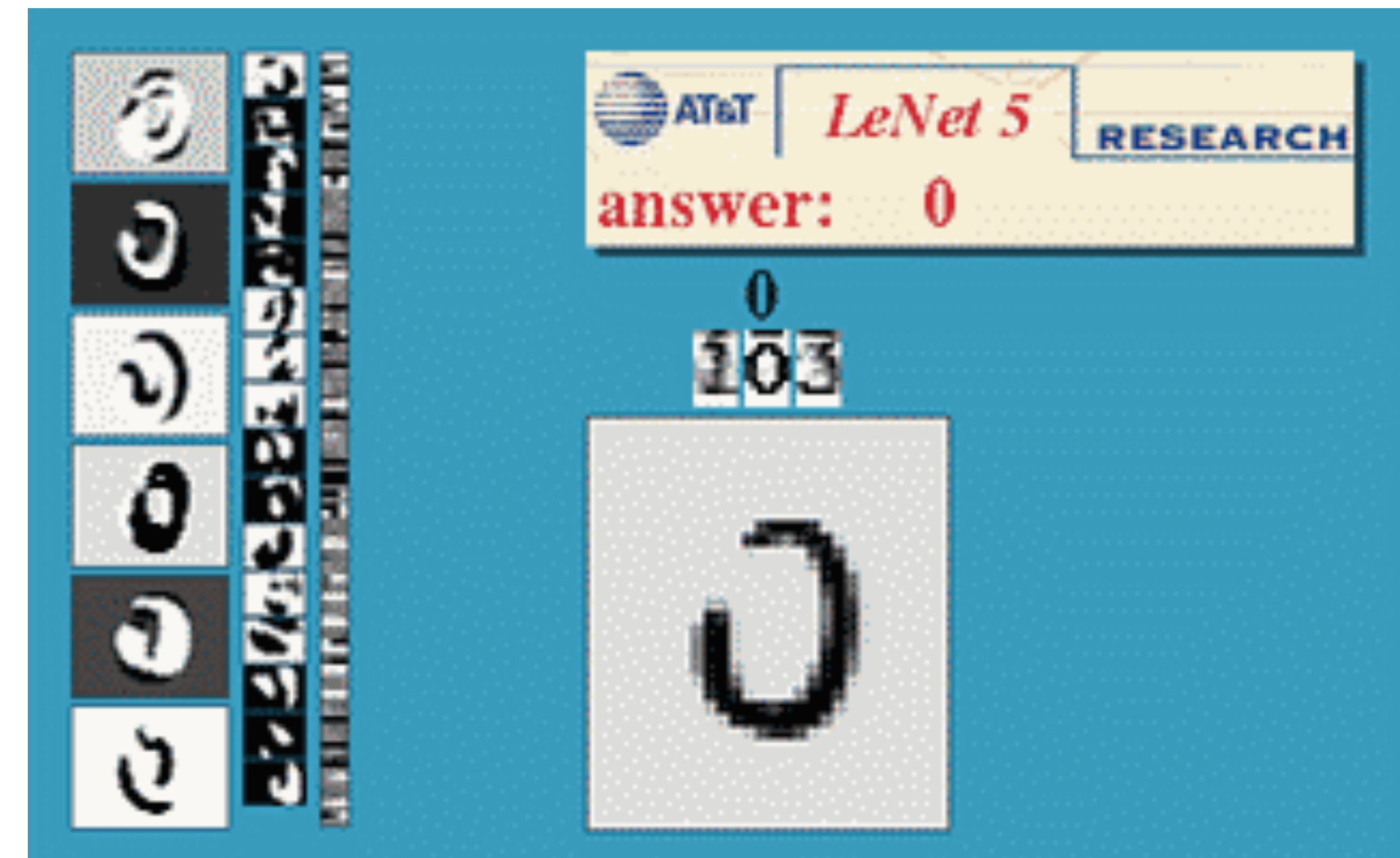
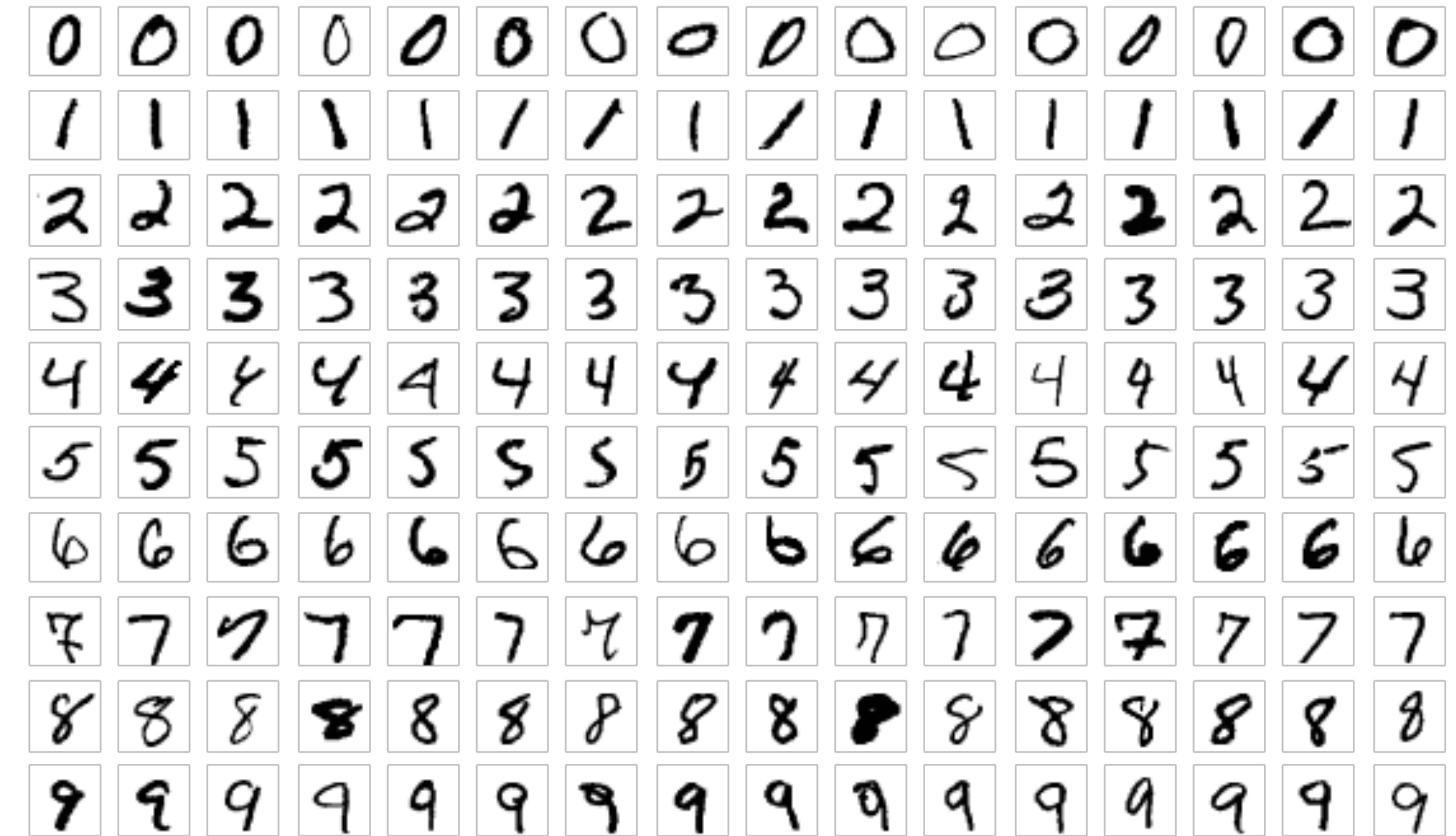
- 5x5 conv filters with stride 1
- 2x2 pooling with stride 2



CONV-POOL-CONV-POOL-FC-FC architecture

MNIST (1998)

- Handwritten digits recognition
- 10 classes (of course!)
- 60k training images (28x28), 10k test images
- Grayscale images
- MNIST: Modified National Institute of Standards and Technology database
- Original NIST: training dataset was taken from American Census Bureau employees, while the testing dataset was taken from American high school students
- Available on Tensorflow
- LeNet on MNIST was the first real NN application!
- MNIST is not much of a challenge anymore...



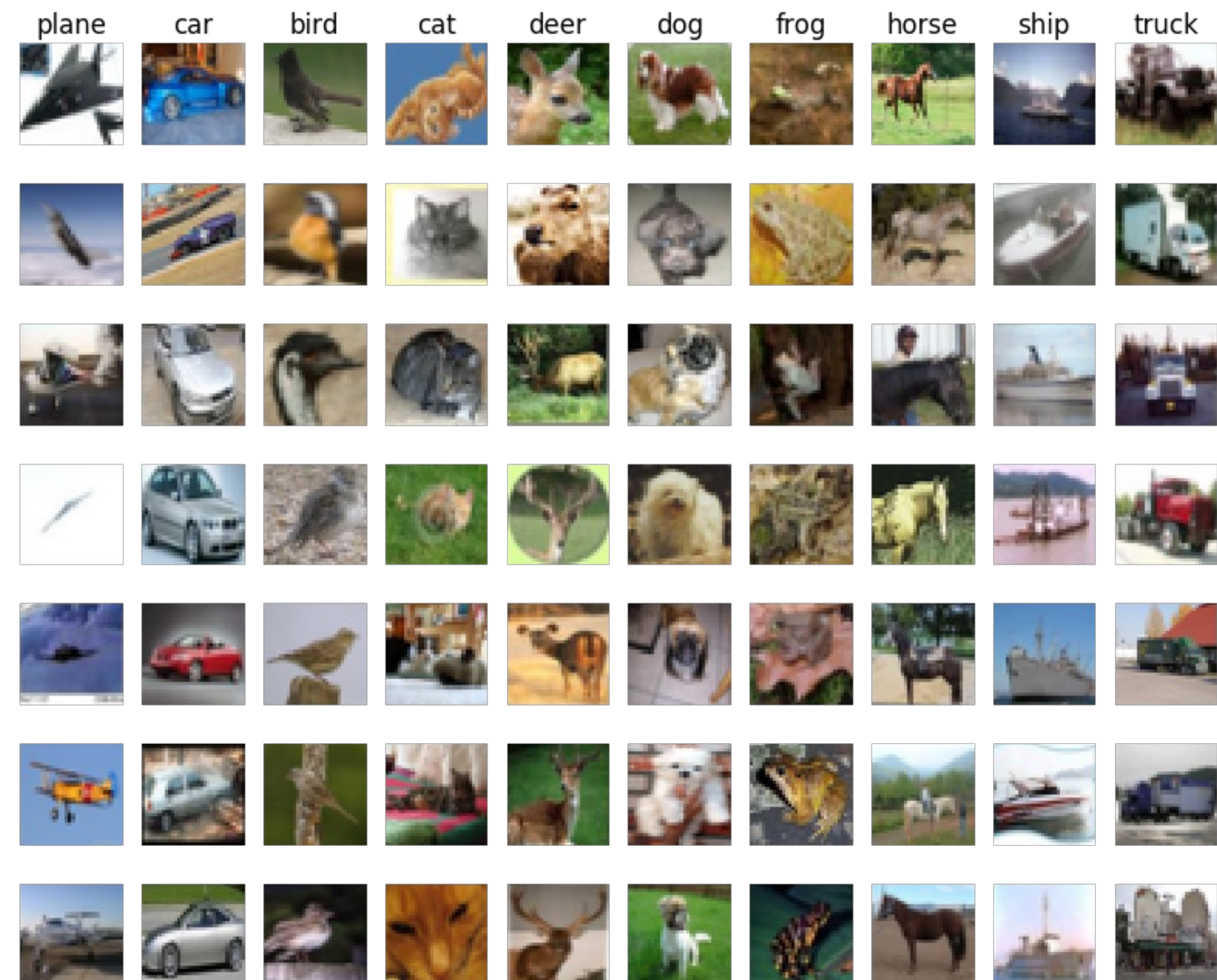
Fashion MNIST (2017)

- Clothing classification
- 10 classes
- 60k training images (28x28), 10k test images
- Grayscale images
- Available on Tensorflow
- More challenging than MNIST



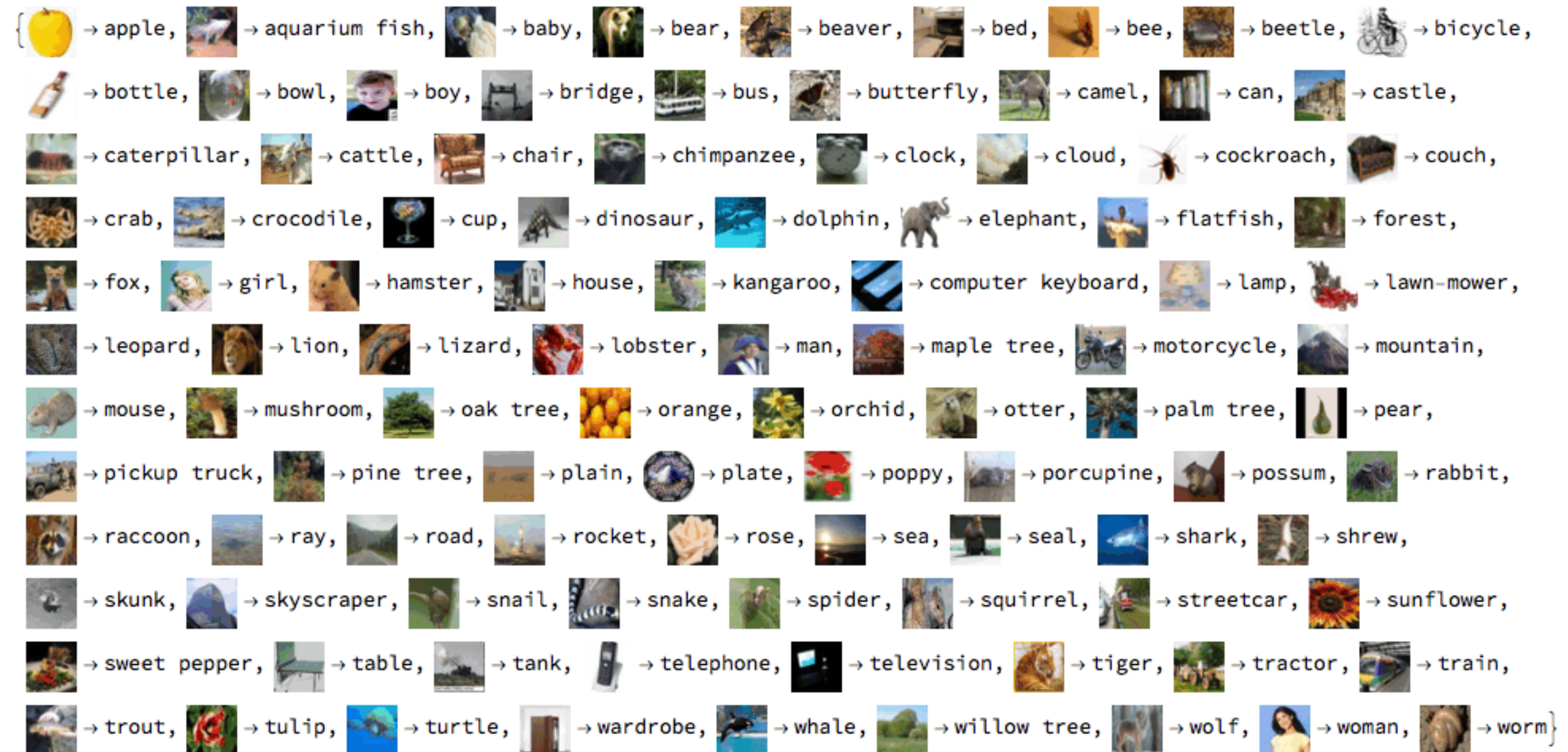
Cifar10 (2010)

- 10 classes
- 60k images (32x32)
- Balanced dataset: 6000 images each
- Colour images
- It is a labeled subset of the 80 million tiny image dataset
- Available on Tensorflow
- Widely used for research nowadays



Cifar100 (2010)

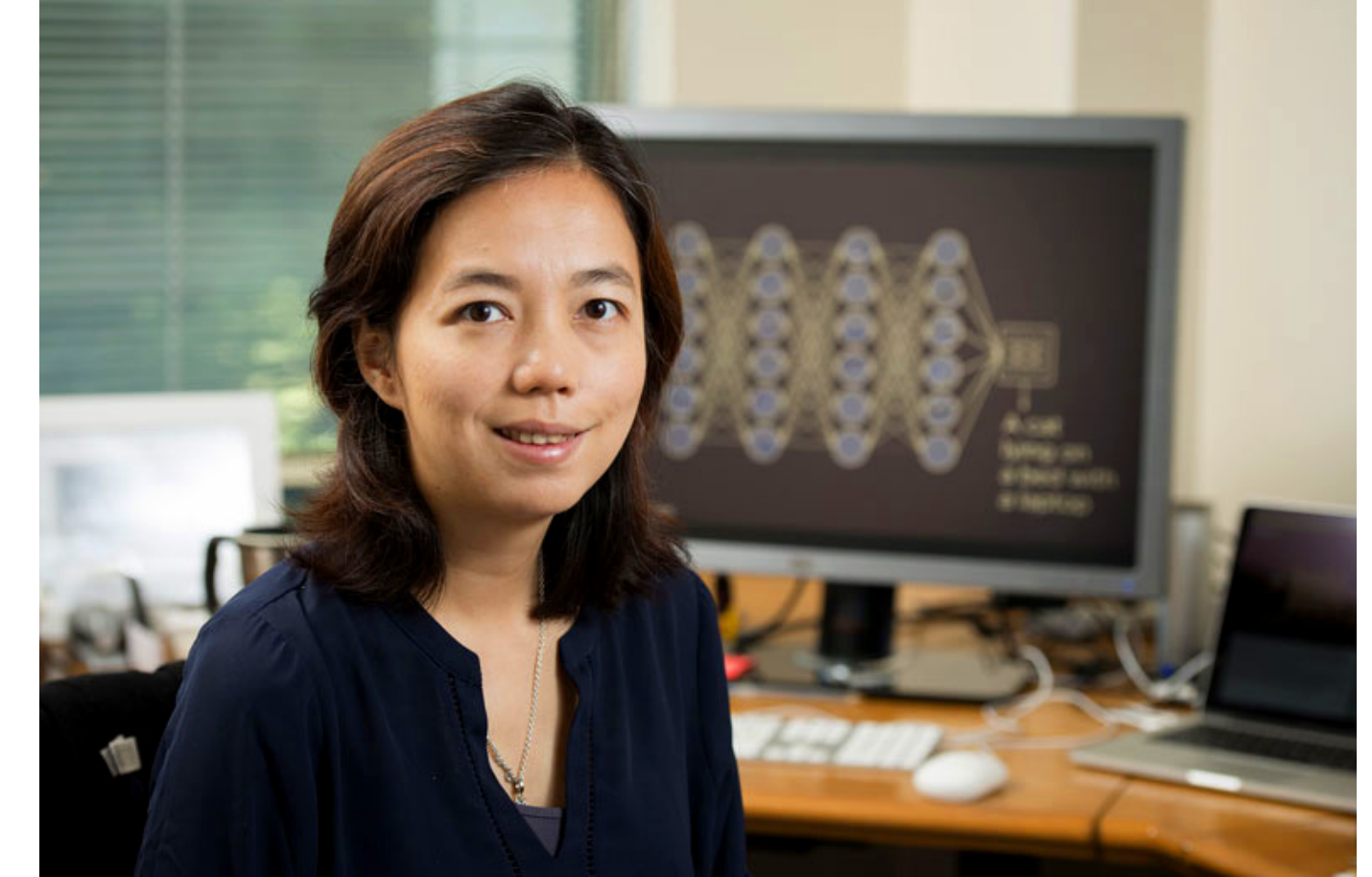
- 100 classes
- 60k images (32x32)
- Balanced dataset: 600 images each
- Colour images
- It is a labeled subset of the 80 million tiny image dataset
- Available on Tensorflow
- Widely used for research nowadays (less than Cifar10)



Imagenet (2009)

- 21841 classes
- 14M images with different dimensions and resolutions (many apply resize to 256x256)
- Unbalanced dataset
- Colour images
- Lead developer Fei-Fei Li

IMAGENET



"Elongated crescent-shaped yellow fruit with soft sweet flesh"





ImageNet Large Scale Visual Recognition Challenges

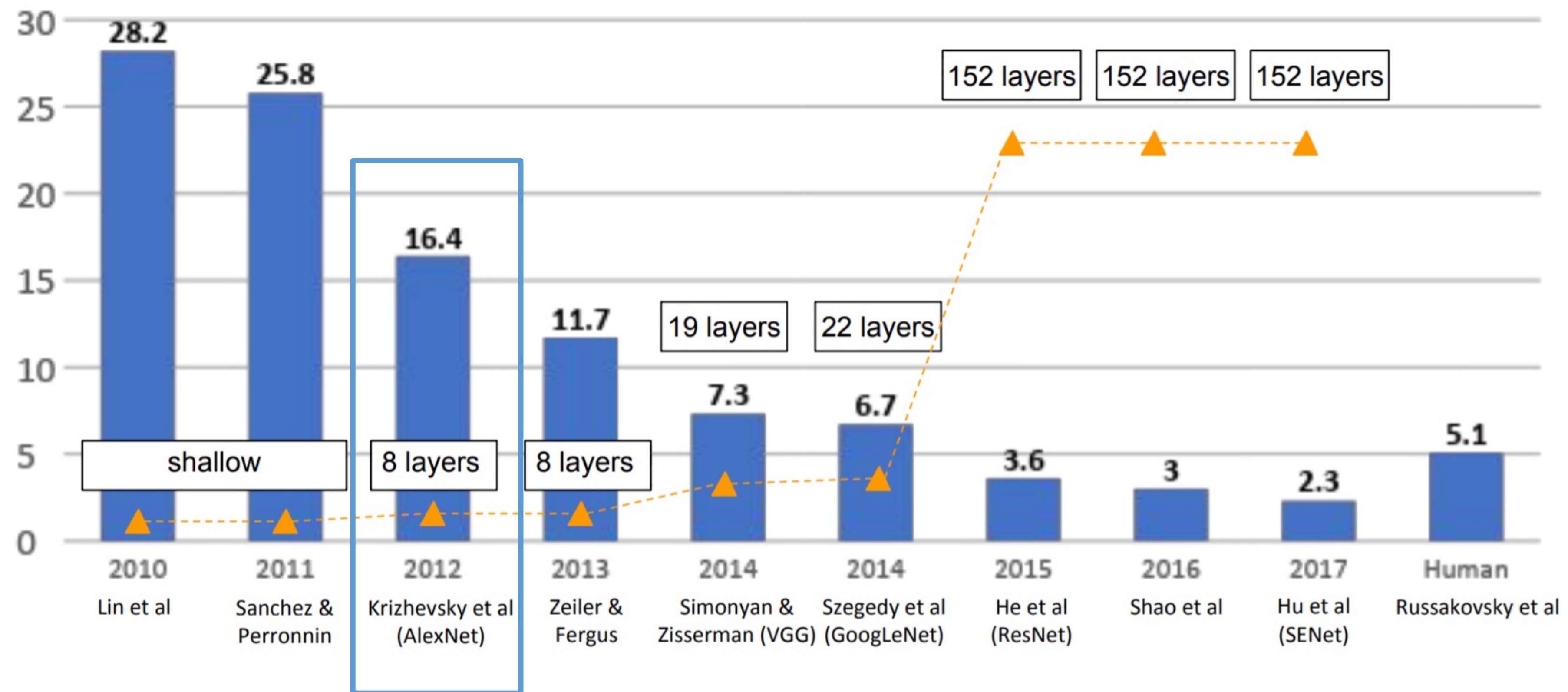
Classification task: produce a list of object categories present in image. 1000 categories.

“Top 5 error”: rate at which the model does not output correct label in top 5 predictions

An overview on the most famous architectures

Imagenet – visual recognition challenge with 1000 classes.

Winners:



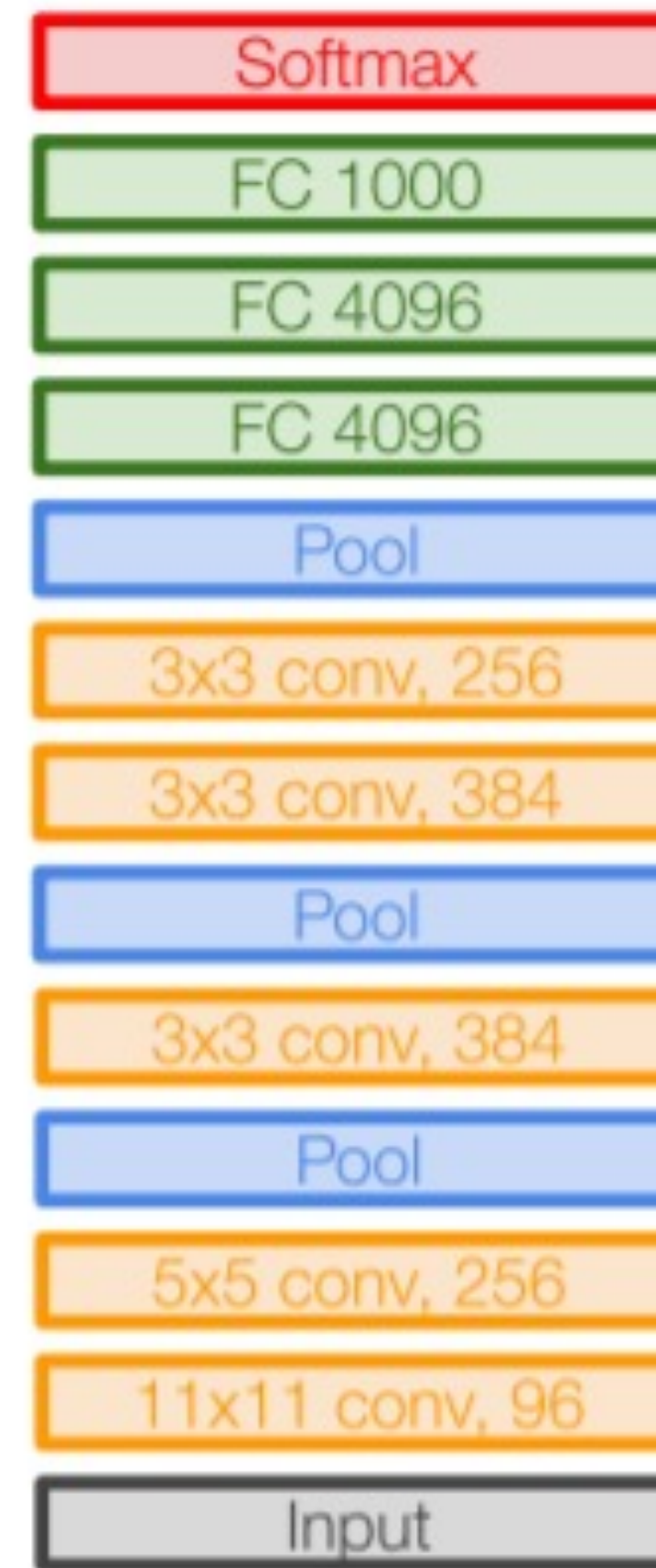
First CNN-based winner

AlexNet

AlexNet (2012) renewed interest in CNN.

It uses:

- RELU (first use)
- Layer normalization
- Dropout
- MaxPooling
- Momentum
- Data augmentation in training



AlexNet

Full (simplified) AlexNet architecture:

[227x227x3] INPUT

[55x55x96] CONV1: 96 11x11 filters at stride 4, pad 0

[27x27x96] MAX POOL1: 3x3 filters at stride 2

[27x27x96] NORM1: Normalization layer

[27x27x256] CONV2: 256 5x5 filters at stride 1, pad 2

[13x13x256] MAX POOL2: 3x3 filters at stride 2

[13x13x256] NORM2: Normalization layer

[13x13x384] CONV3: 384 3x3 filters at stride 1, pad 1

[13x13x384] CONV4: 384 3x3 filters at stride 1, pad 1

[13x13x256] CONV5: 256 3x3 filters at stride 1, pad 1

[6x6x256] MAX POOL3: 3x3 filters at stride 2

[4096] FC6: 4096 neurons

[4096] FC7: 4096 neurons

[1000] FC8: 1000 neurons (class scores)

AlexNet

AlexNet (2012) renewed interest in CNN.

It uses:

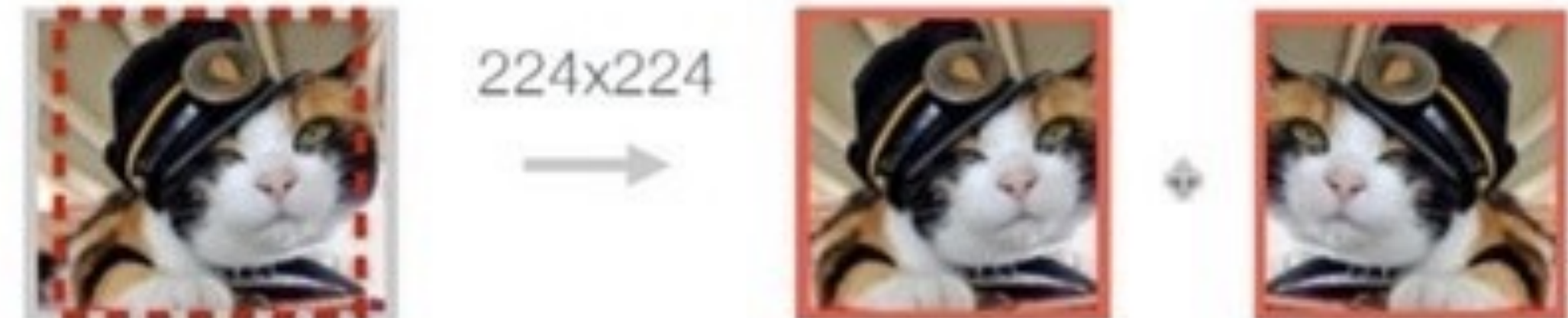
- RELU (first use)
- Layer normalization
- Dropout
- MaxPooling
- Momentum
- Data augmentation in training

Data Augmentation:

a. No augmentation (= 1 image)



b. Flip augmentation (= 2 images)



c. Crop+Flip augmentation (= 10 images)



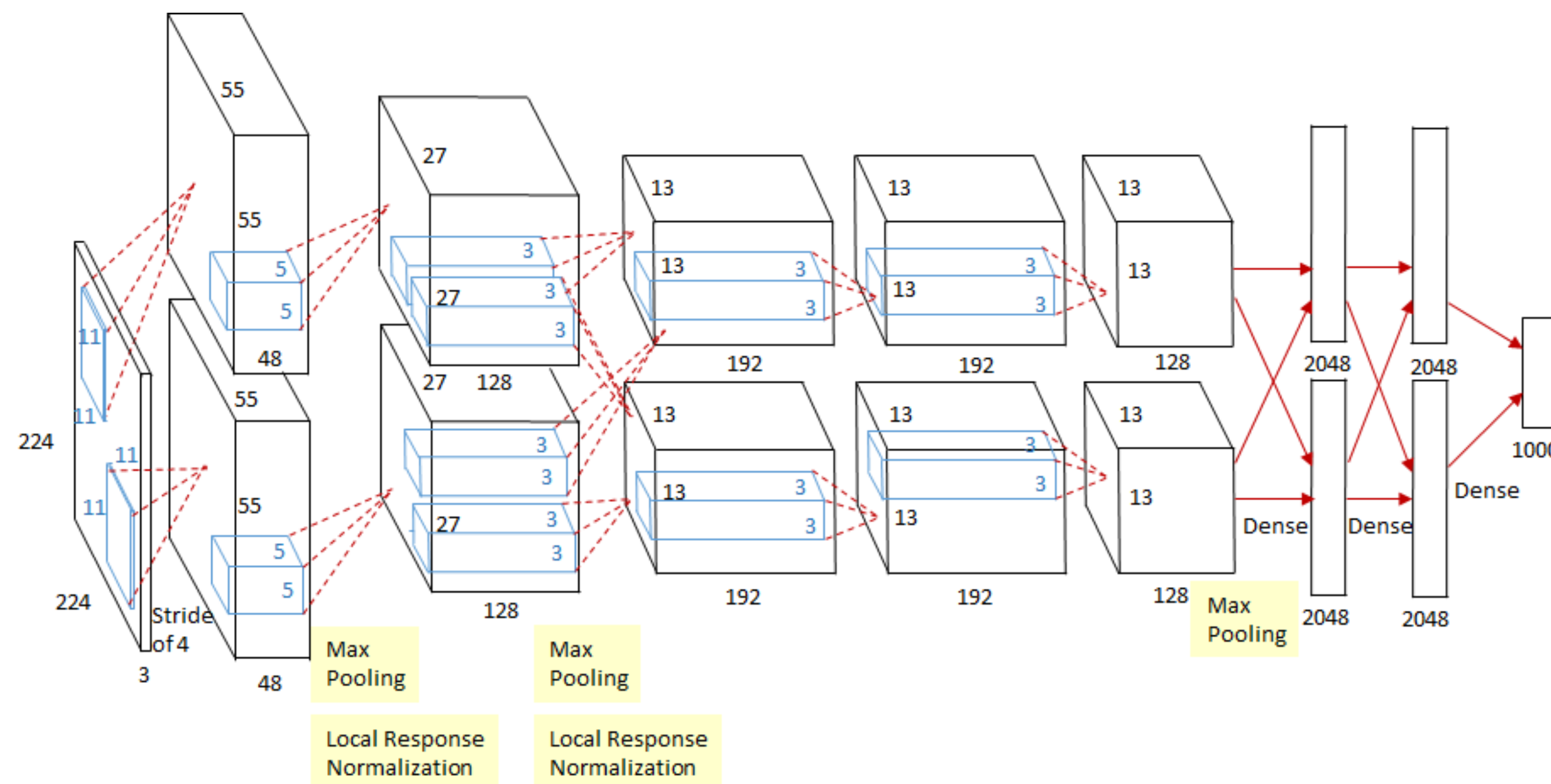
AlexNet

AlexNet (2012) renewed interest in CNN.

It uses:

- RELU (first use)
- Layer normalization
- Dropout
- MaxPooling
- Momentum
- **Data augmentation in training**

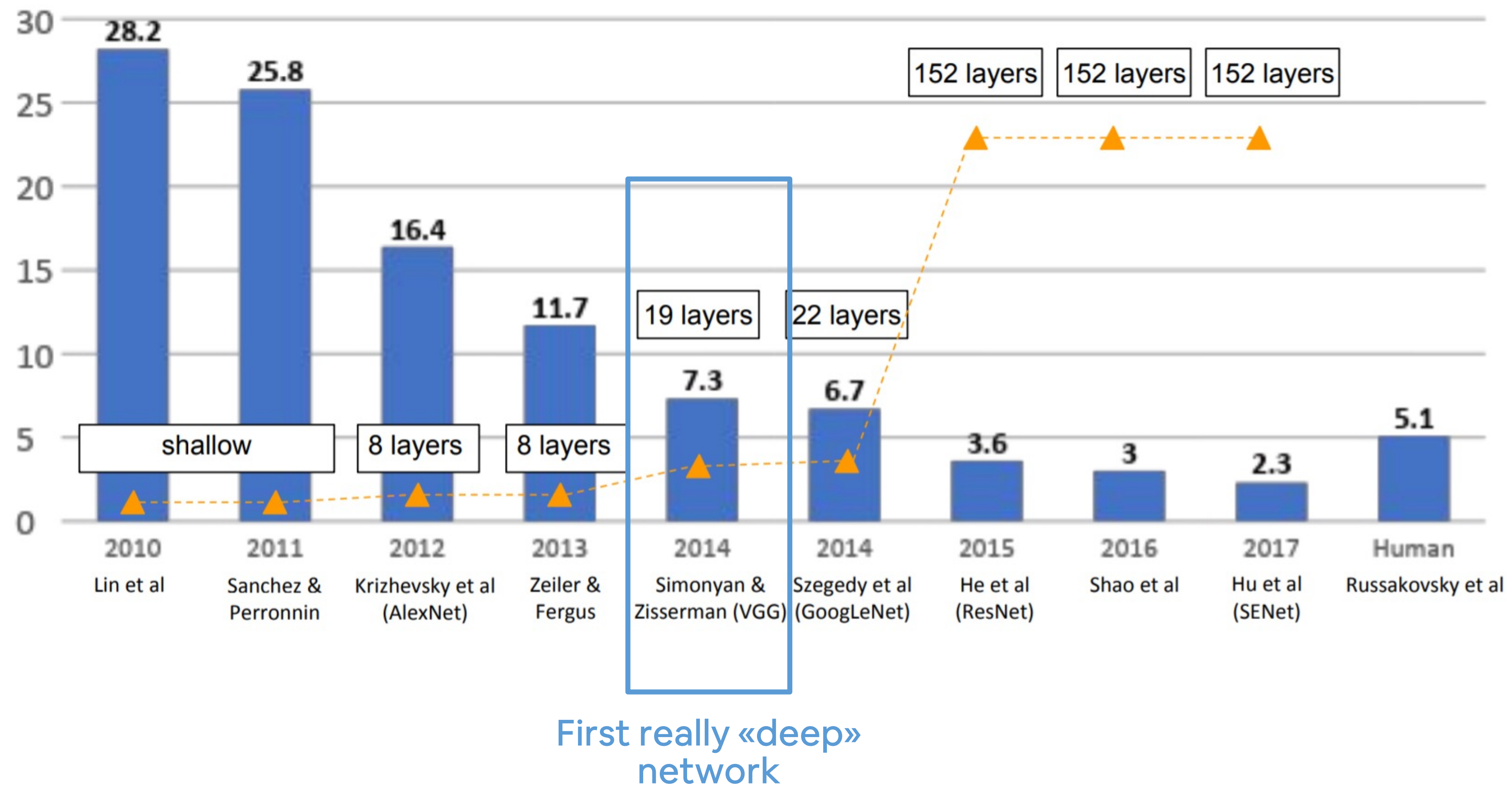
62.3 million parameters: the original paper showed that the depth of the model was essential for its high performance, which was computationally expensive, but made feasible due to the utilization of (GPUs) during training.



An overview on the most famous architectures

Imagenet – visual recognition challenge with 1000 classes.

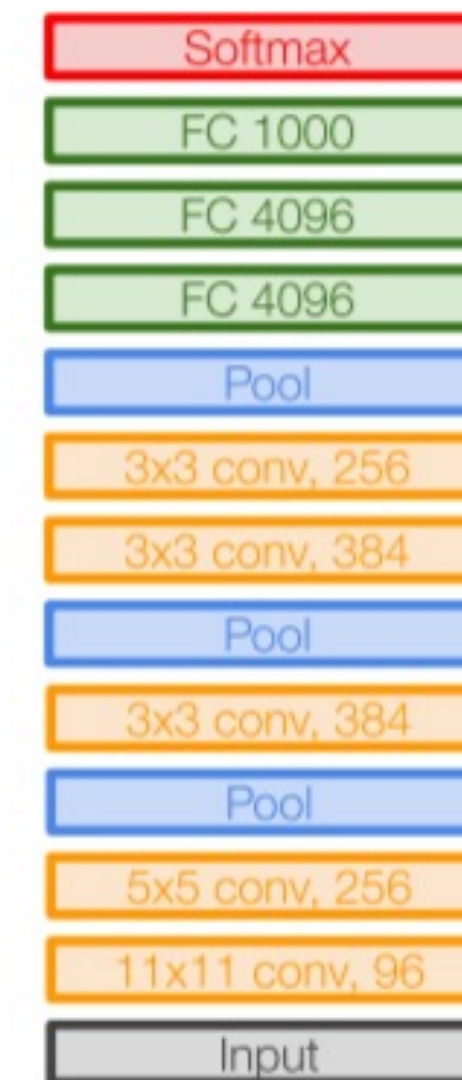
Winners:



VGG

VGG (2014):

- many small convolutional filters stacked together before pooling
- very deep (at the time) with 16/19 layers



AlexNet

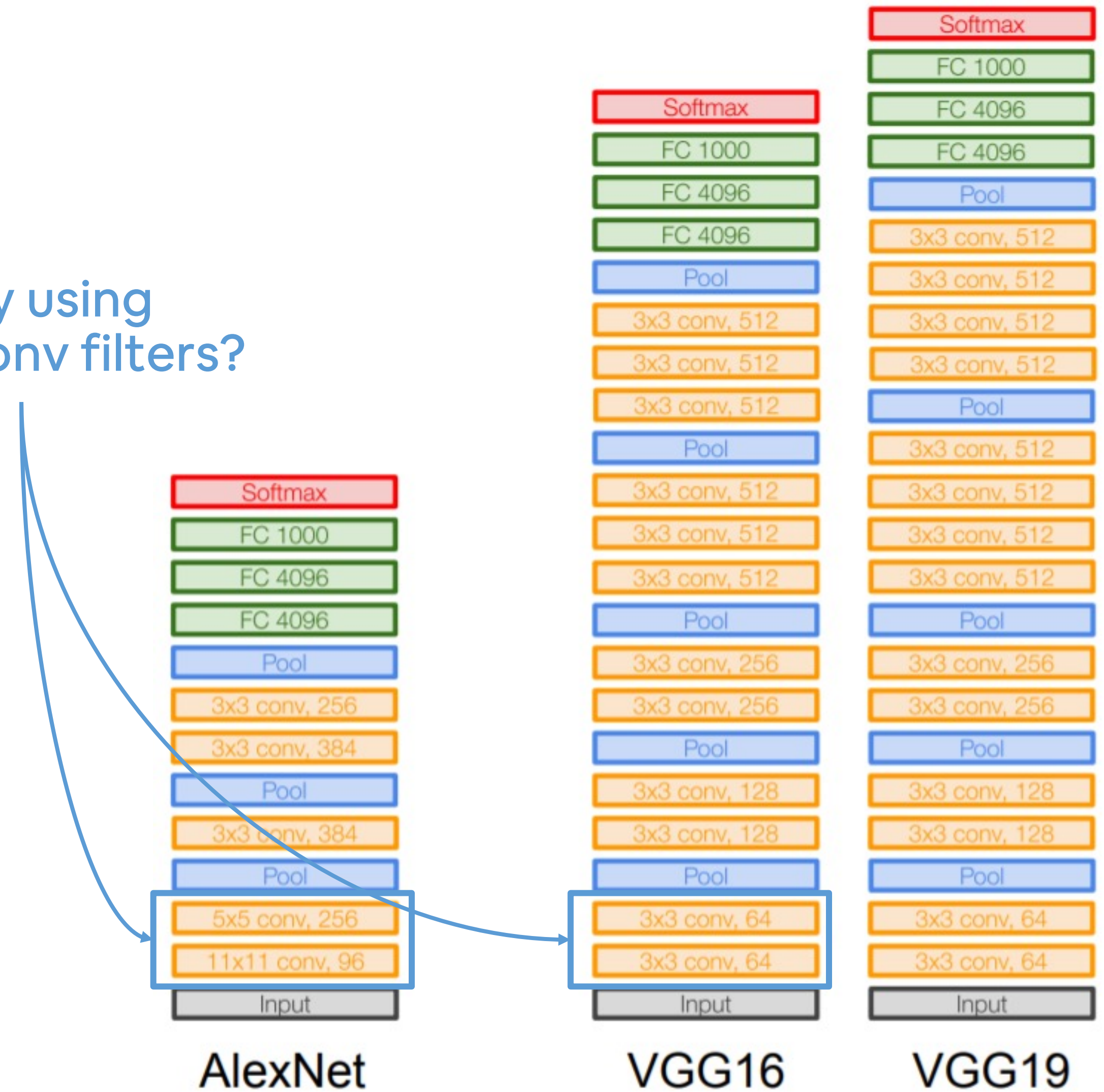


VGG16

VGG19

VGG

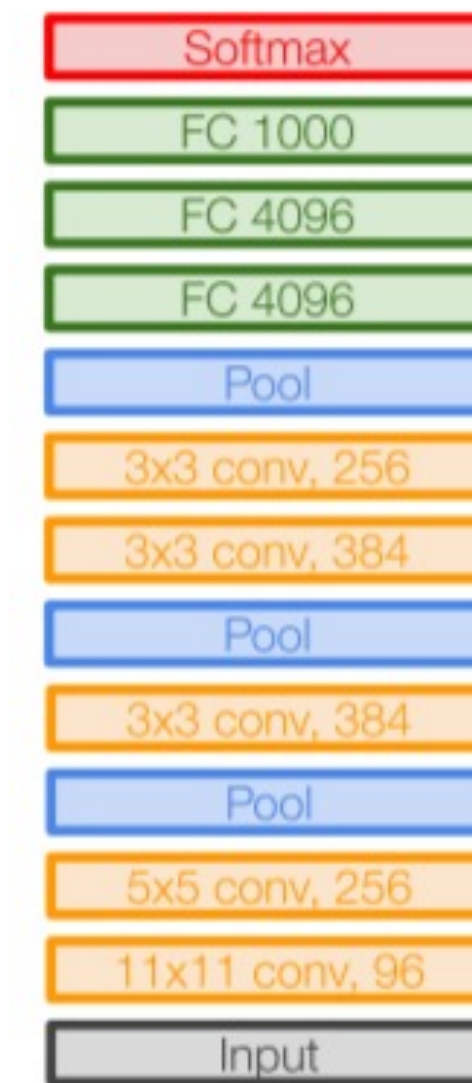
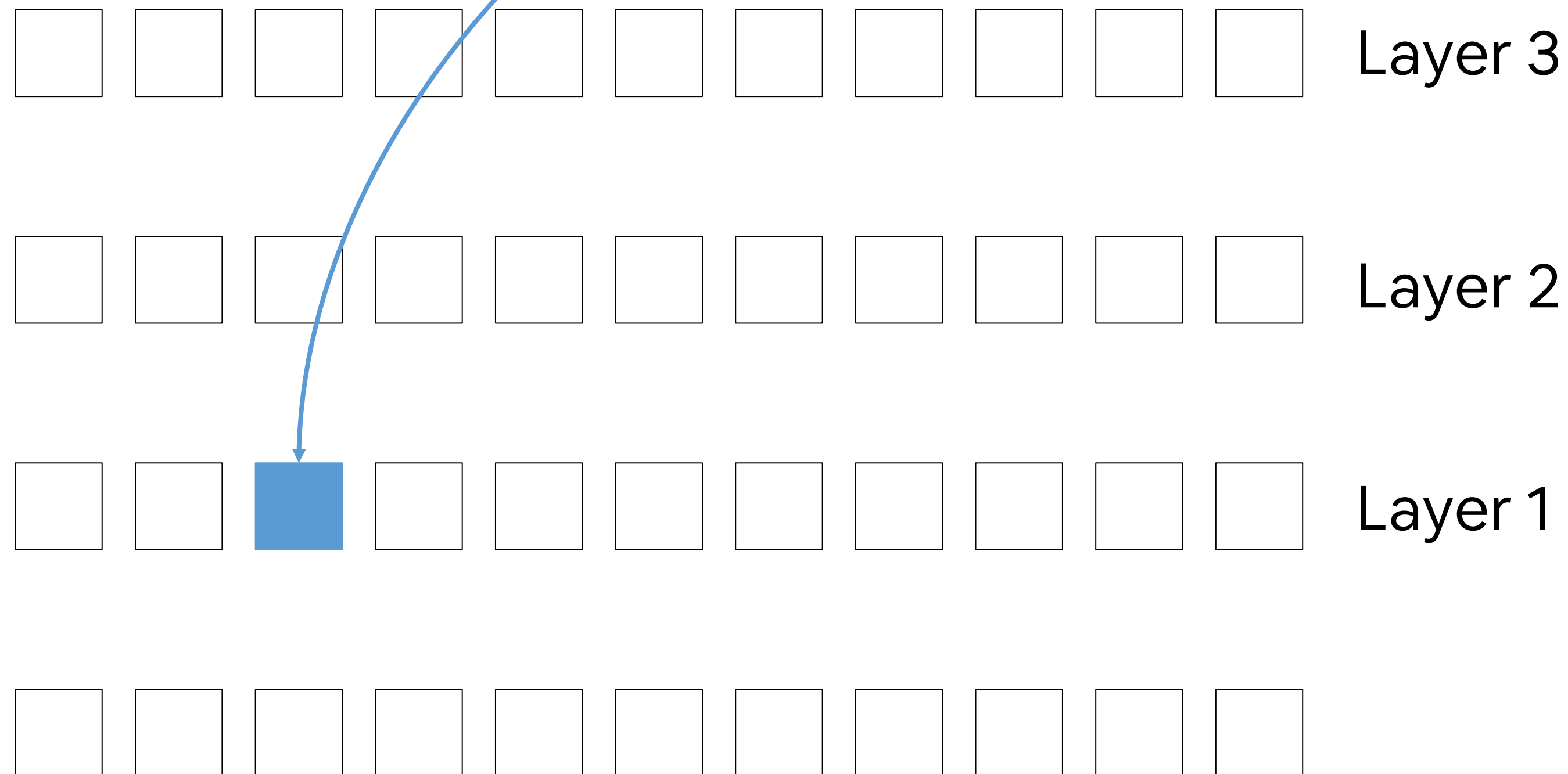
Wait. Why using smaller conv filters?



VGG

Consider a stack of three 3x3 conv layers.

Which is the receptive field of this hidden unit?



AlexNet



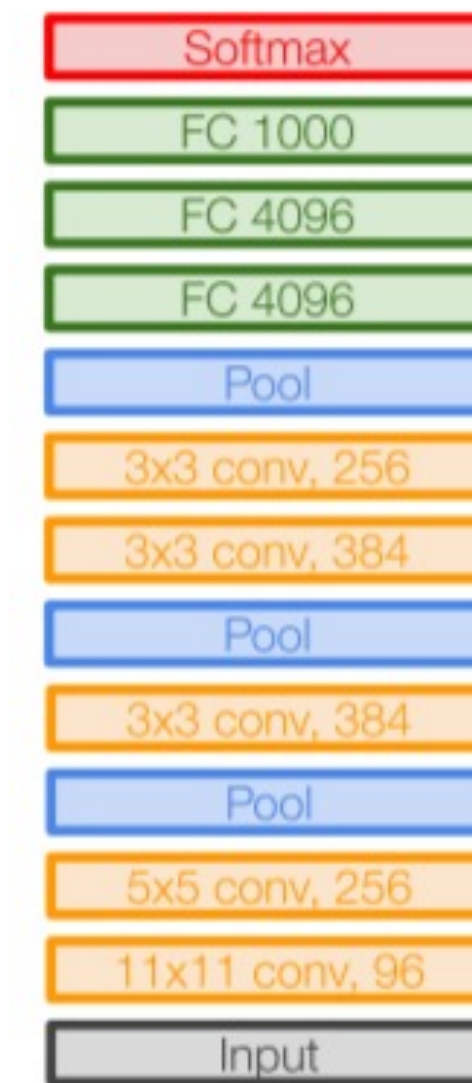
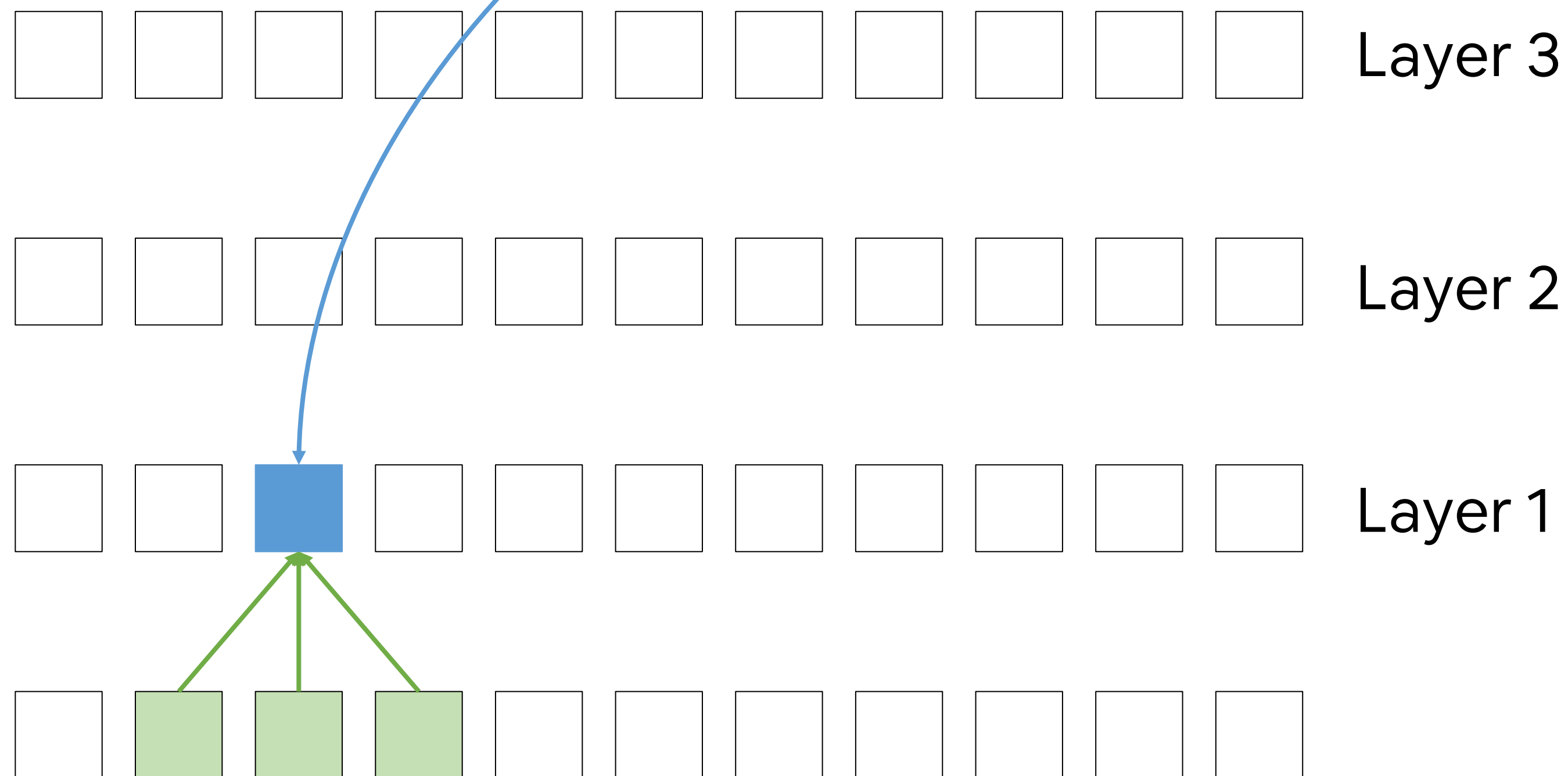
VGG16

VGG19

VGG

Consider a stack of three 3x3 conv layers.

Which is the receptive field of this hidden unit?



AlexNet



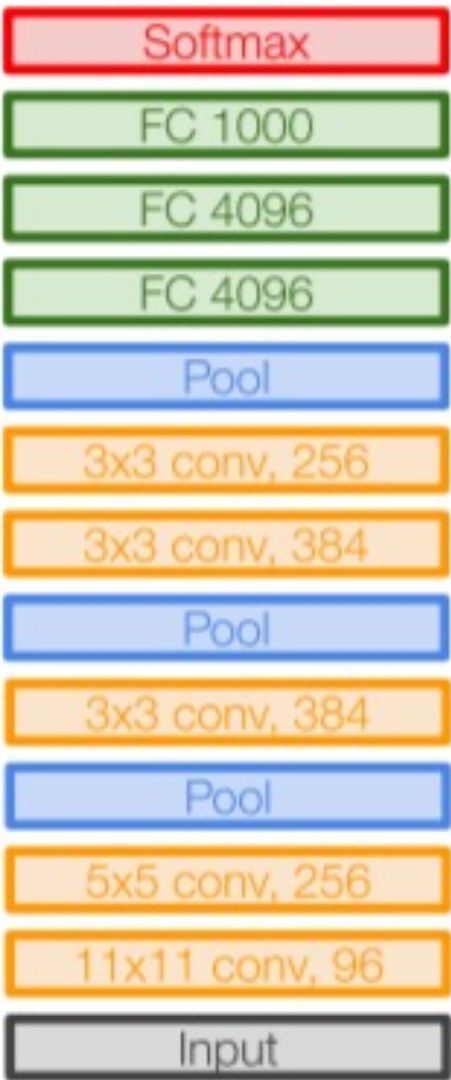
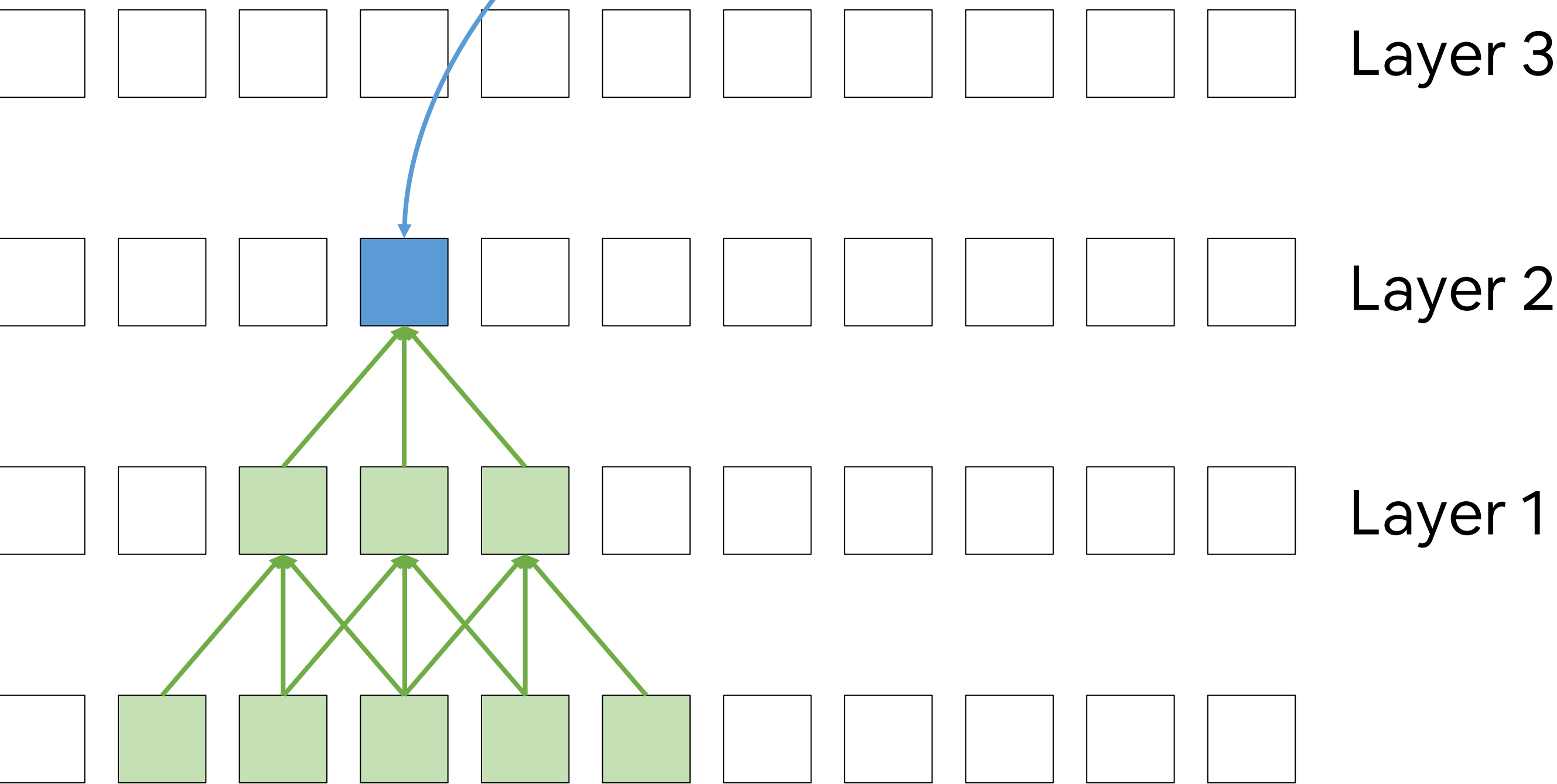
VGG16

VGG19

VGG

Consider a stack of three 3x3 conv layers.

Which is the receptive field of this hidden unit?



AlexNet



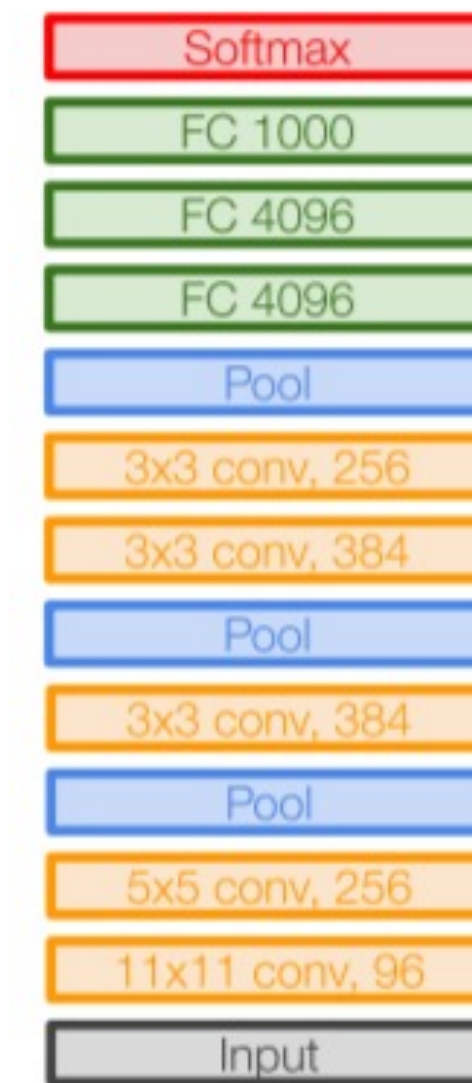
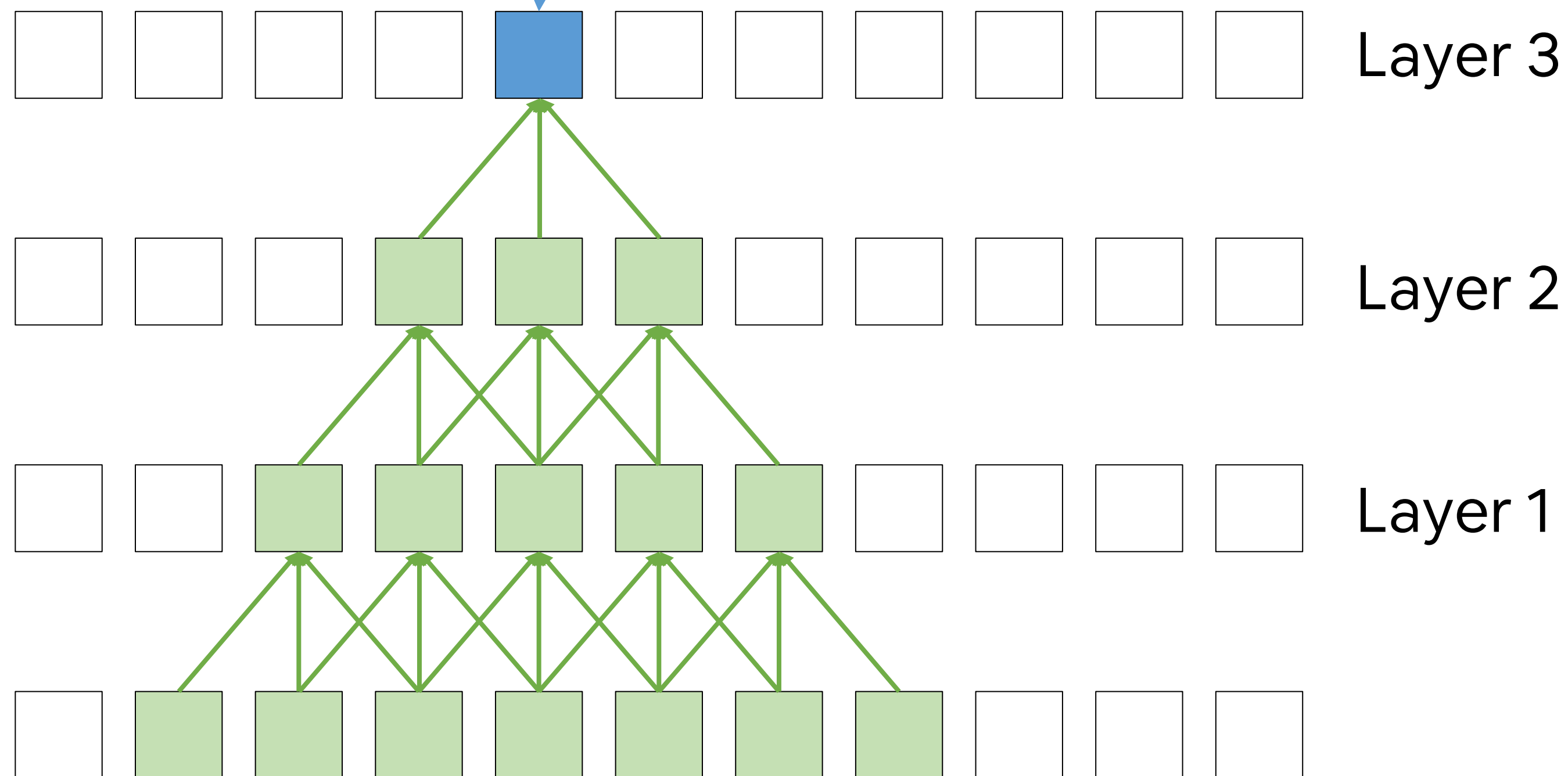
VGG16

VGG19

VGG

Consider a stack of three 3x3 conv layers.

Which is the receptive field of this hidden unit?



AlexNet



VGG16

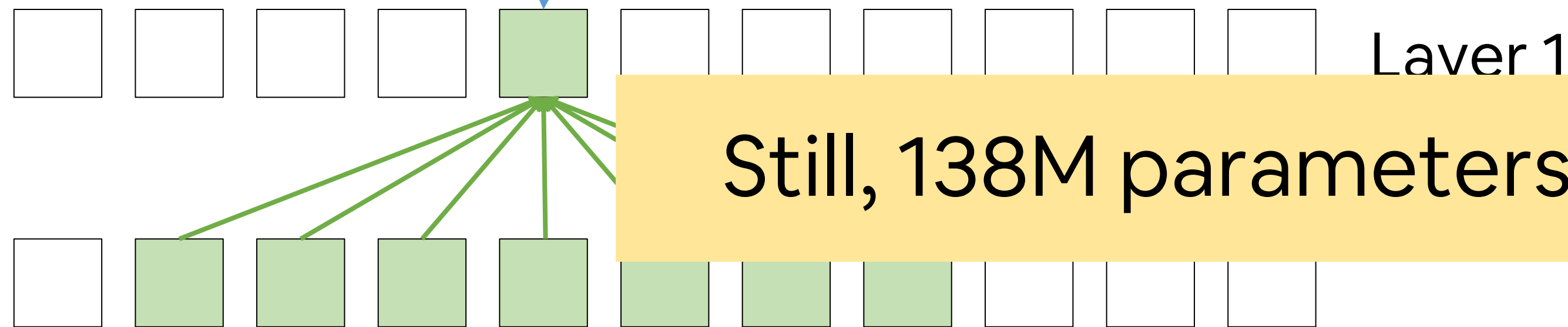
VGG19

VGG

Consider a stack of three 3x3 conv layers.

Which is the receptive field of this hidden unit?

Same as one layer with 7x7 conv filters.



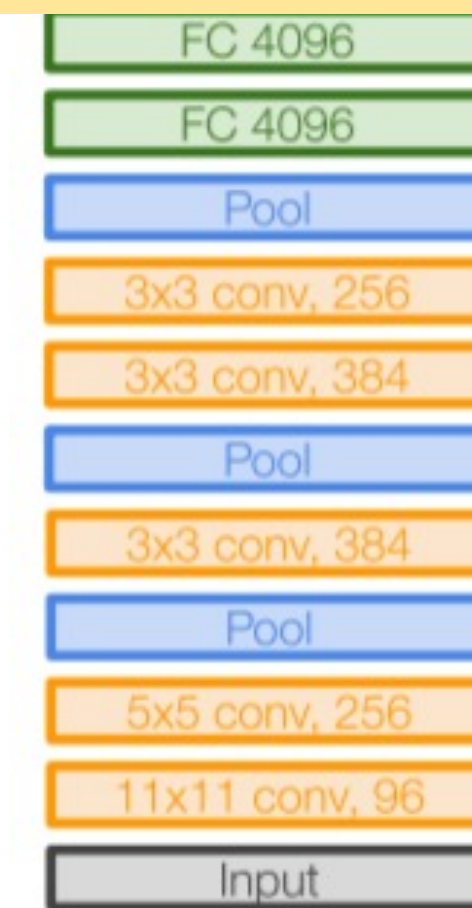
Still, 138M parameters for VGG16!

But three layers mean more non-linearities, i.e. more complex features...

... and fewer parameters!

C channels (e.g. C=3 for RGB images):

- 7x7 conv has $7^2 \times C = 147$ parameters
- 3 layers of 3x3 conv have $3 \times (3^2 \times C) = 81$ parameters



AlexNet



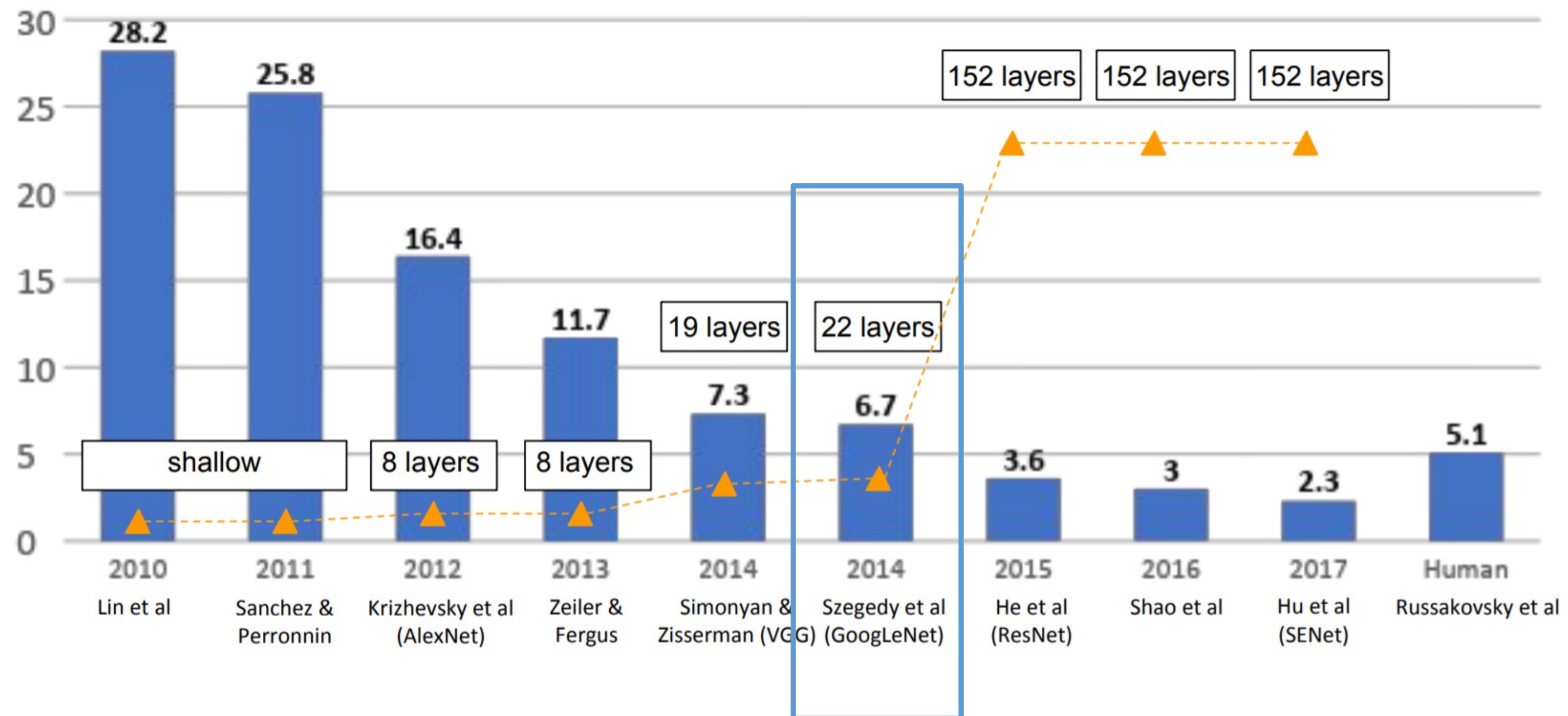
VGG16

VGG19

An overview on the most famous architectures

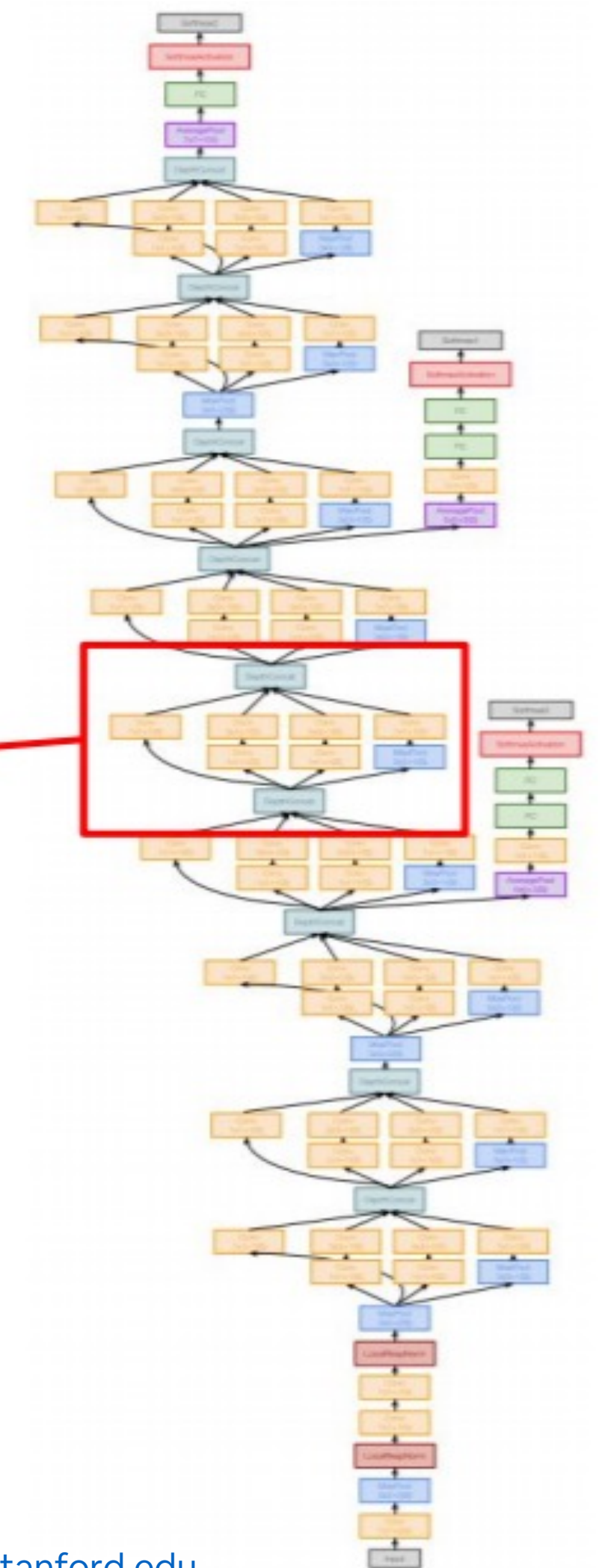
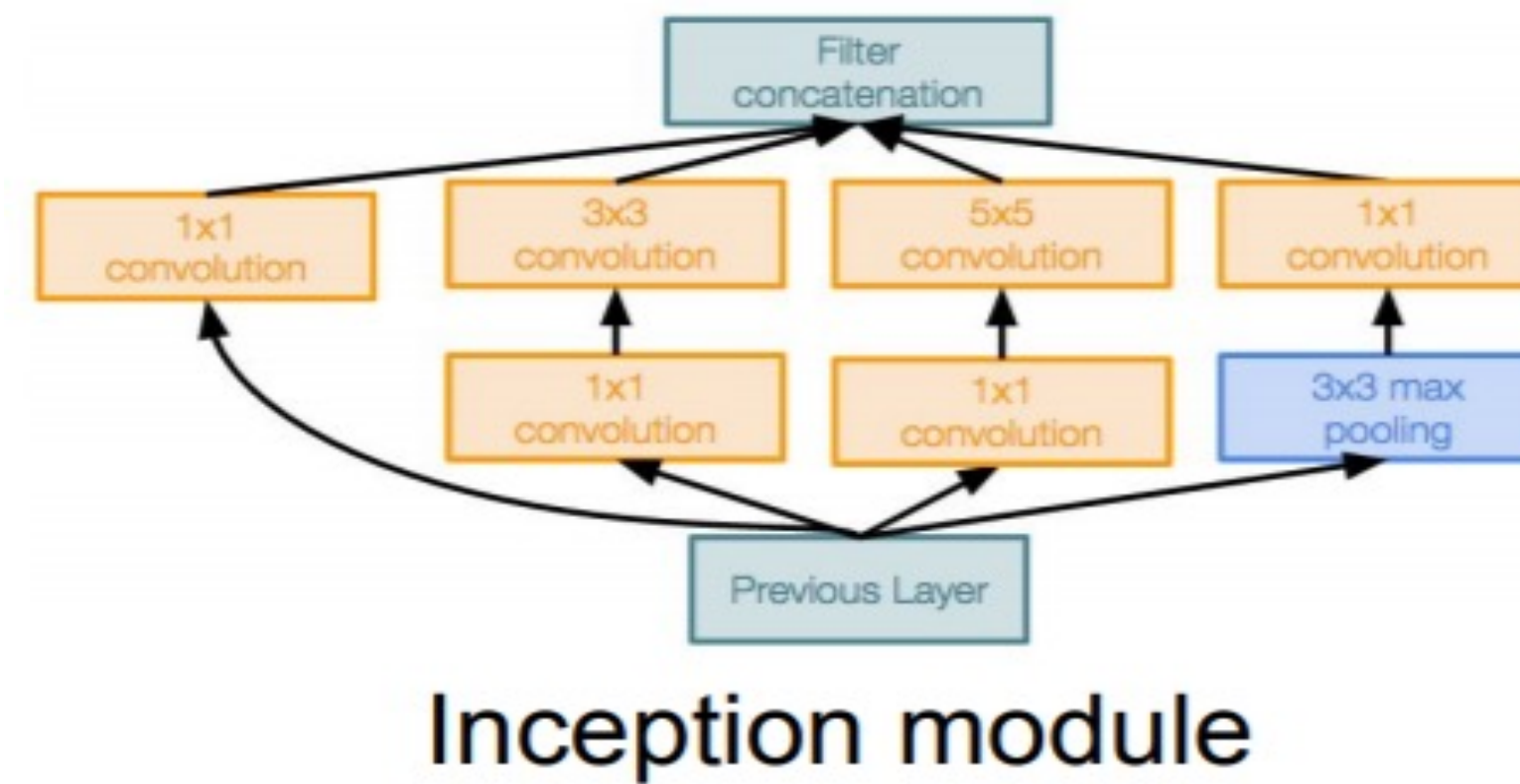
Imagenet – visual recognition challenge with 1000 classes.

Winners:



GoogLeNet

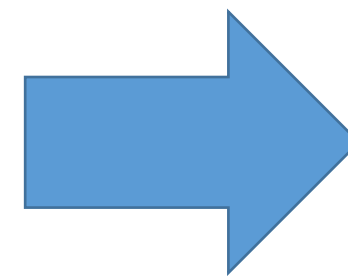
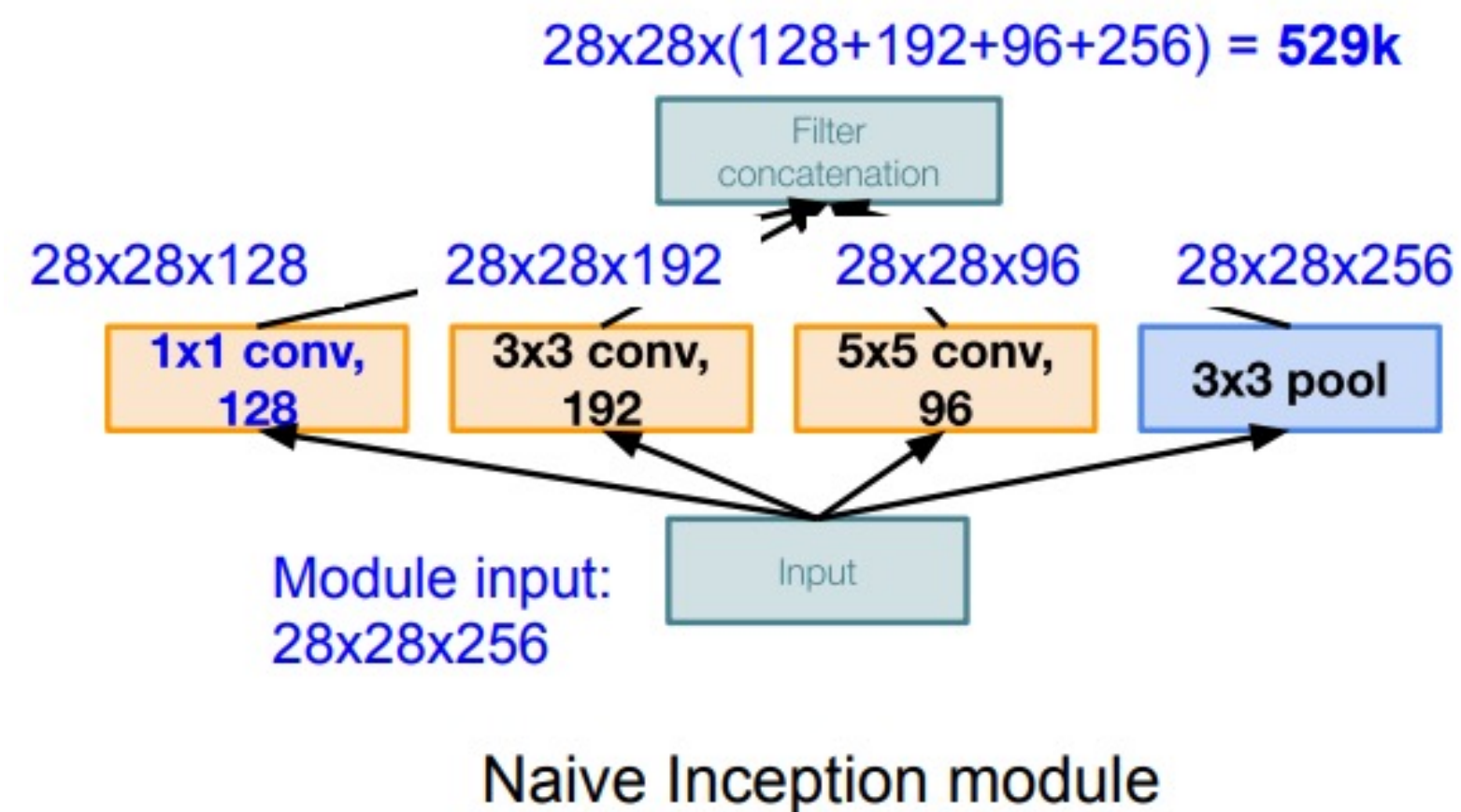
Inception + GoogLeNet (2015)– introduces parallel conv blocks (inception)



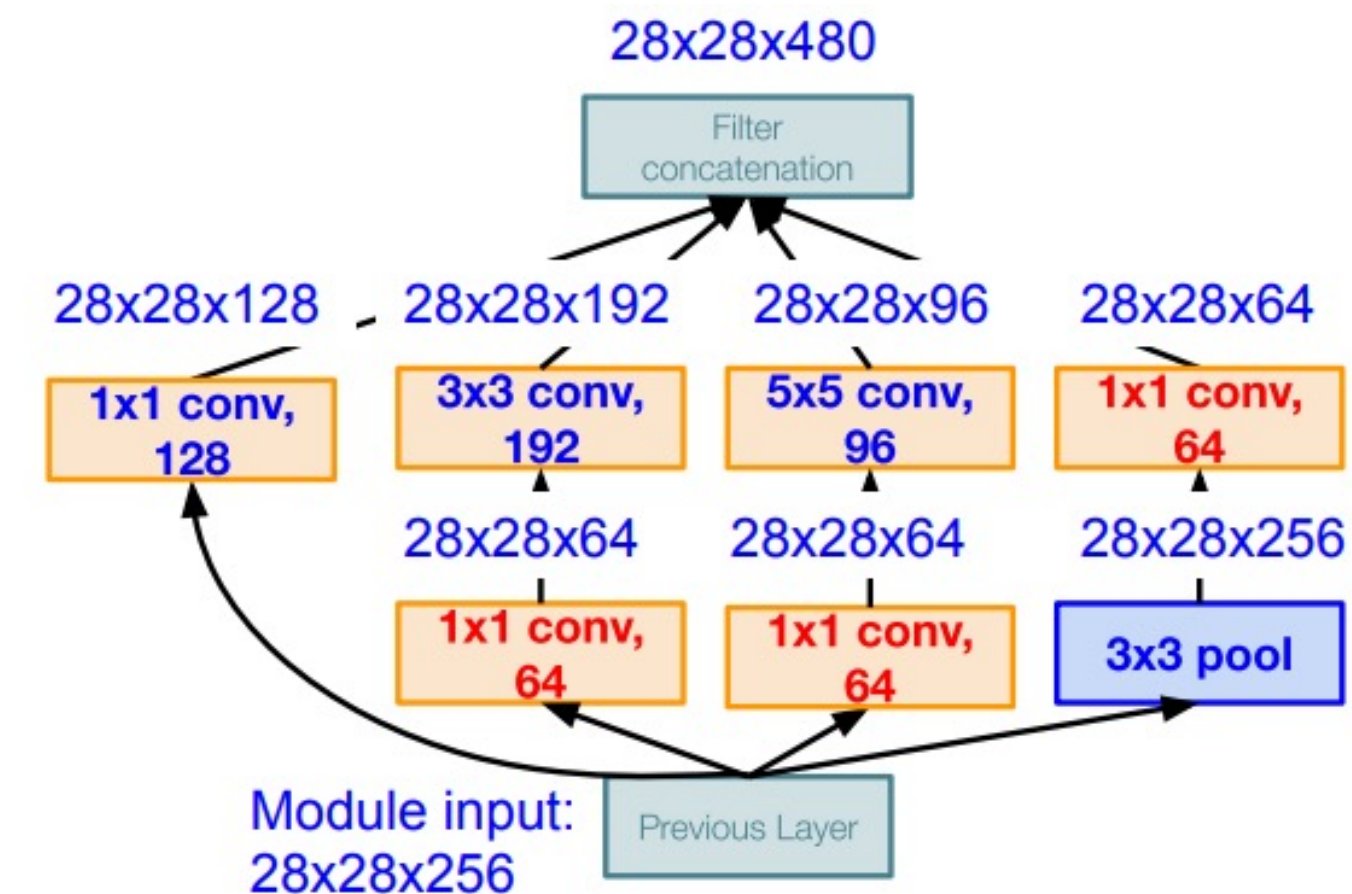
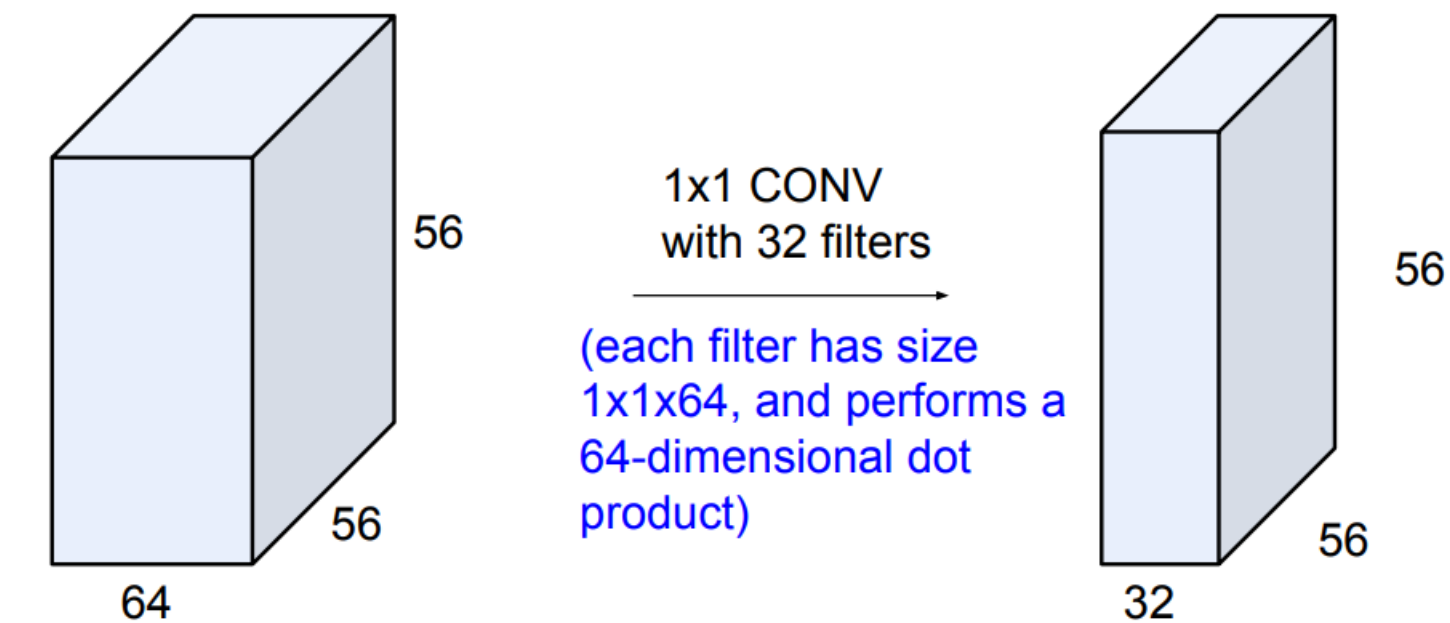
GoogLeNet

Inception + GoogLeNet (2015)– introduces parallel conv blocks (inception)

Cleverly uses 1x1 convolutions



Preserves spatial dimensions, but reduces depth! Feature maps (depth) are projected to lower dimension



Inception module with dimension reduction

GoogLeNet

Inception + GoogLeNet (2015)– introduces parallel conv blocks (inception)

Cleverly

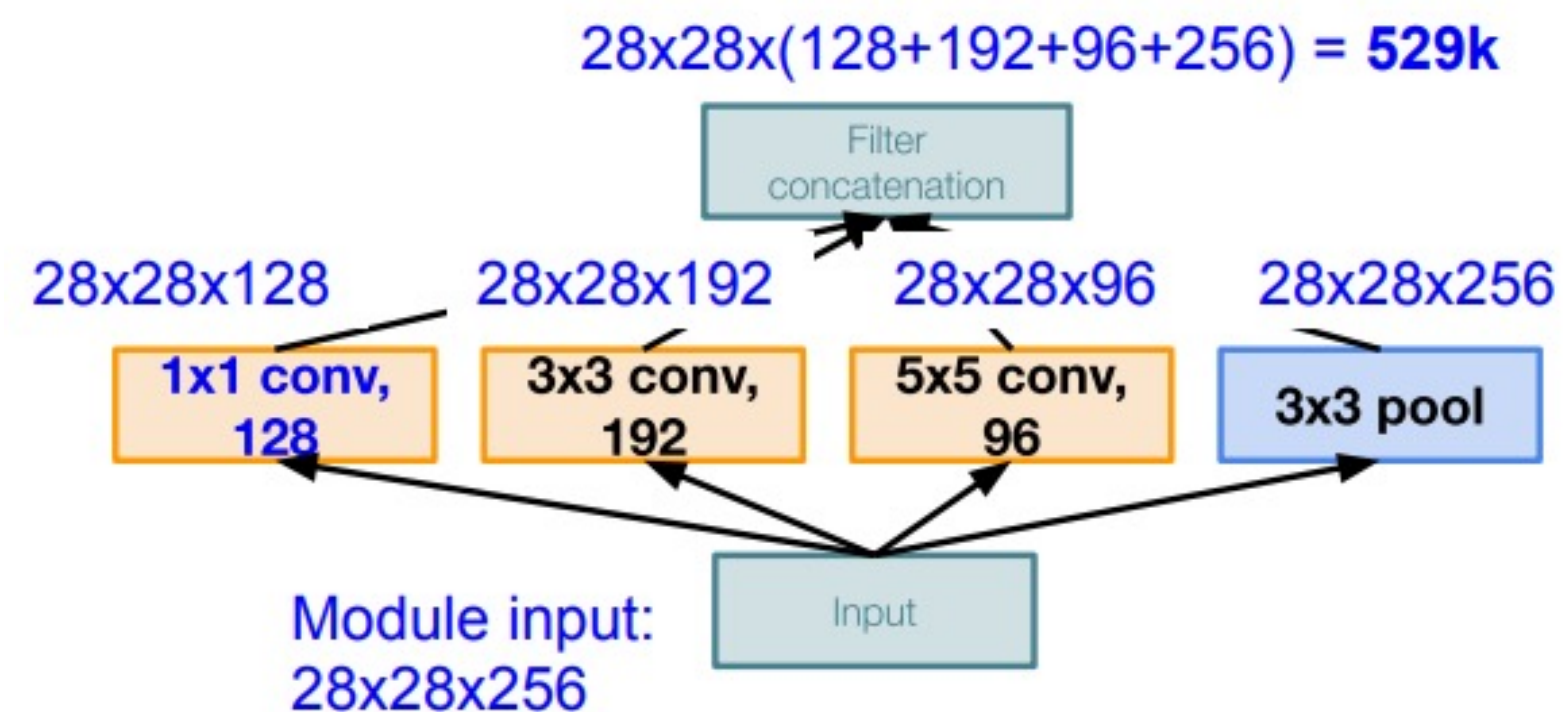
Conv Ops:

[1x1 conv, 128] 28x28x128x1x1x256

[3x3 conv, 192] 28x28x192x3x3x256

[5x5 conv, 96] 28x28x96x5x5x256

Total: 854M ops



Naive Inception module

Preserves spatial dimensions, but reduces depth! Feature maps (depth) are projected to lower dimension

Conv Ops:

[1x1 conv, 64] 28x28x64x1x1x256

[1x1 conv, 64] 28x28x64x1x1x256

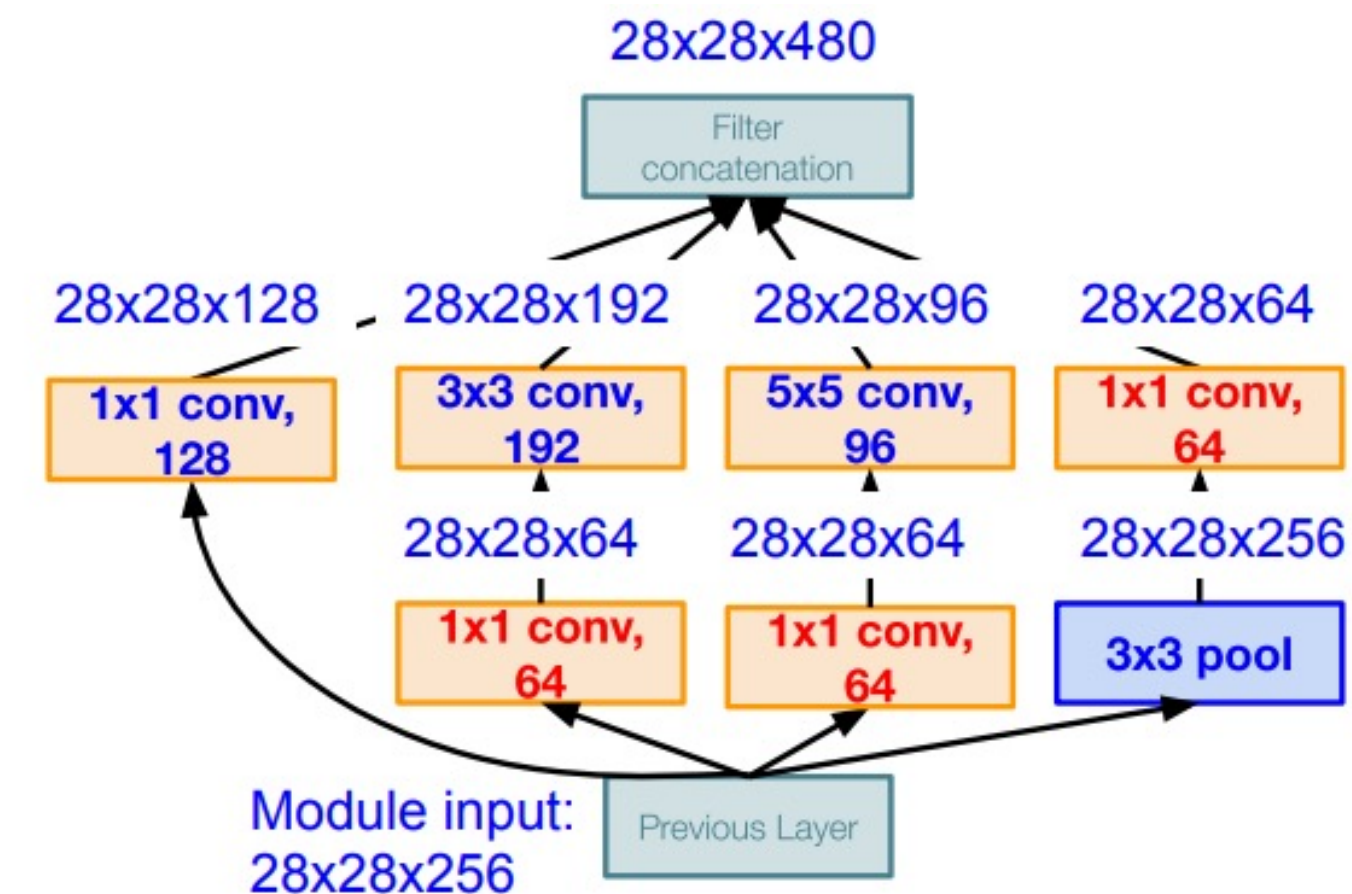
[1x1 conv, 128] 28x28x128x1x1x256

[3x3 conv, 192] 28x28x192x3x3x64

[5x5 conv, 96] 28x28x96x5x5x64

[1x1 conv, 64] 28x28x64x1x1x256

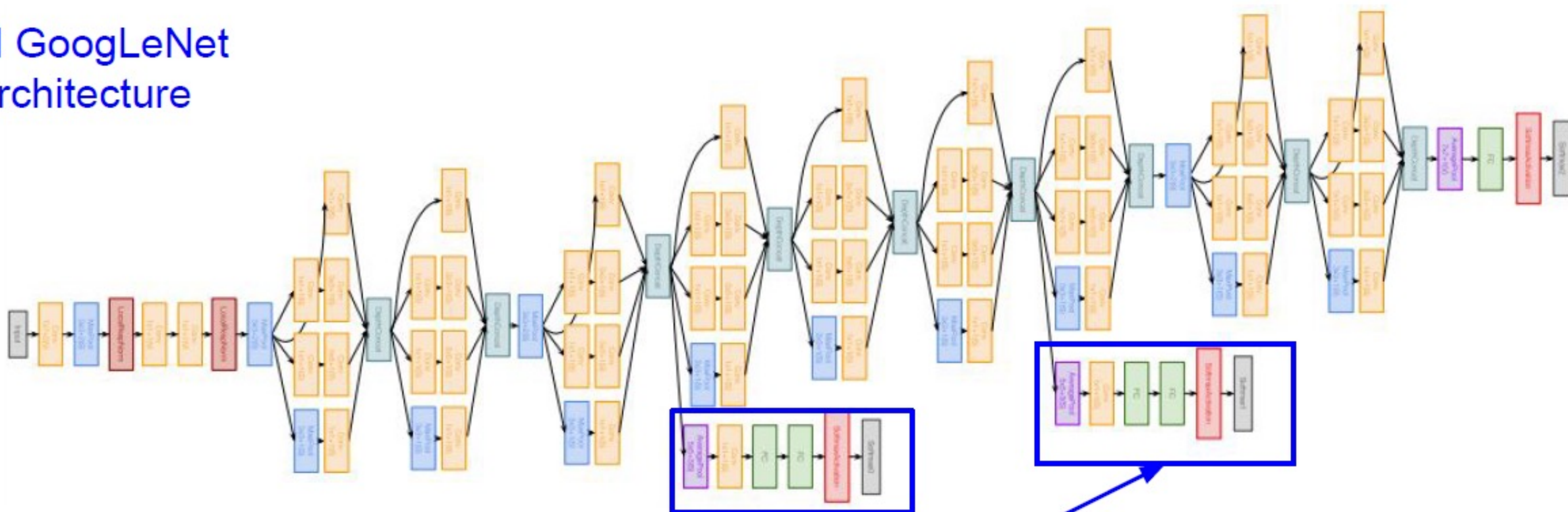
Total: 358M ops



Inception module with dimension reduction

GoogLeNet

Full GoogLeNet architecture

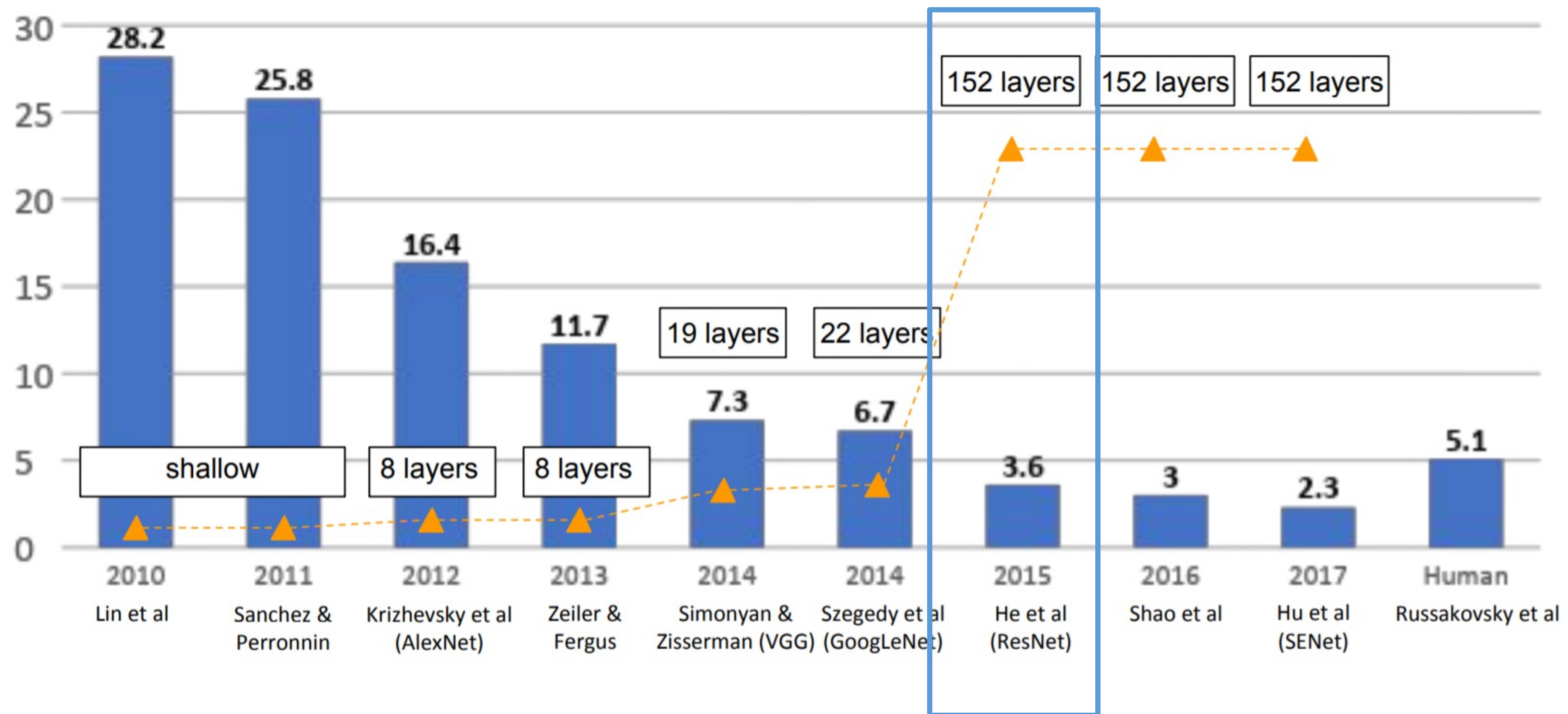


Auxiliary classification outputs to inject additional gradient at lower layers
(AvgPool-1x1Conv-FC-FC-Softmax)

An overview on the most famous architectures

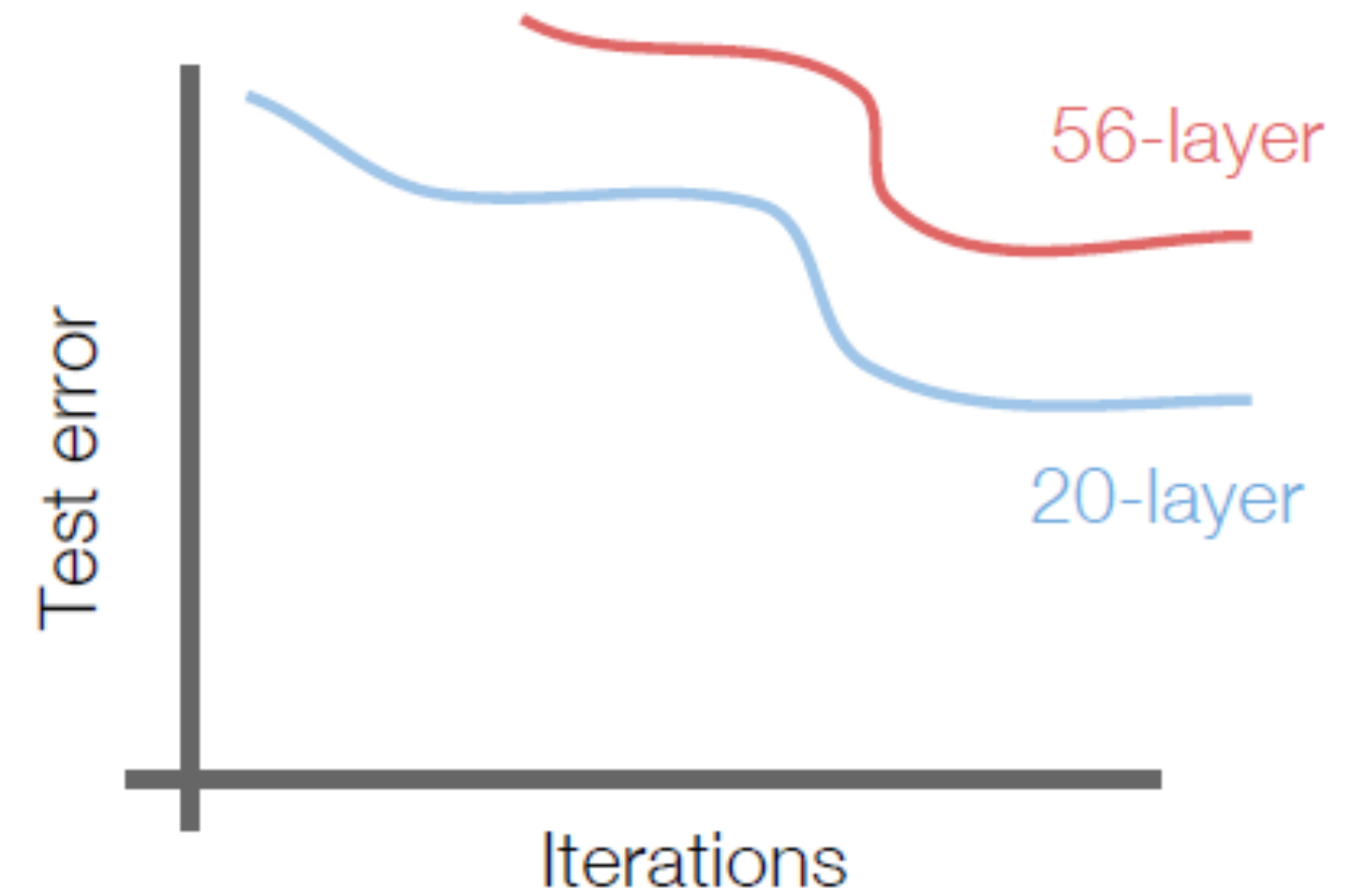
Imagenet – visual recognition challenge with 1000 classes.

Winners:



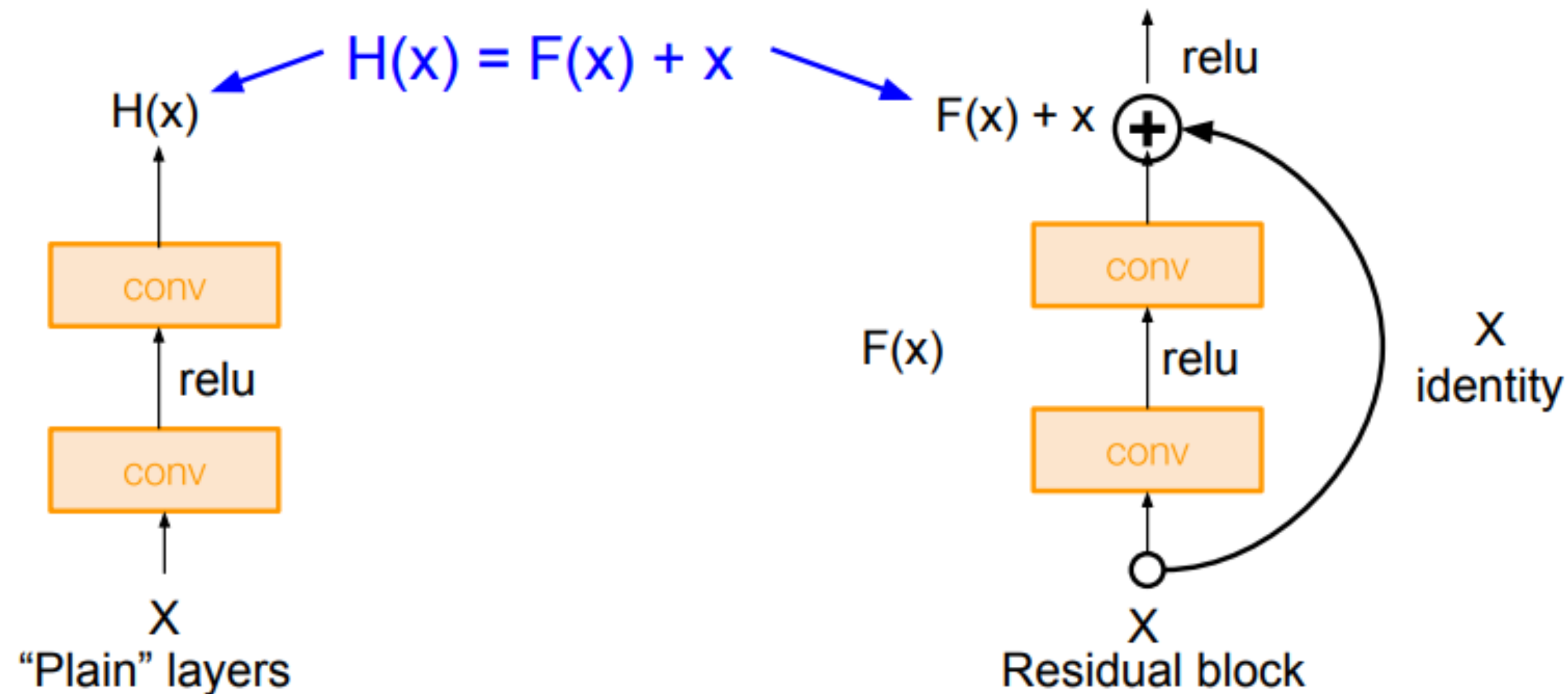
ResNet

Why not stacking more and more layers?

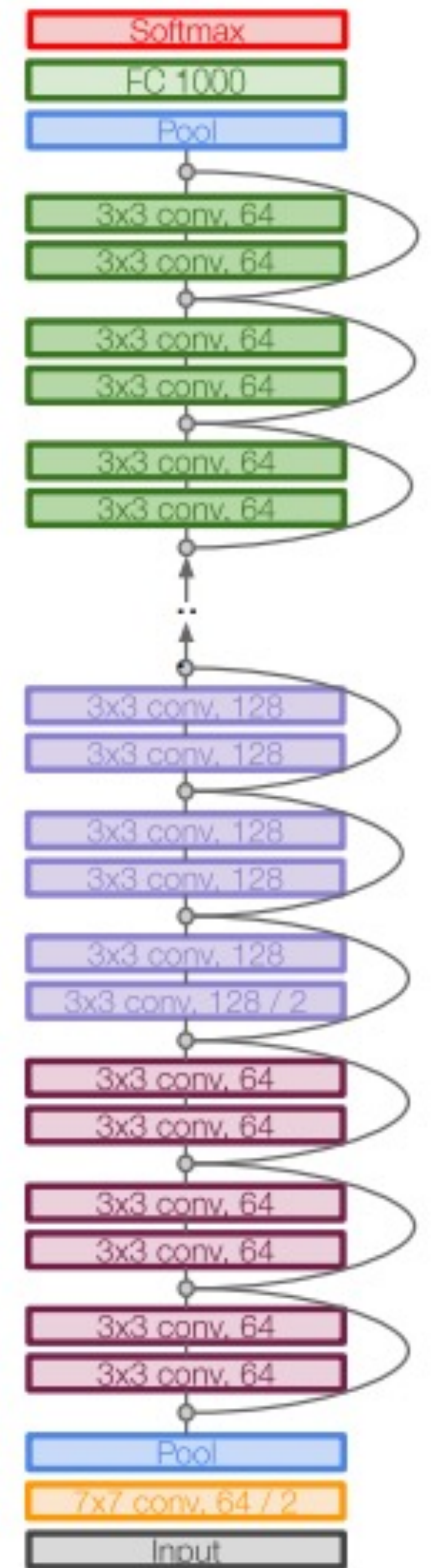


ResNet

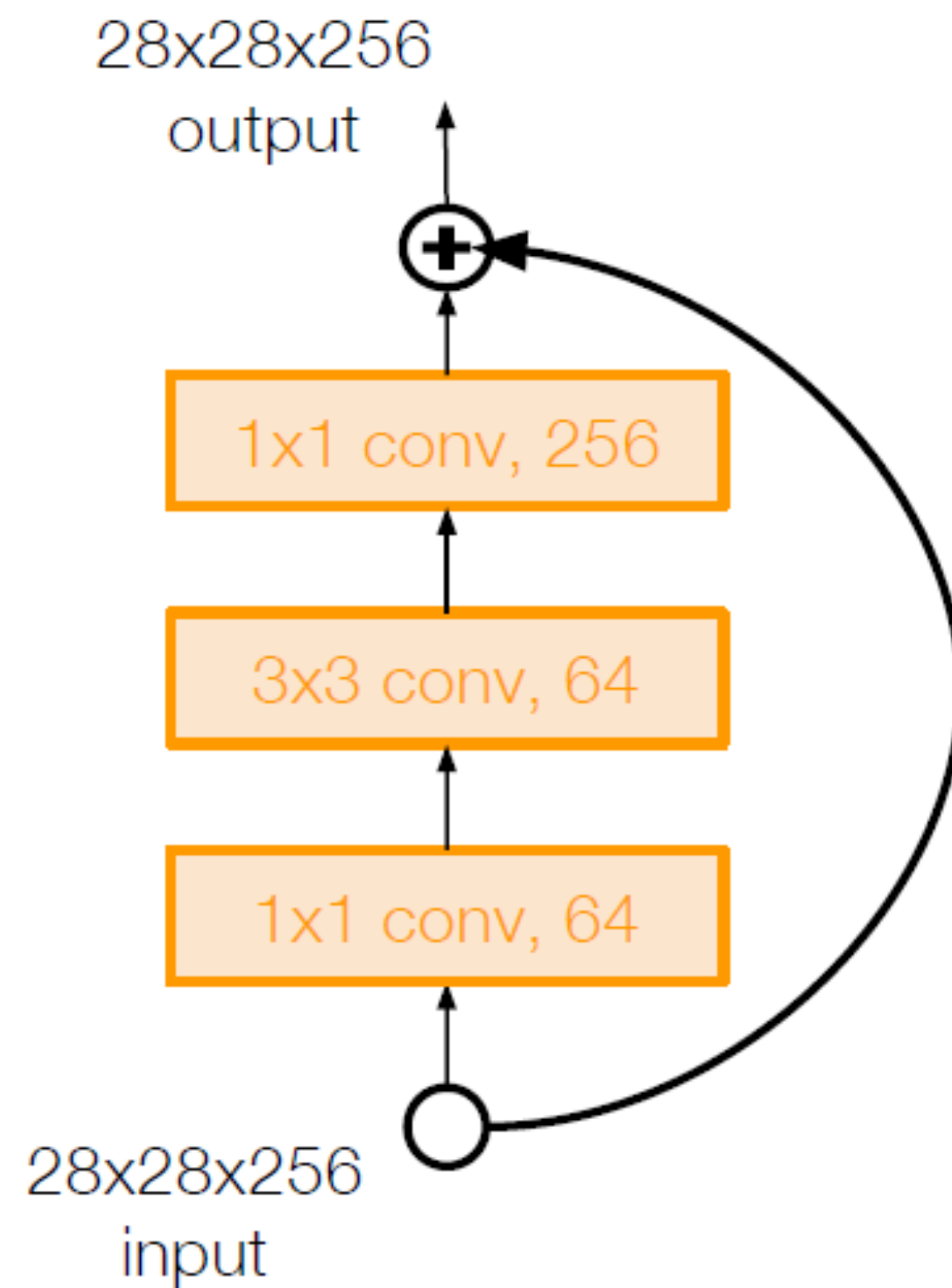
ResNet (2015) – very deep model (152 layer) with shortcut connections



Use layers to fit residual $F(x) = H(x) - x$ instead of $H(x)$ directly



ResNet



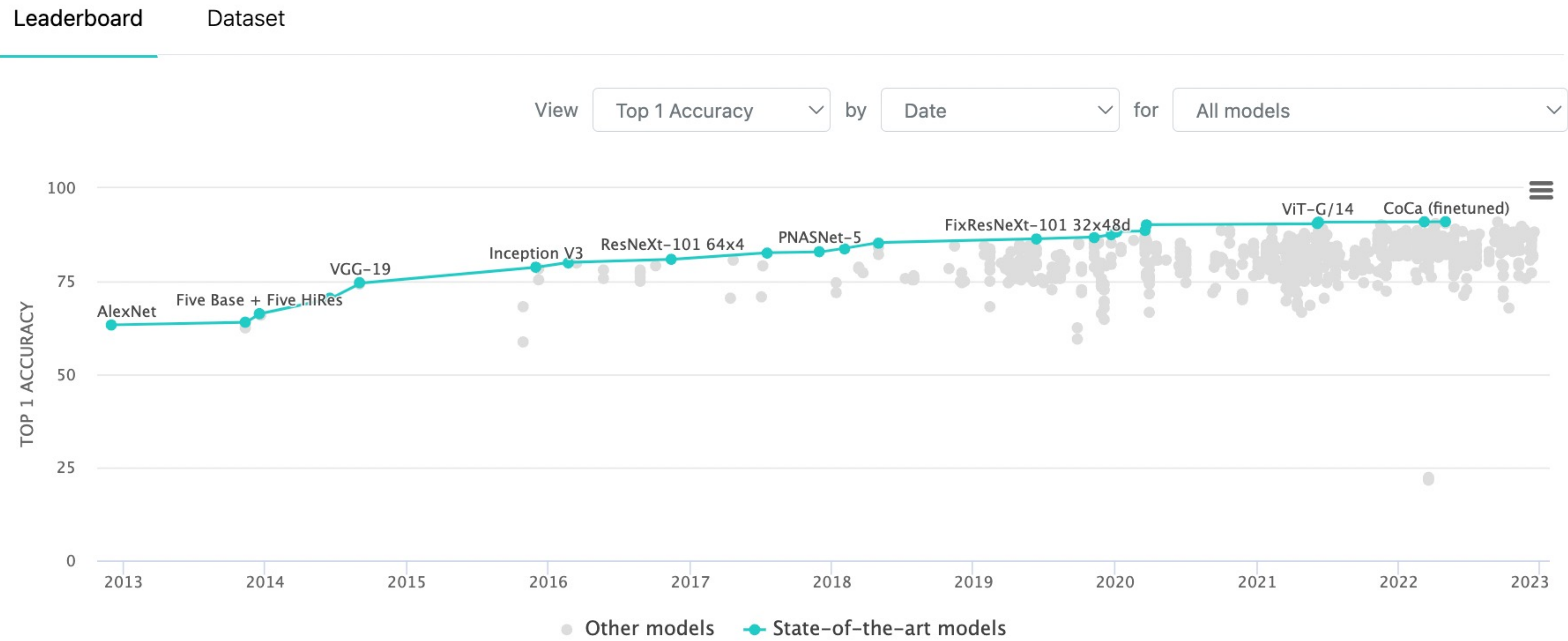
For ResNet with
more than 50 layers

ResNet training:

- Batch Normalization after every CONV layer
- Xavier/2 initialization from He et al.
- SGD + Momentum (0.9)
- Learning rate: 0.1, divided by 10 when validation error plateaus
- Mini-batch size 256
- Weight decay of $1e-5$
- No dropout used

State of the art is always on the move...

Image Classification on ImageNet



Is not all about accuracy... EfficientNet

Tan, M., & Le, Q. (2019, May).
Efficientnet: Rethinking model
scaling for convolutional neural
networks. In *International
Conference on Machine
Learning* (pp. 6105-6114). PMLR.

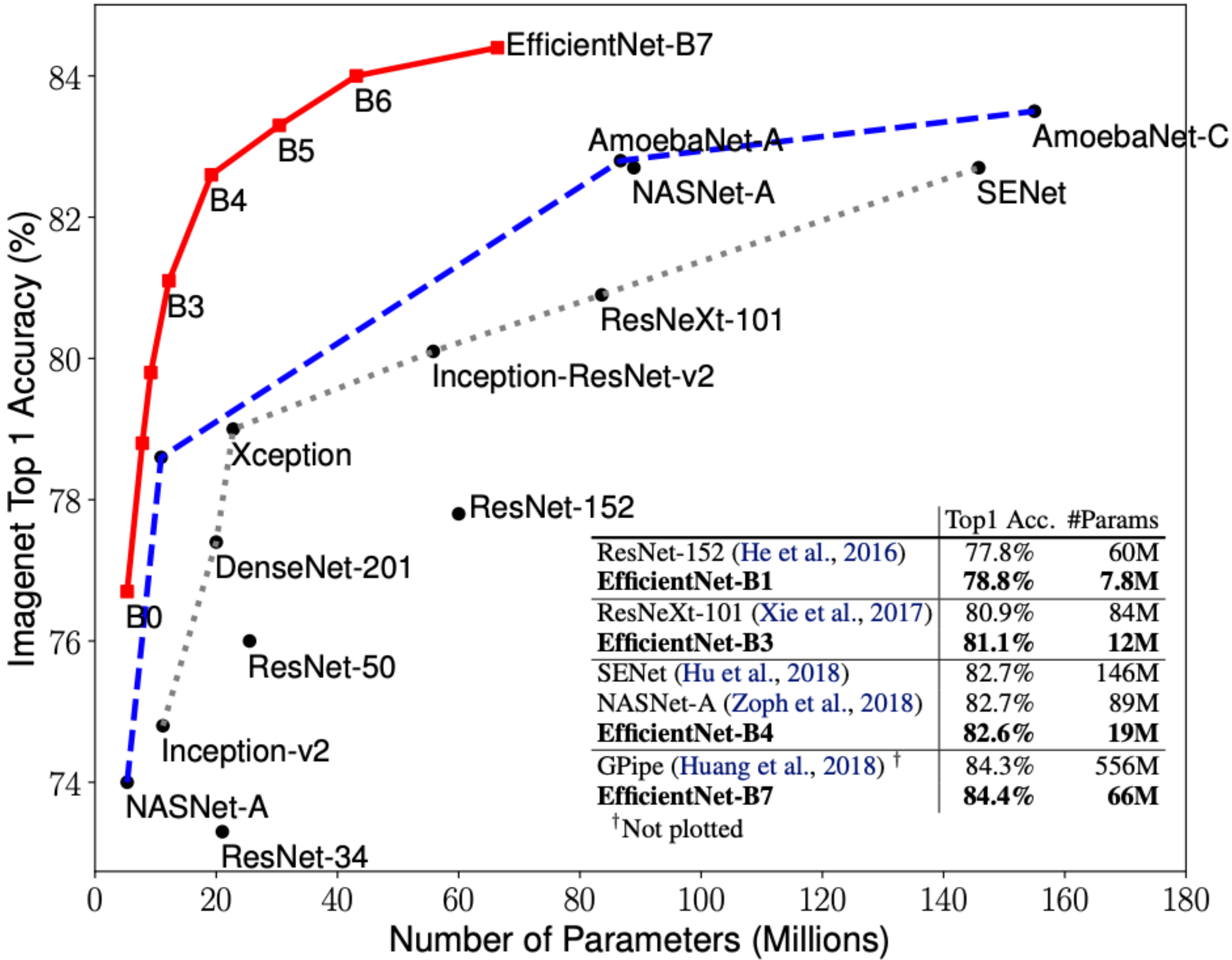
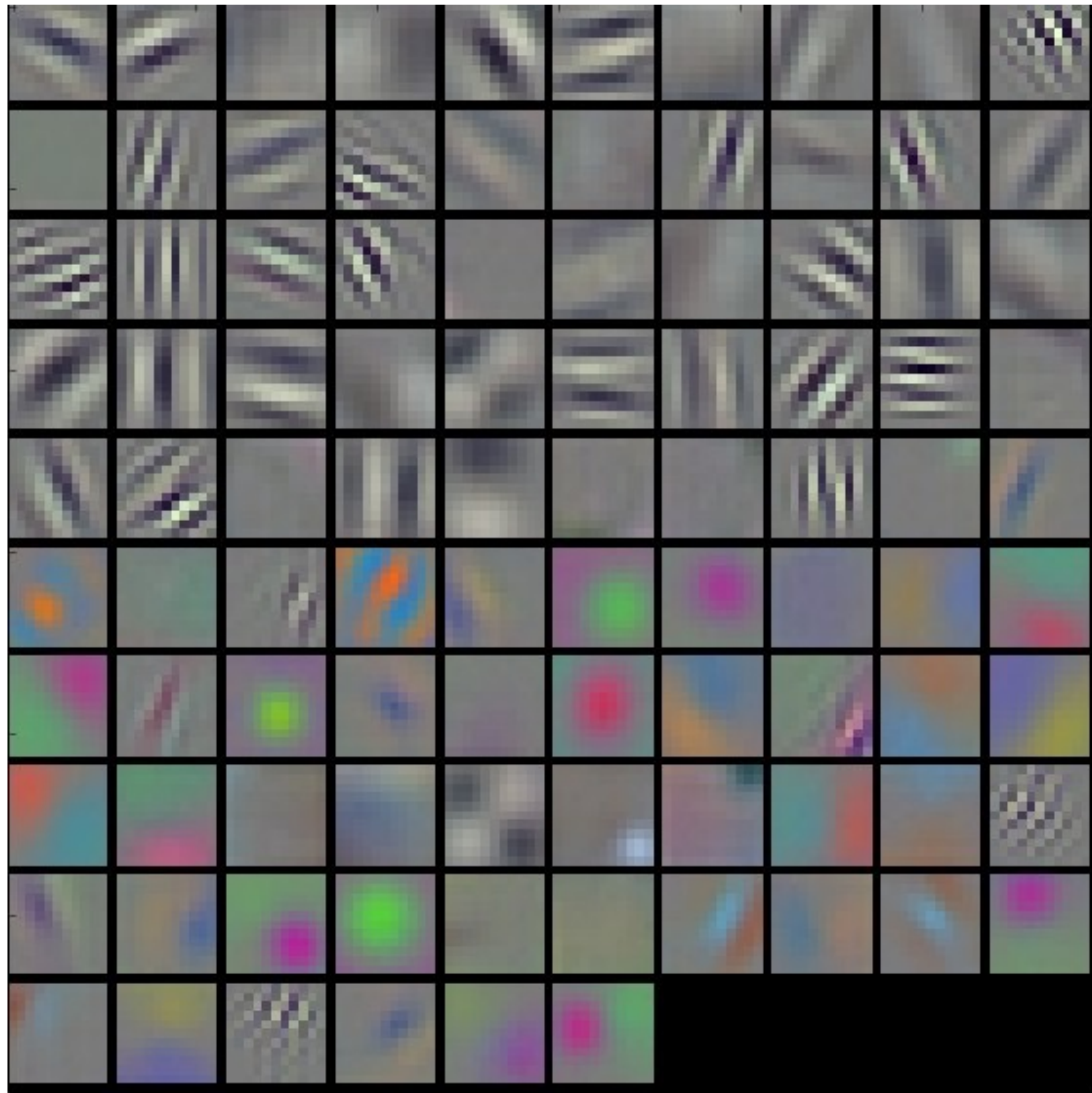


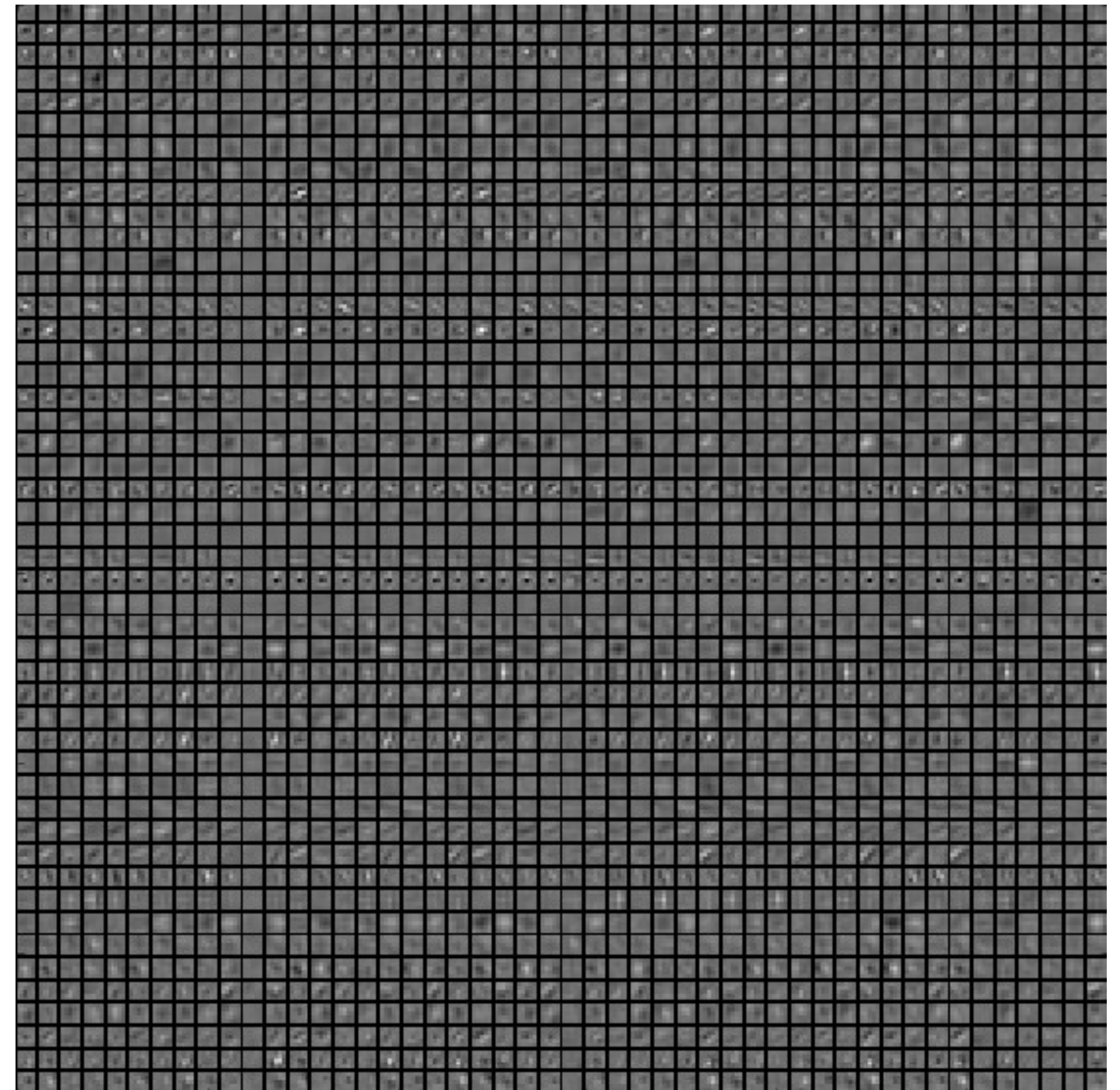
Figure 1. Model Size vs. ImageNet Accuracy. All numbers are for single-crop, single-model. Our EfficientNets significantly outperform other ConvNets. In particular, EfficientNet-B7 achieves new state-of-the-art 84.4% top-1 accuracy but being 8.4x smaller and 6.1x faster than GPipe. EfficientNet-B1 is 7.6x smaller and 5.7x faster than ResNet-152. Details are in Table 2 and 4.

What do CCN see?

What do CNN see? Visualize the layers



AlexNet 1CONV layer

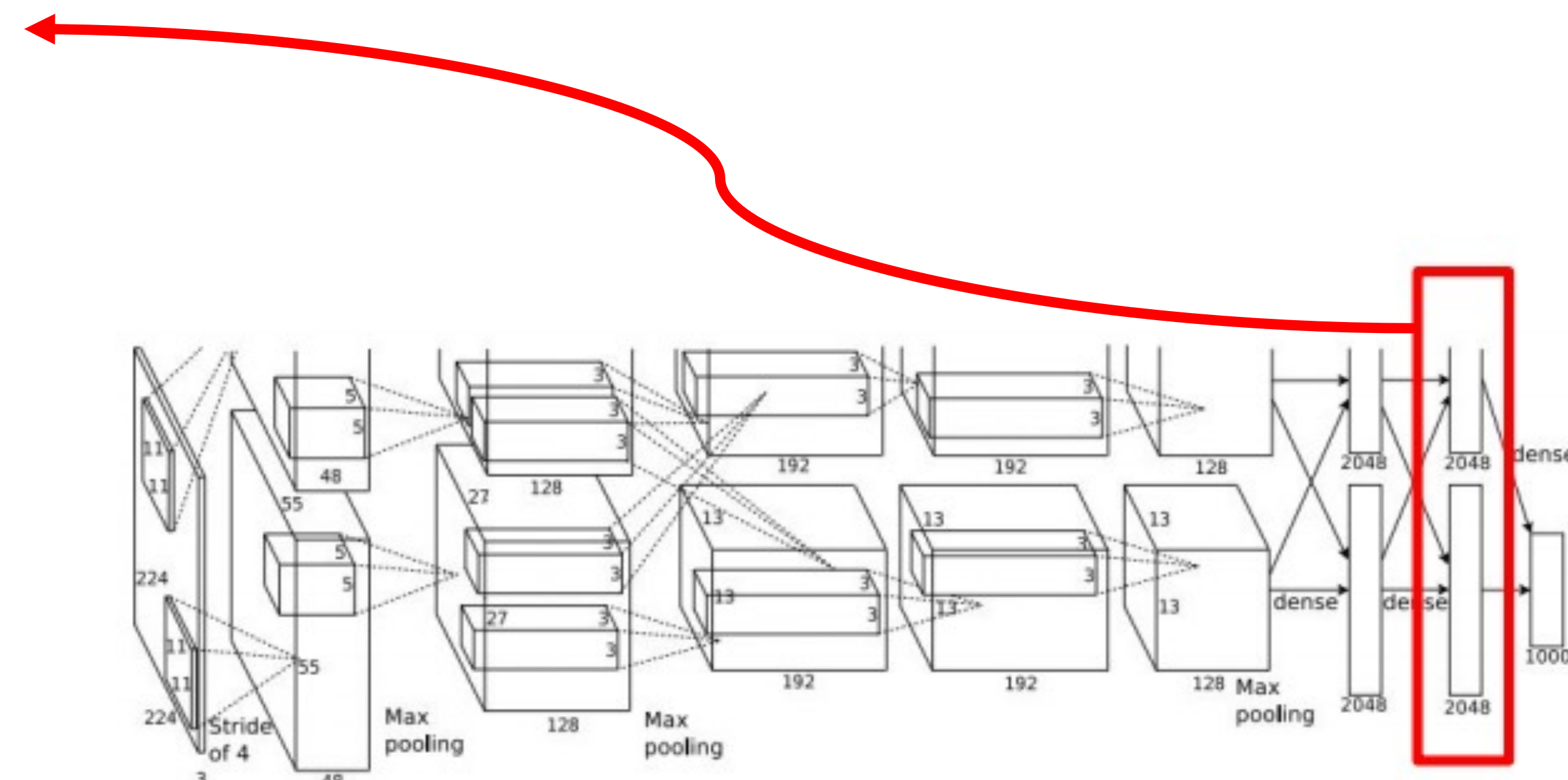
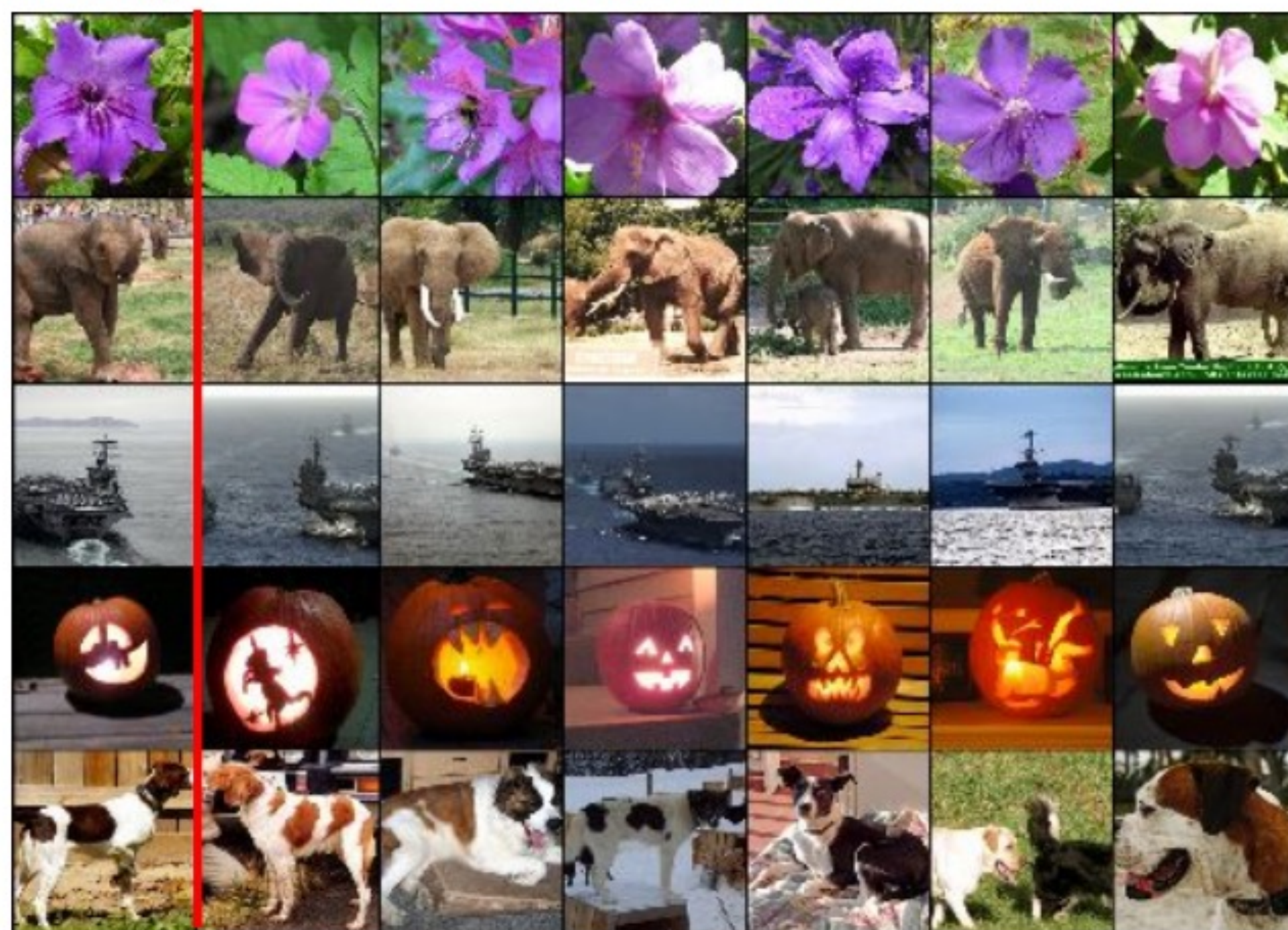


AlexNet 2CONV layer

What do CNN see? Embedding space for features

We can consider k-nearest neighbors in embedding space for last FC layer:

Test image L2 Nearest neighbors in feature space



Krizhevsky et al, "ImageNet Classification with Deep Convolutional Neural Networks", NIPS 2012.

From: <http://cs231n.stanford.edu>

What do CNN see? Embedding space for features

We can plot final FC
embedding layer by means of
dimensionality reduction, e.g.
tSNE (more powerful than PCA)
or UMAP

Van der Maaten and Hinton, “Visualizing Data using t-SNE”, JMLR 2008

From: <http://cs231n.stanford.edu>

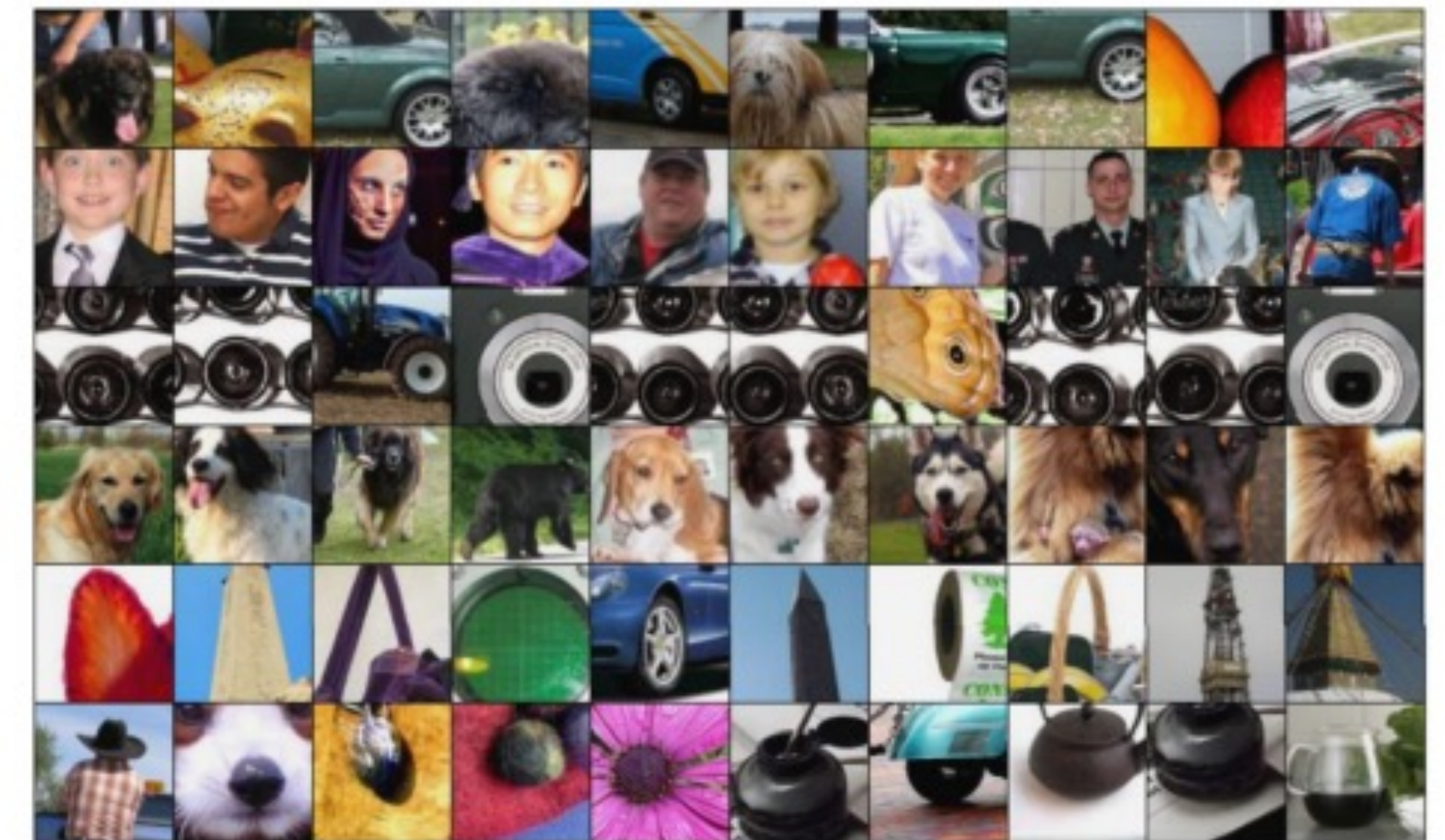


What do CNN see? Maximally activating neuros

We can compute maximally activating patches.

Run many images through the network, record values of chosen channel (e.g. channel 17/128 in conv5).

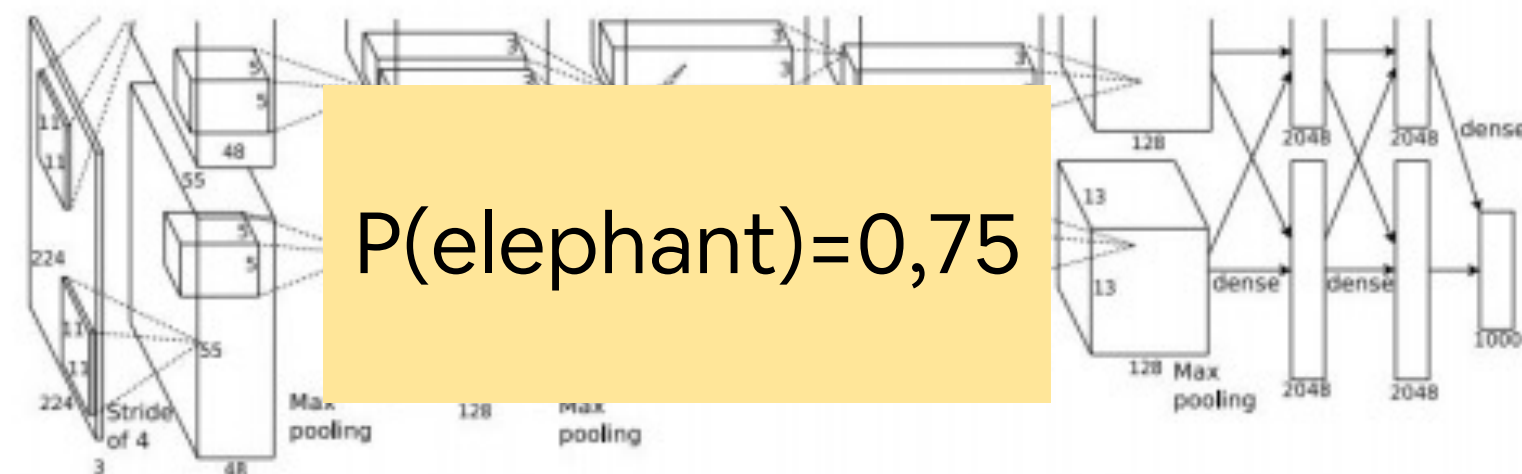
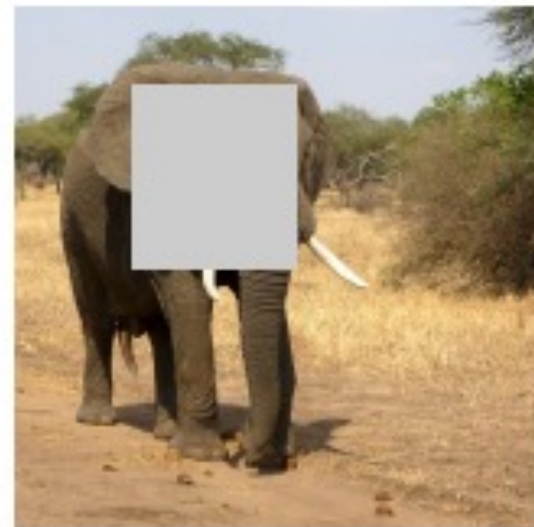
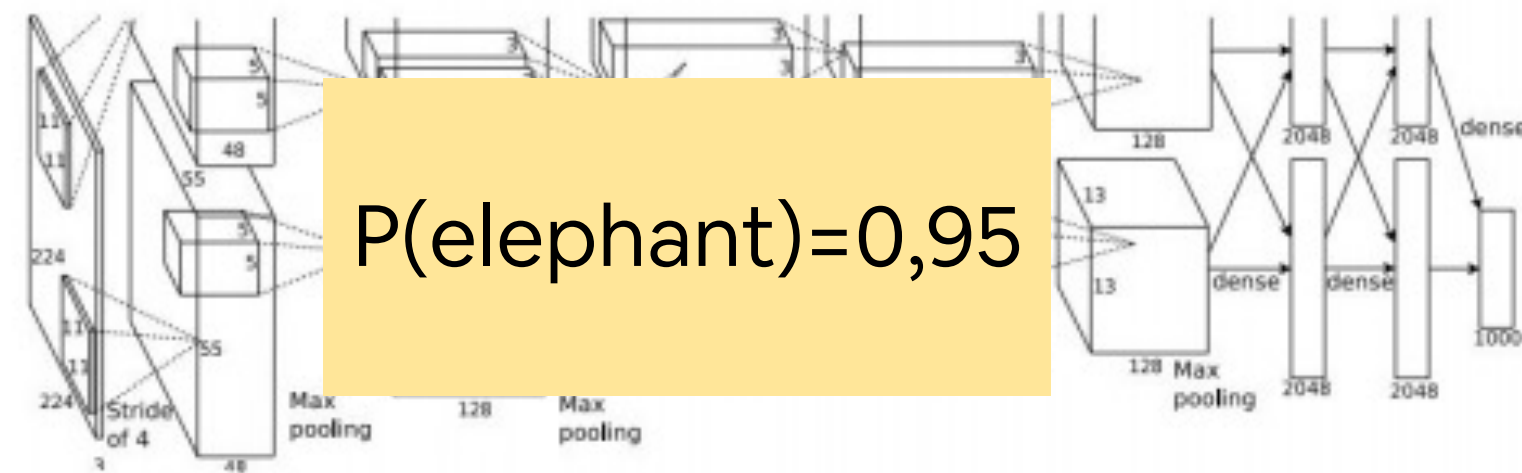
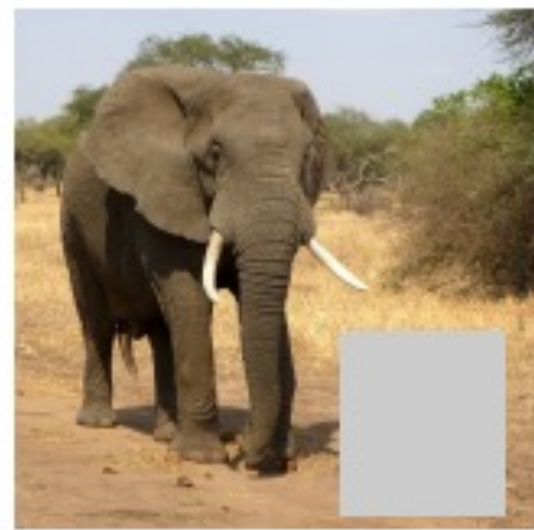
Visualize image patches that correspond to maximal activations.



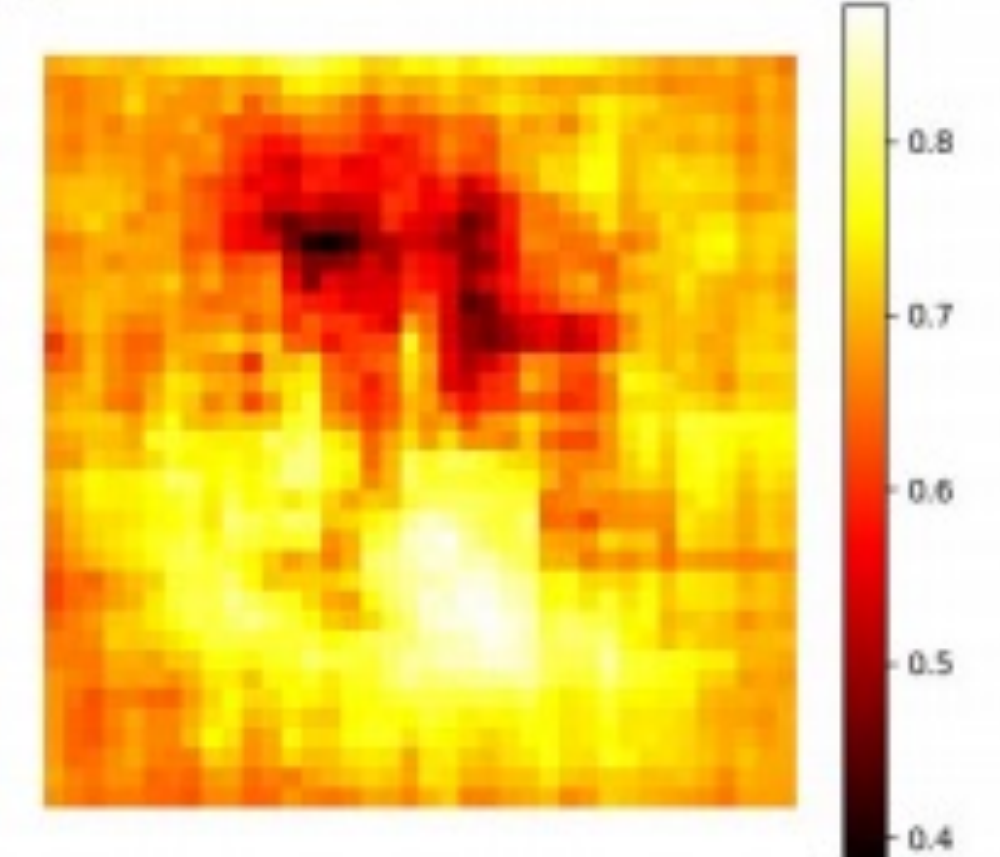
Springenberg et al, "Striving for Simplicity: The All Convolutional Net", ICLR Workshop 2015 Figure copyright Jost Tobias Springenberg, Alexey Dosovitskiy, Thomas Brox, Martin Riedmiller, 2015;

What do CNN see? Most relevant pixels

Saliency maps, e.g. by occlusion:



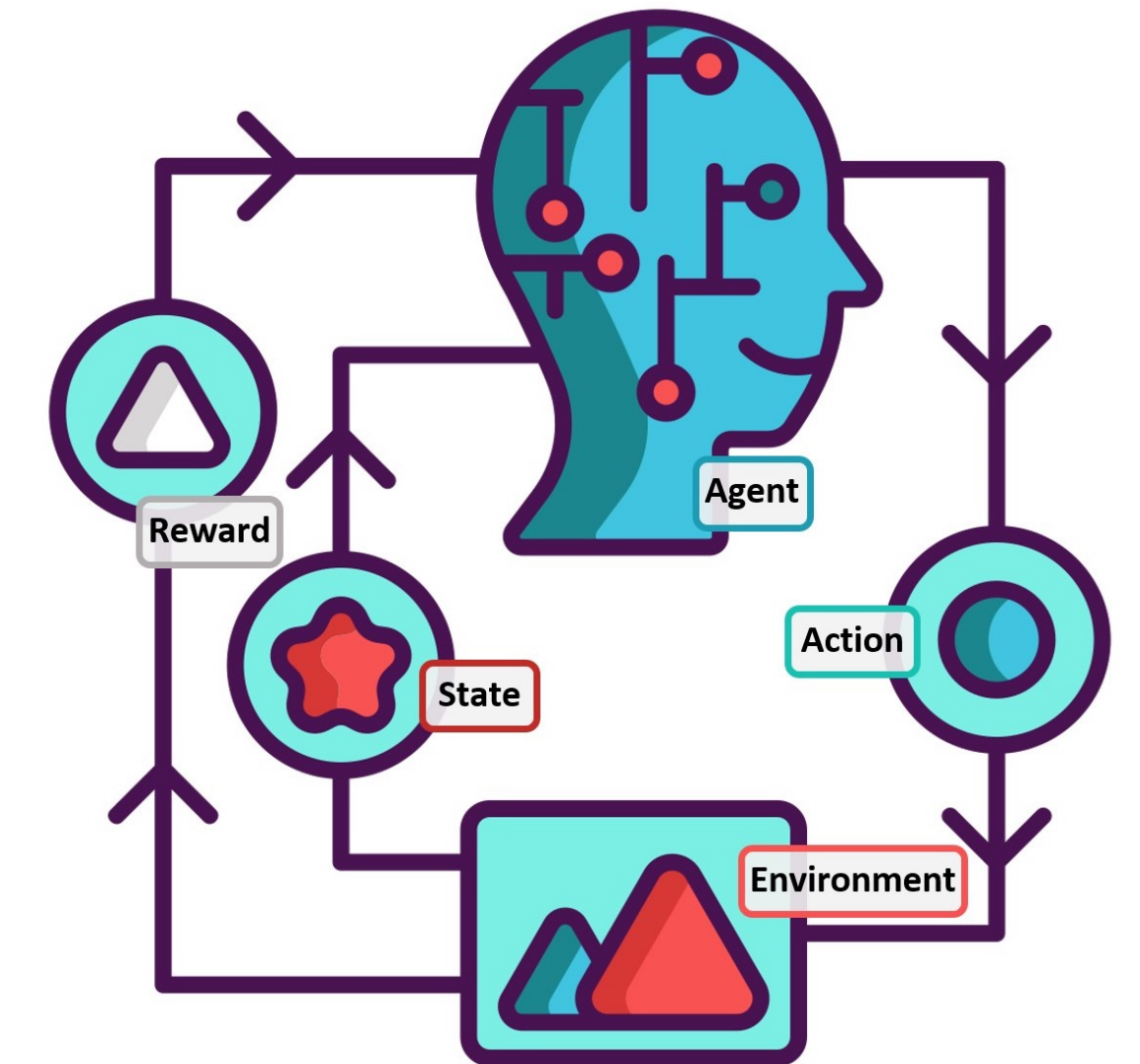
African elephant, *Loxodonta africana*



Zeiler and Fergus, "Visualizing and Understanding Convolutional Networks", ECCV 2014

From: <http://cs231n.stanford.edu>

Reinforcement Learning



**Thank you for your
attention!**

Gian Antonio Susto
Ruggero Carli

